



GNOMON[®]
DIGITAL

INTRODUCTION AUX RÉSEAUX DE NEURONES

Jiqiong QIU

Objectif du cours



- Le but de ce cours est l'introduction aux réseaux de neurones, nous allons aborder ensemble les principaux composants d'un réseau de neurone simple, ainsi que les différents types de réseaux suivant la problématique à résoudre.
- Le cours se décompose en une partie théorique, des lectures (Drive into deep learning), et Des TP sous *python* afin de mieux assimiler les différents concepts.
- Le cours se focalise essentiellement sur les aspects suivants :
 - *Introduction à AI et ML*
 - *Introduction aux modèles réseaux de neurones*

Evaluation



GNOMON[®]
DIGITAL

- 2 Q&A (50%)
- 2 TP noté (50%)

Qu'est-ce que c'est l'intelligence Artificielle

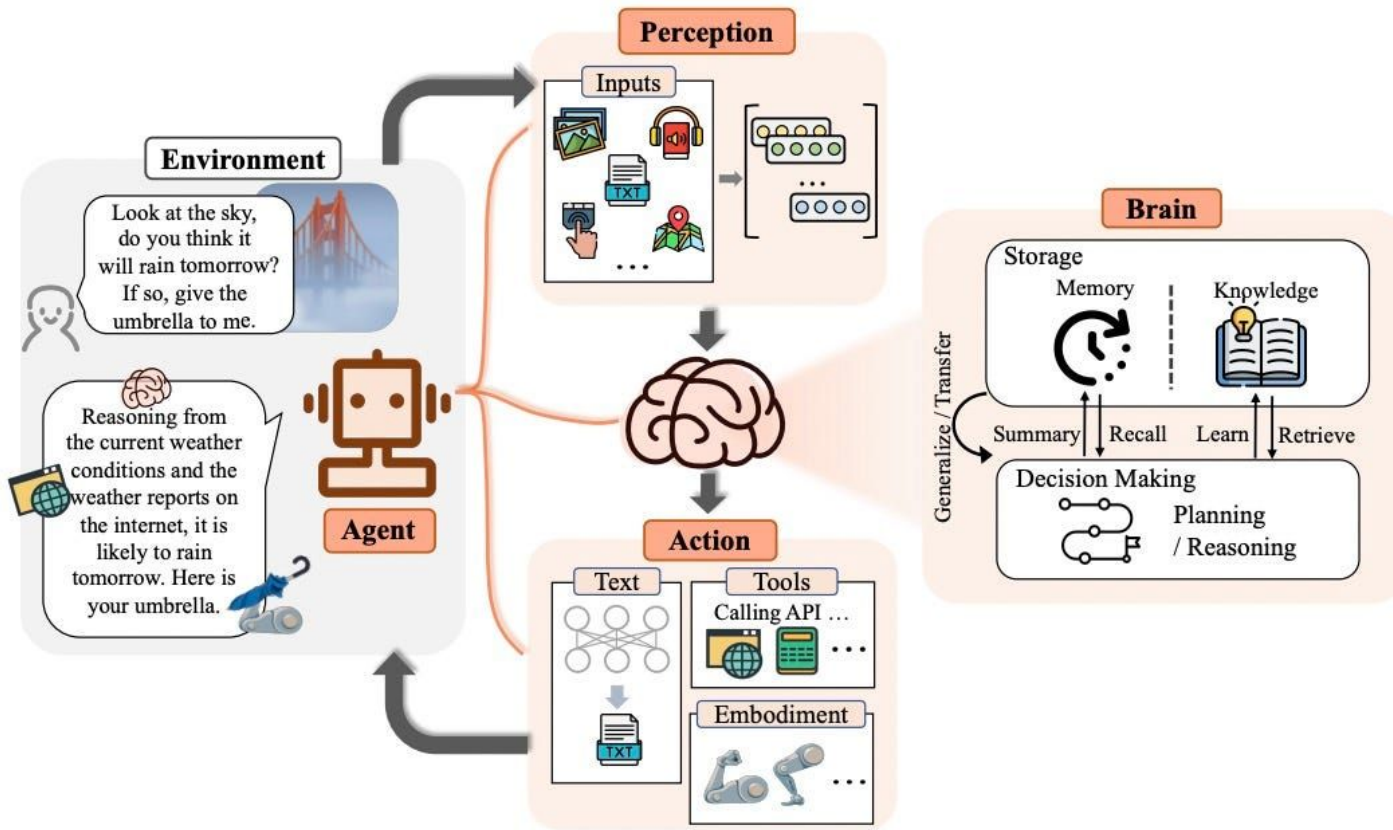


GNOMON[®]
DIGITAL



L'expérience de la chambre chinoise par John Searle

Qu'est-ce que c'est l'intelligence Artificielle

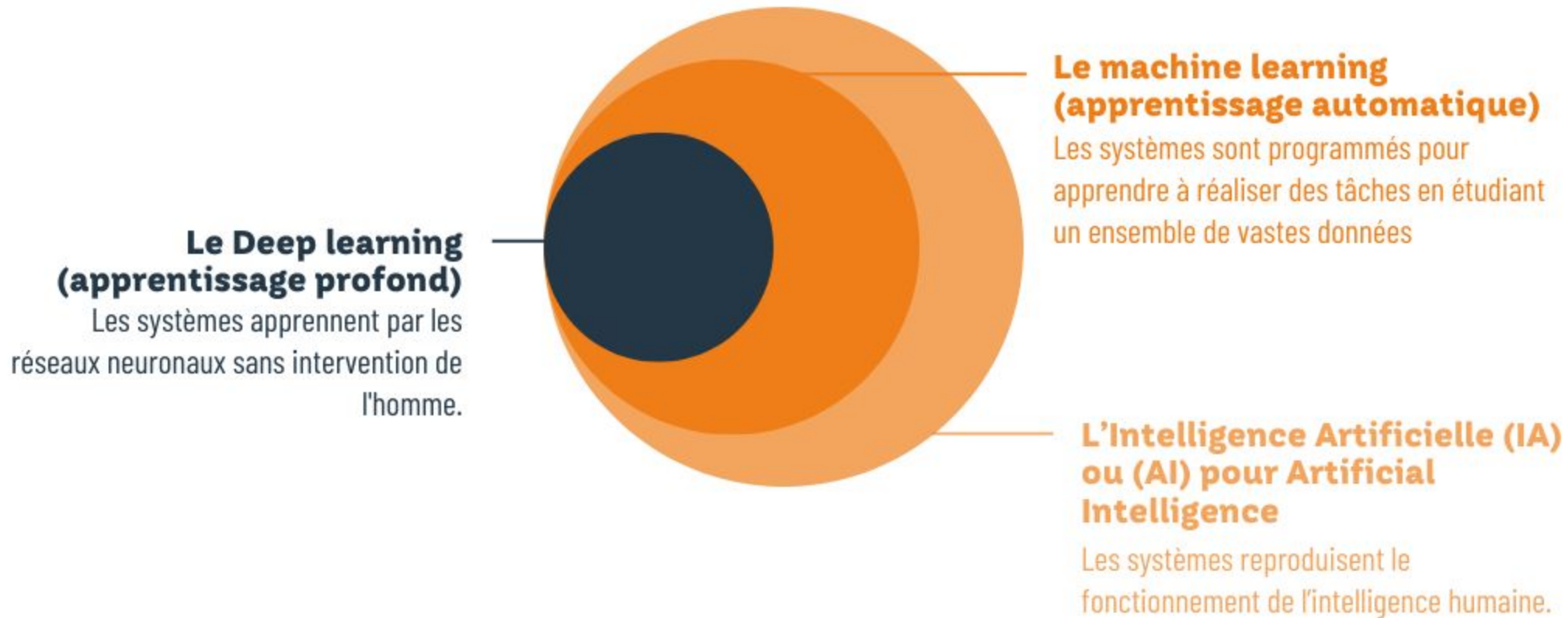


Créer un Agent qui peut avoir un comportement intelligente

Qu'est-ce que c'est l'intelligence Artificielle



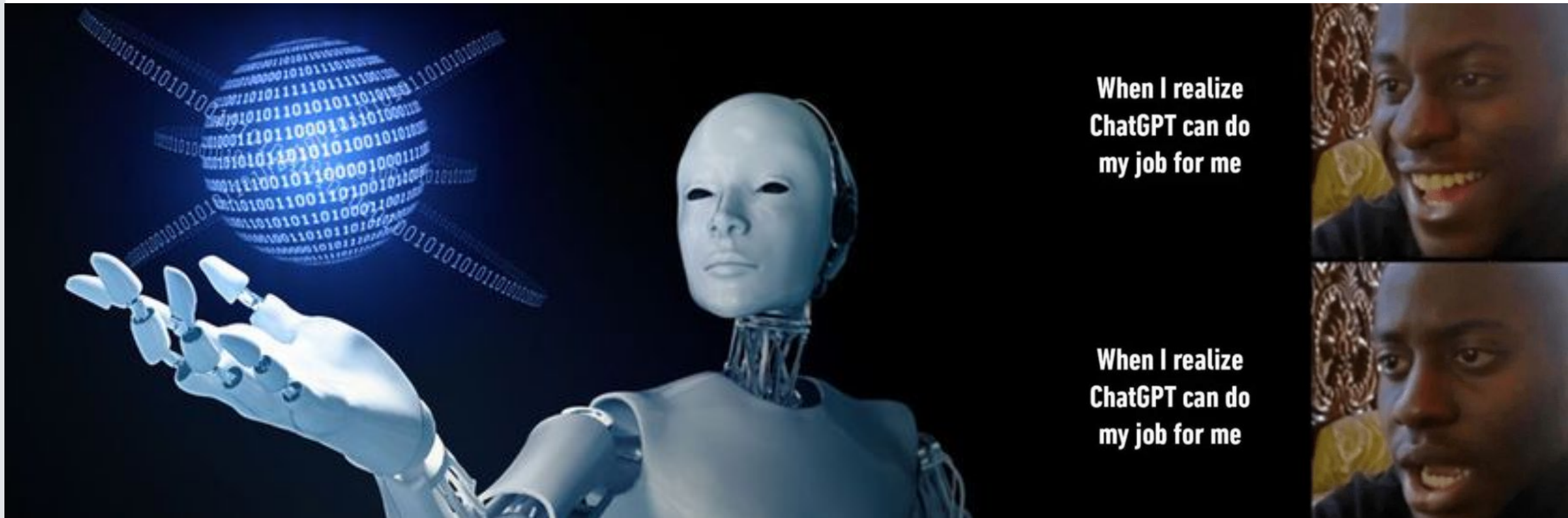
GNOMON[®]
DIGITAL



Qu'est-ce que c'est l'intelligence Artificielle



GNOMON[®]
DIGITAL



When I realize
ChatGPT can do
my job for me

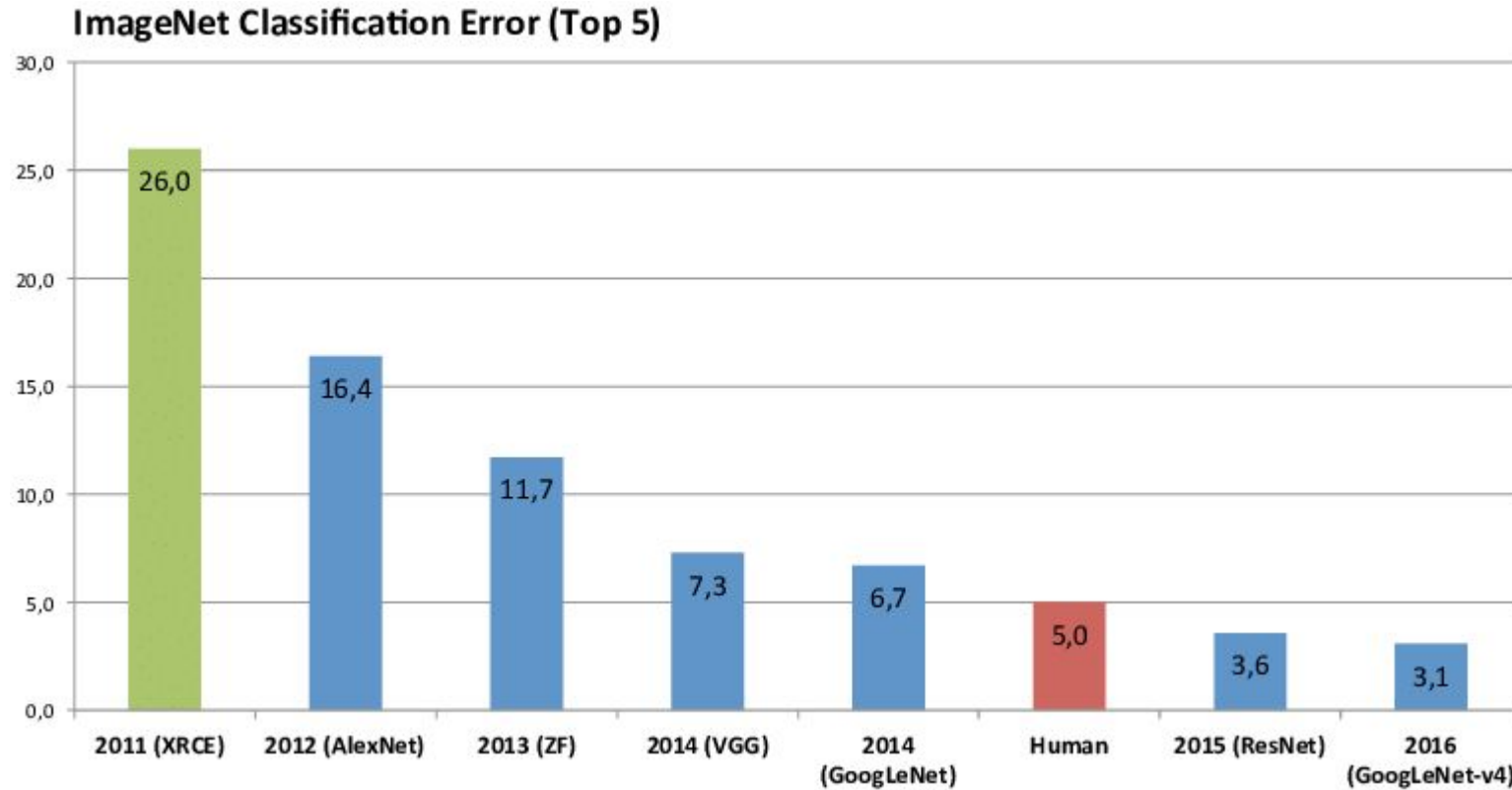
When I realize
ChatGPT can do
my job for me

Brace yourself, AI is coming

Qu'est-ce que c'est l'intelligence Artificielle

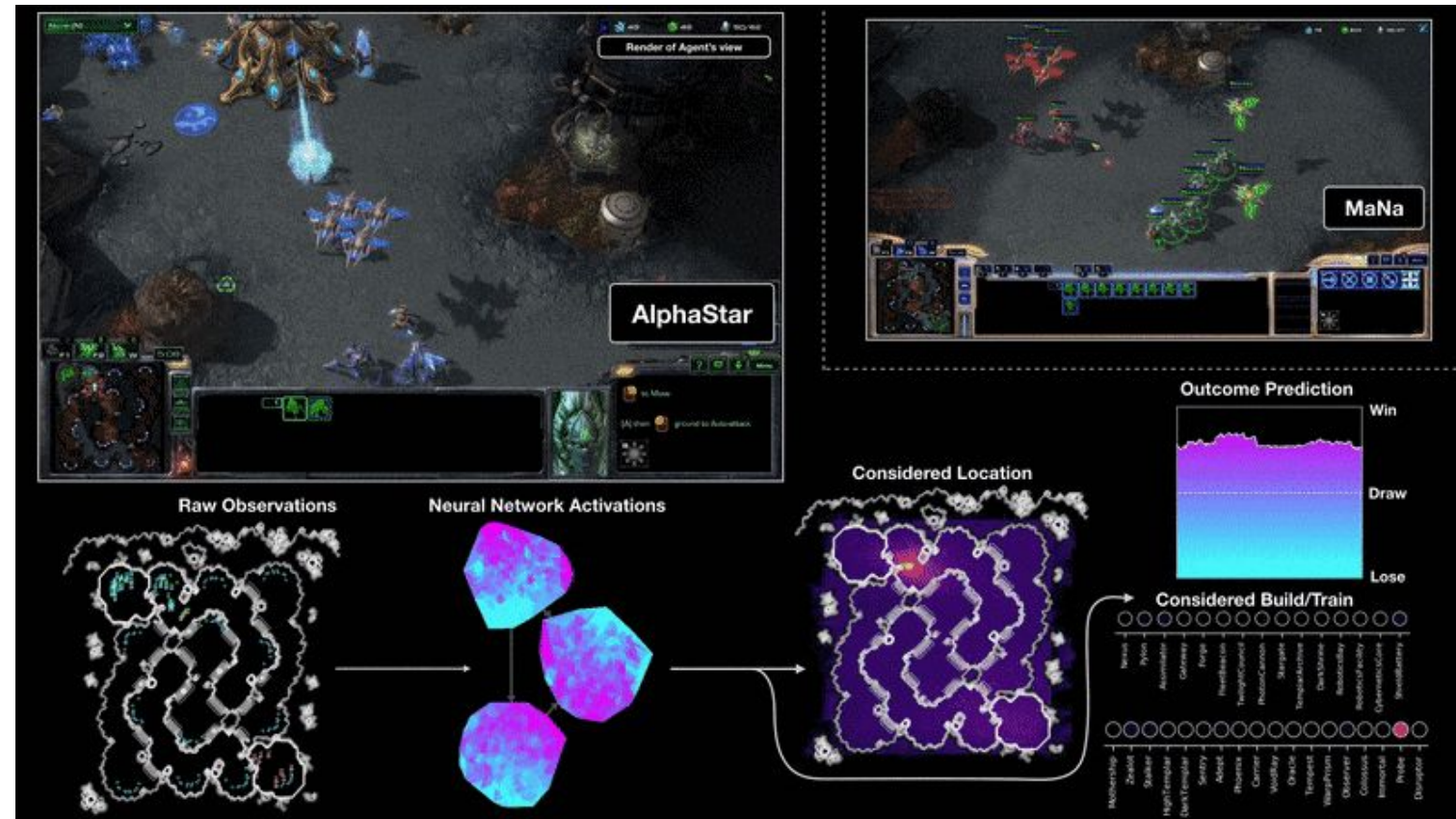


GNOMON[®]
DIGITAL



Dans certains domaines, un modèle deep learning peut surperformer un humain...
Avant la naissance de ChatGPT

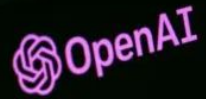
Qu'est-ce que c'est l'intelligence Artificielle



Qu'est-ce que c'est l'intelligence Artificielle



GNOMON[®]
DIGITAL



ChatGPT: Optimizing Language Models for Dialogue

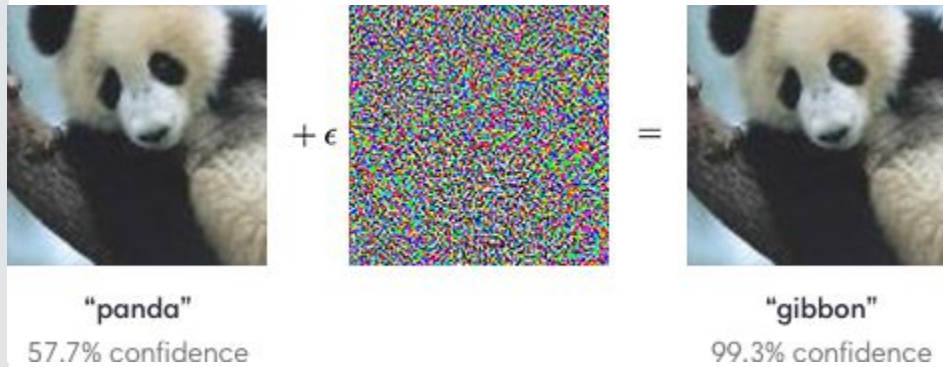
We've trained a model called ChatGPT which interacts in a conversational way. The dialogue format makes it possible for ChatGPT to answer followup questions, admit its mistakes, challenge incorrect premises, and reject inappropriate requests. ChatGPT is a sibling model to InstructGPT, which is trained to follow an instruction in a prompt and provide a response.

Rank	Model	EM	F1
	Human Performance <i>Stanford University</i> (Rajpurkar et al. '16)	82.304	91.221
1 Oct 05, 2018	BERT (ensemble) <i>Google AI Language</i> https://arxiv.org/abs/1810.04805	87.433	93.160
2 Oct 05, 2018	BERT (single model) <i>Google AI Language</i> https://arxiv.org/abs/1810.04805	85.083	91.835
2 Sep 09, 2018	nlNet (ensemble) <i>Microsoft Research Asia</i>	85.356	91.202
2 Sep 26, 2018	nlNet (ensemble) <i>Microsoft Research Asia</i>	85.954	91.677
3 Jul 11, 2018	QANet (ensemble) <i>Google Brain & CMU</i>	84.454	90.490
4 Jul 08, 2018	r-net (ensemble) <i>Microsoft Research Asia</i>	84.003	90.147
5 Mar 19, 2018	QANet (ensemble) <i>Google Brain & CMU</i>	83.877	89.737

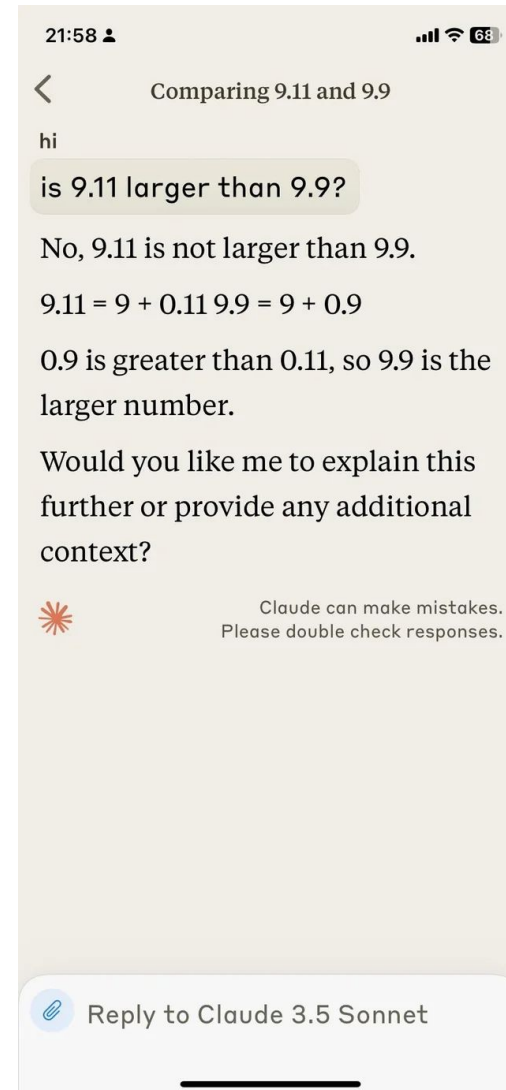
Qu'est-ce que c'est l'intelligence Artificielle



GNOMON[®]
DIGITAL



<https://arxiv.org/pdf/1412.6572.pdf>

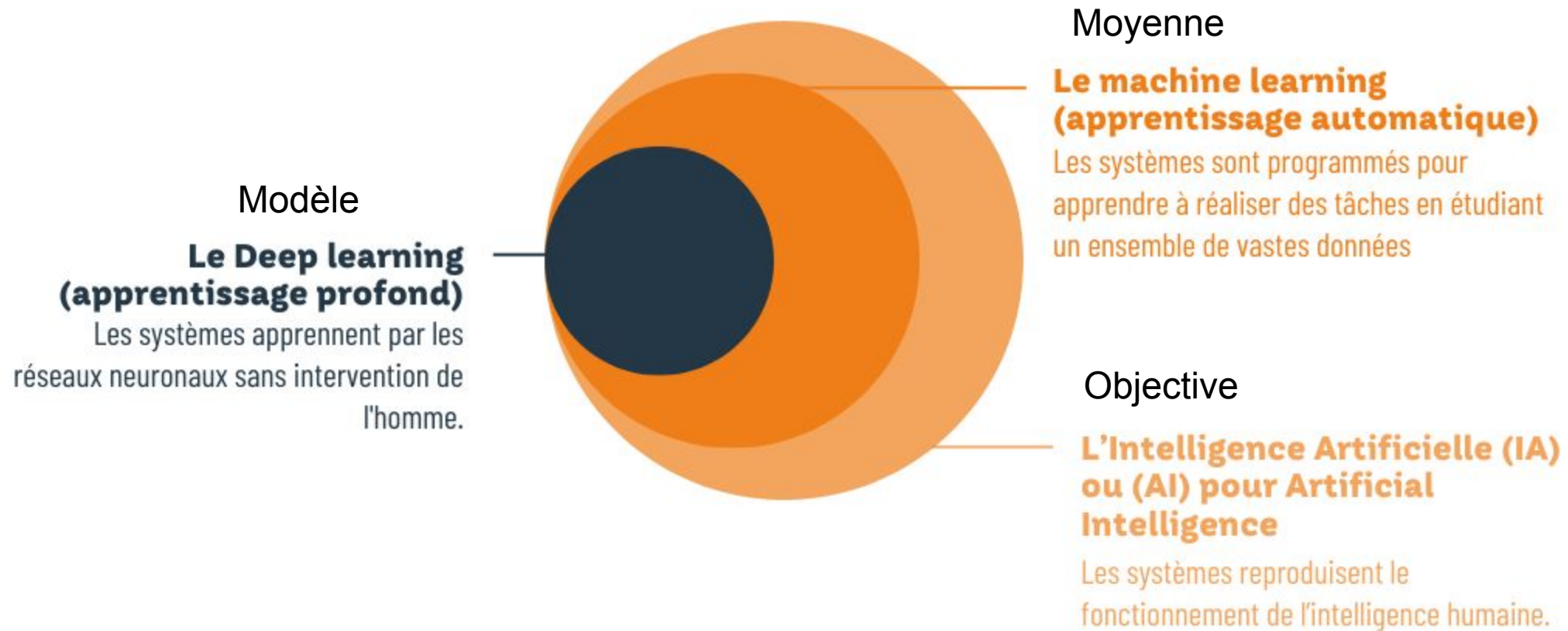


<https://www.linkedin.com/pulse/do-llms-make-errors-way-troubleshoot-llm-error-rates-the-next-tech-i70nc/>

Qu'est-ce que c'est Machine Learning



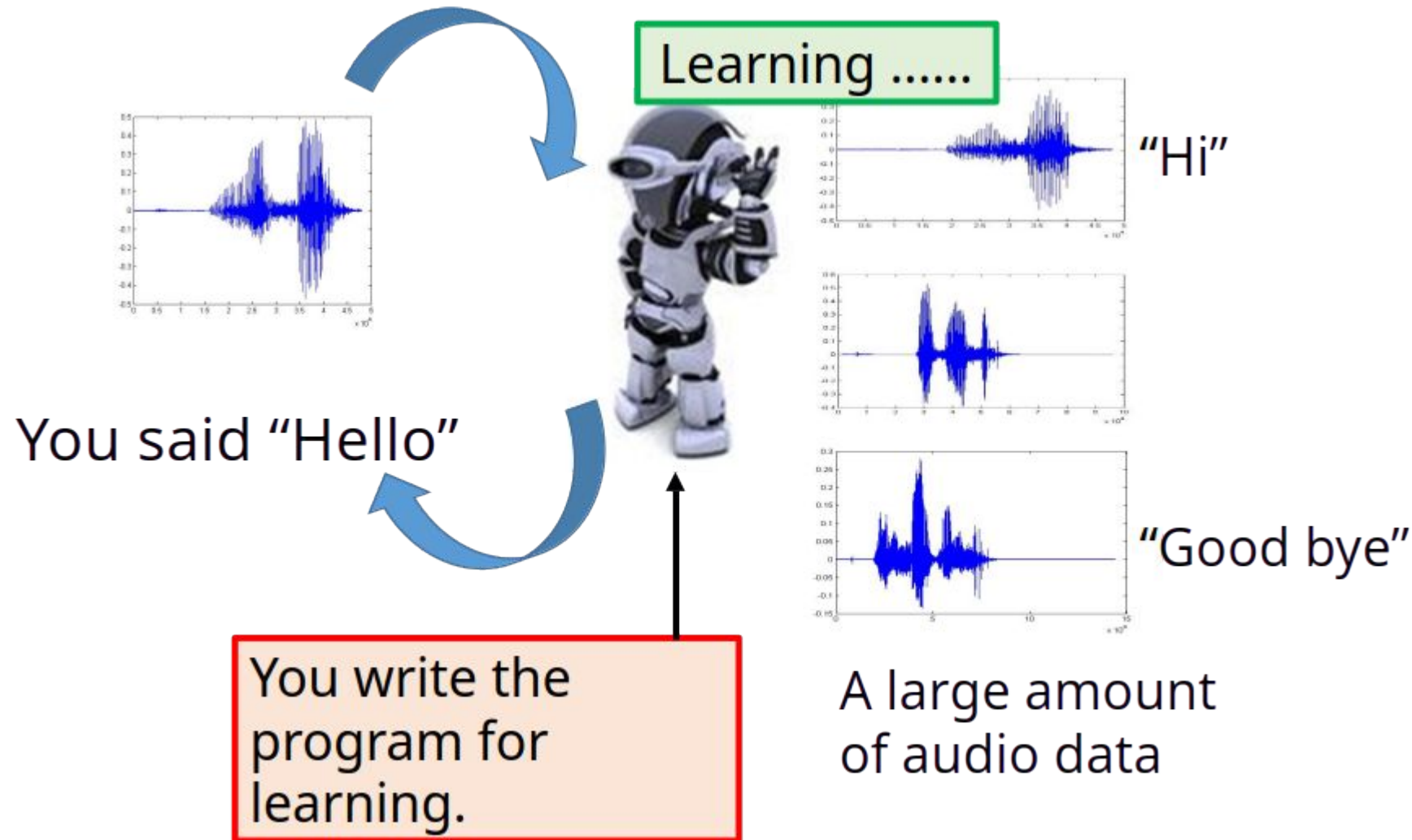
GNOMON[®]
DIGITAL



Qu'est-ce que c'est Machine Learning



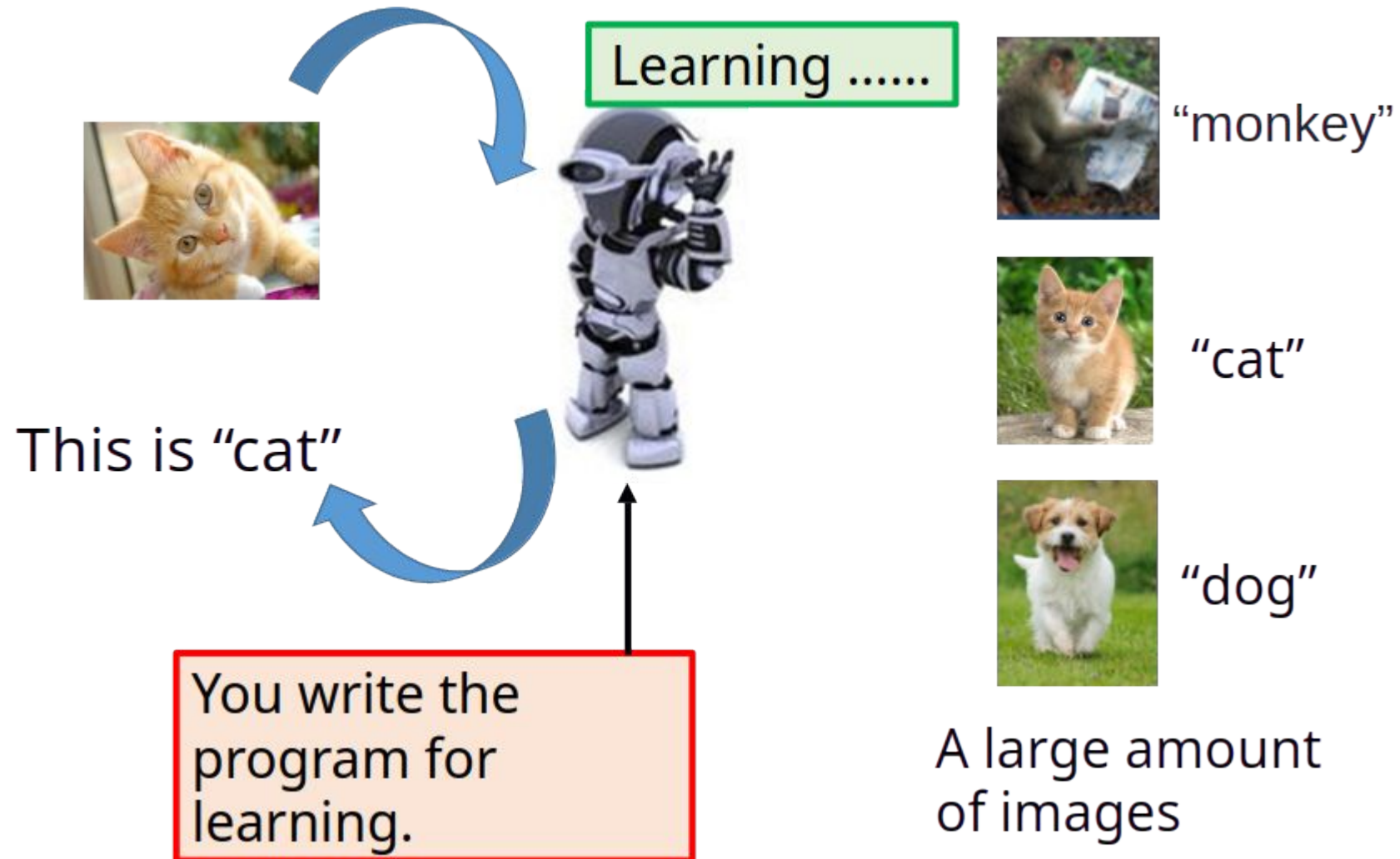
GNOMON[®]
DIGITAL



Qu'est-ce que c'est Machine Learning



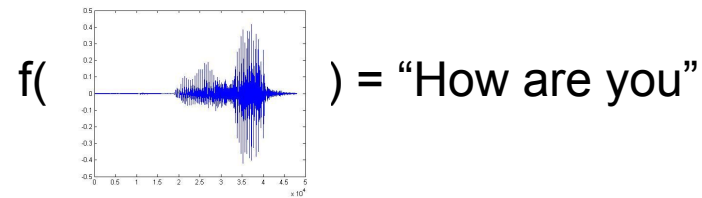
GNOMON[®]
DIGITAL



Qu'est-ce que c'est Machine Learning



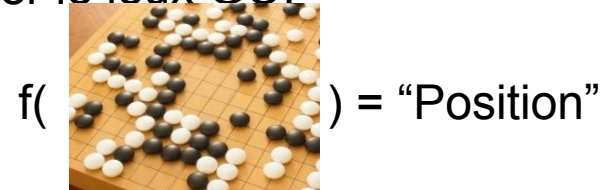
- Reconnaissance vocal:



- Reconnaissance image:



- Jouer le jeu GO:



- Chatbot:

$f(\text{"Bonjour, Ca va bien?"}) = \text{"Oui, très bien."}$

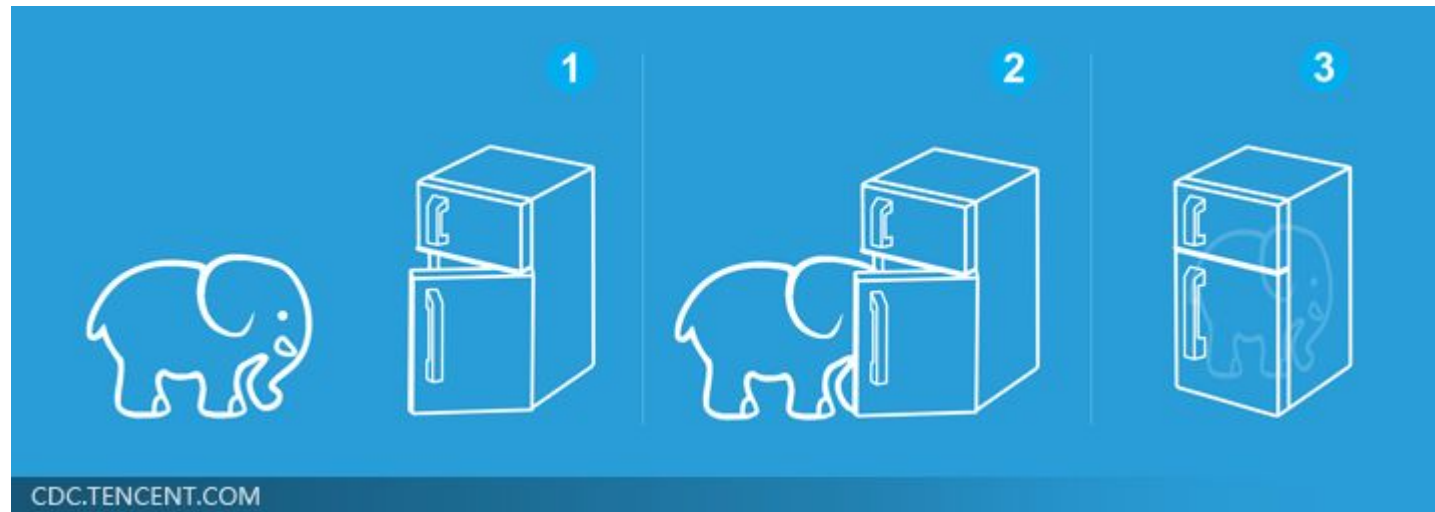
<https://ml-playground.com/#>

Qu'est-ce que c'est Machine Learning



GNOMON[®]
DIGITAL

- Une modèle à apprendre
- Des données pour entraîner le modèle
- Une fonction de coût qui évalue l'apprentissage de ce modèle

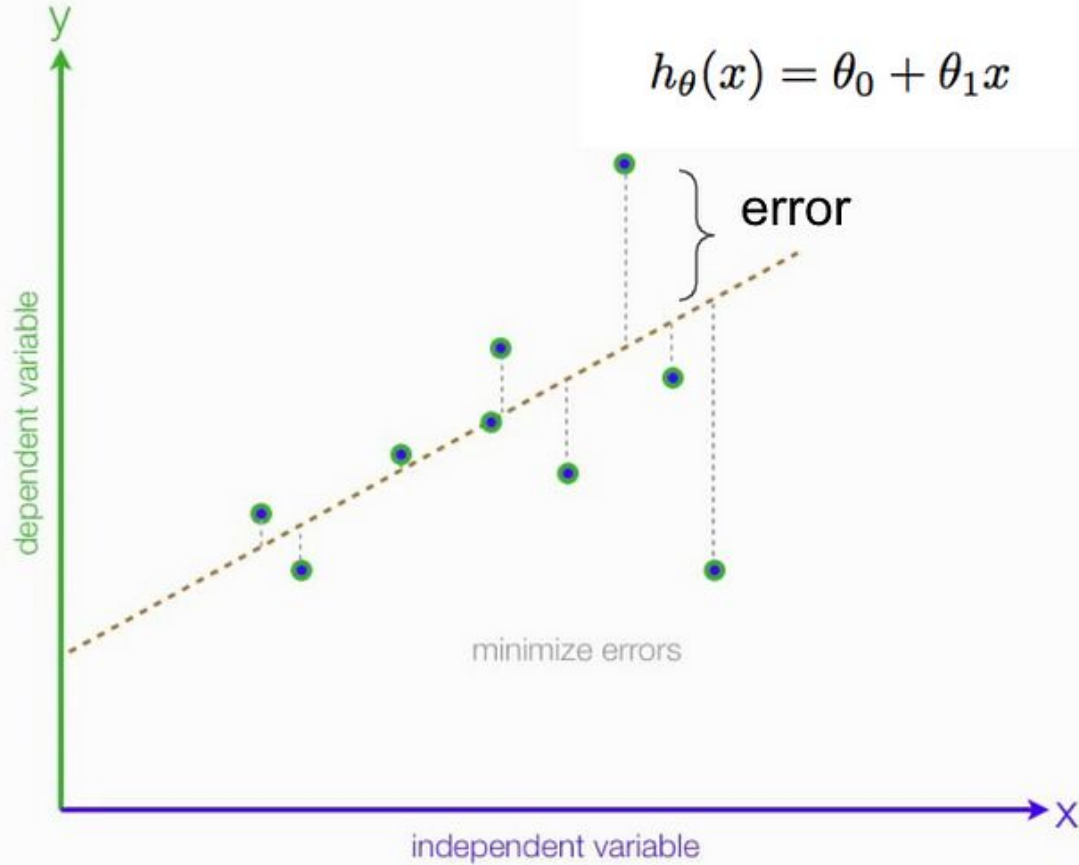


<https://ml-playground.com/#>

Qu'est-ce que c'est Machine Learning



GNOMON[®]
DIGITAL



Hypothesis:

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

Parameters:

$$\theta_0, \theta_1$$

Cost Function:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

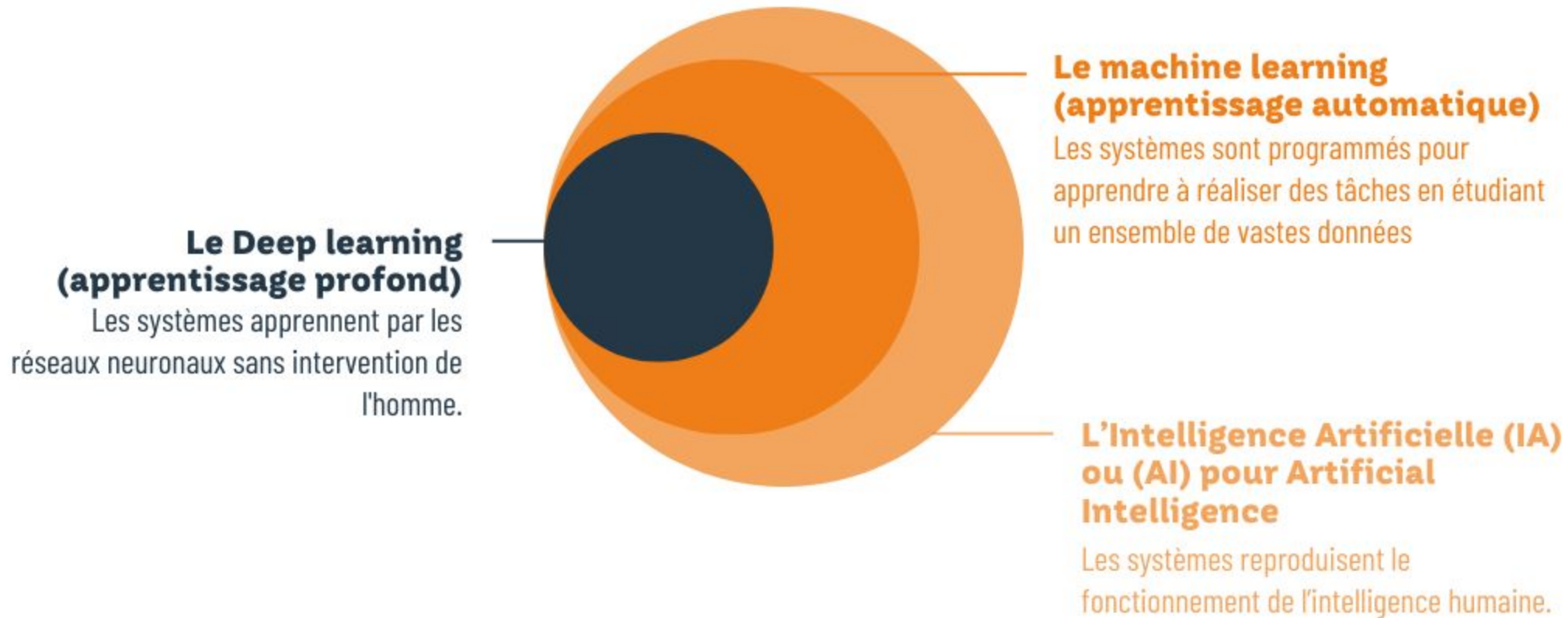
Goal:

$$\underset{\theta_0, \theta_1}{\text{minimize}} J(\theta_0, \theta_1)$$

Qu'est-ce que c'est l'intelligence Artificielle



GNOMON[®]
DIGITAL



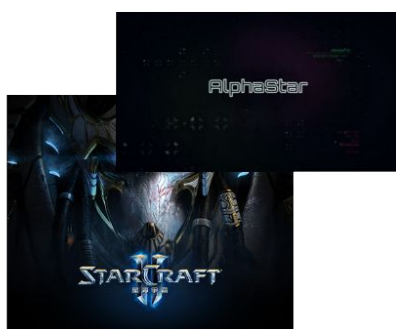
Deep learning



GNOMON[®]
DIGITAL

- Pour le moment, un modèle peut souvent résoudre une type de problème

On est là!



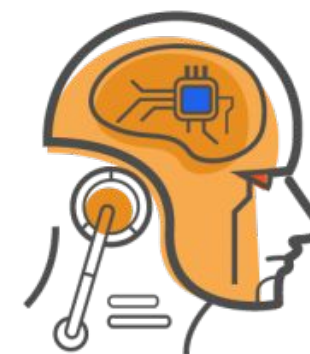
Narrow AI

Dedicated to assist with or take over specific tasks.



General AI

Takes knowledge from one domain, transfers to other domain.



Super AI

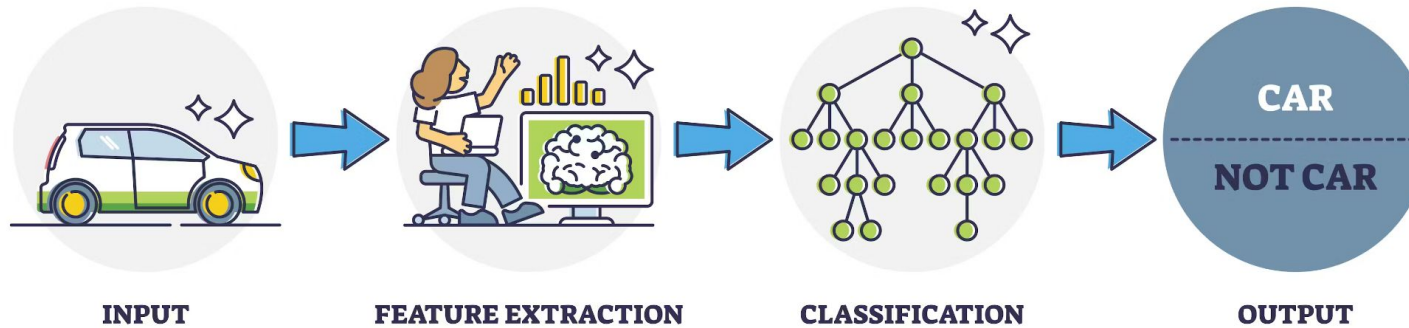
Machines that are an order of magnitude smarter than humans.

Quelle est la différence entre DL et ML

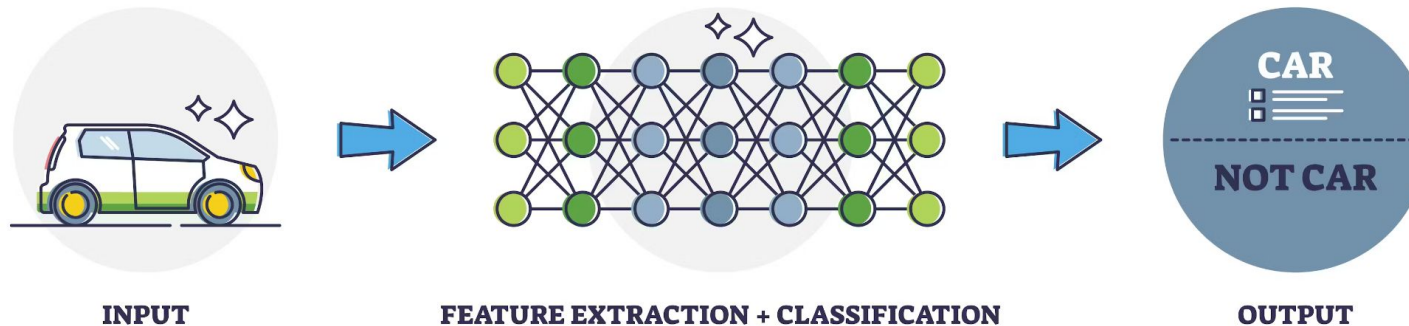


GNOMON[®]
DIGITAL

MACHINE LEARNING



DEEP LEARNING



Quelle est la différence entre DL et ML



GNOMON[®]
DIGITAL

Machine learning vs. deep learning



Machine learning

Uses algorithms and learns on its own but may need human intervention to correct errors



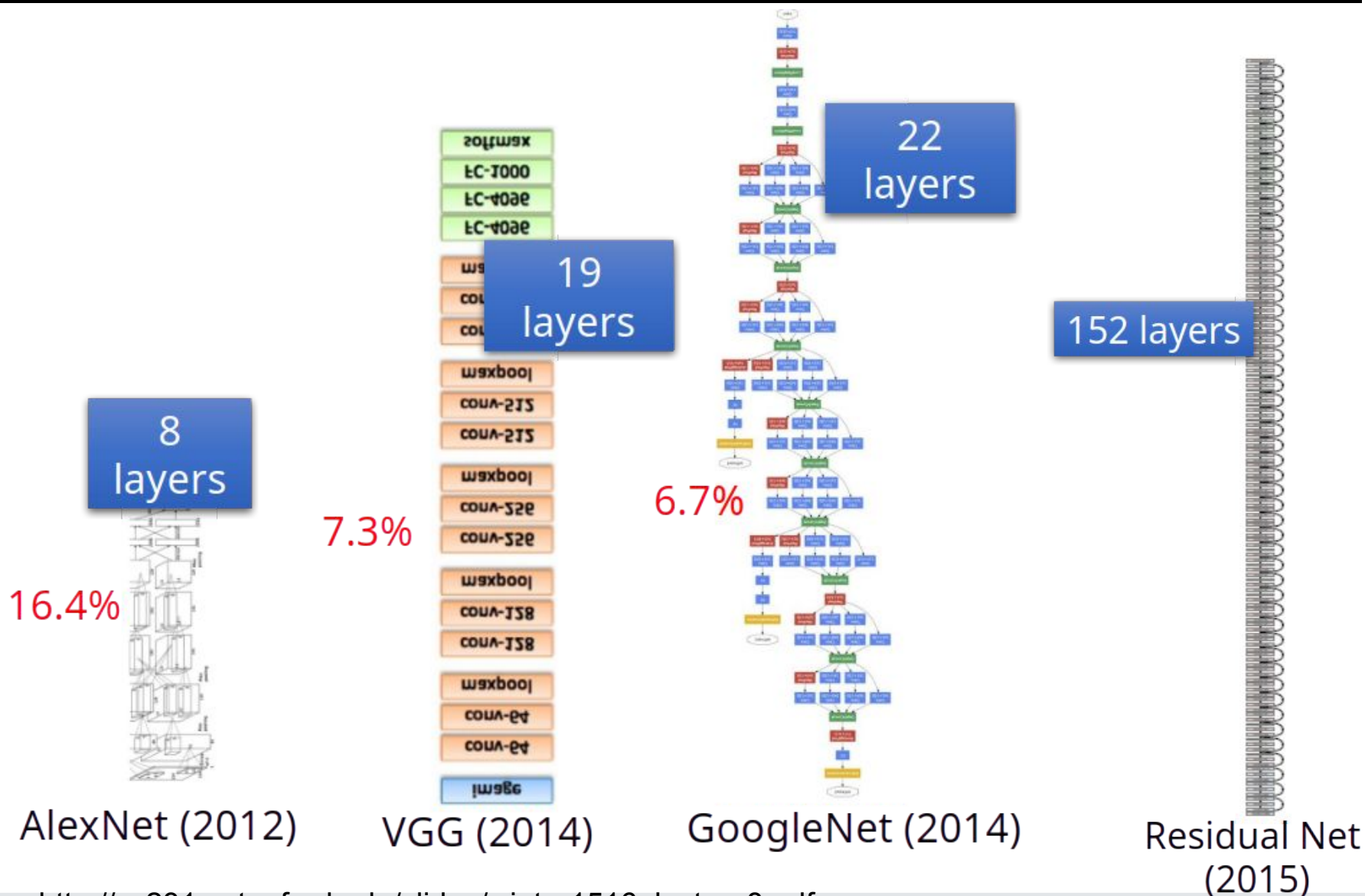
Deep learning

Uses advanced computing, its own neural network, to adapt with little to no human intervention

Quelle est la différence entre DL et NN (before 2010)



GNOMON[®]
DIGITAL

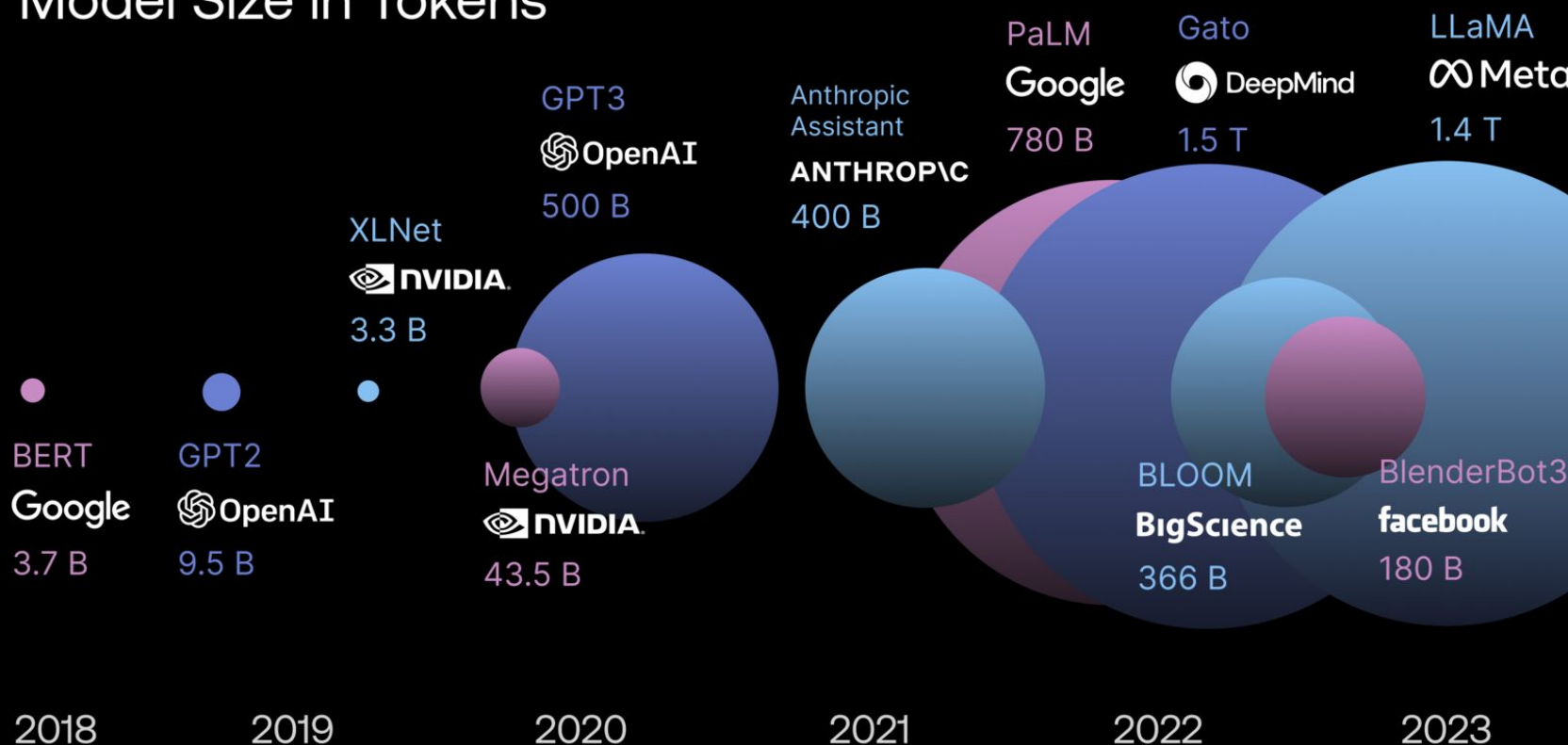


Quelle est la différence entre DL et NN (before 2010)



GNOMON[®]
DIGITAL

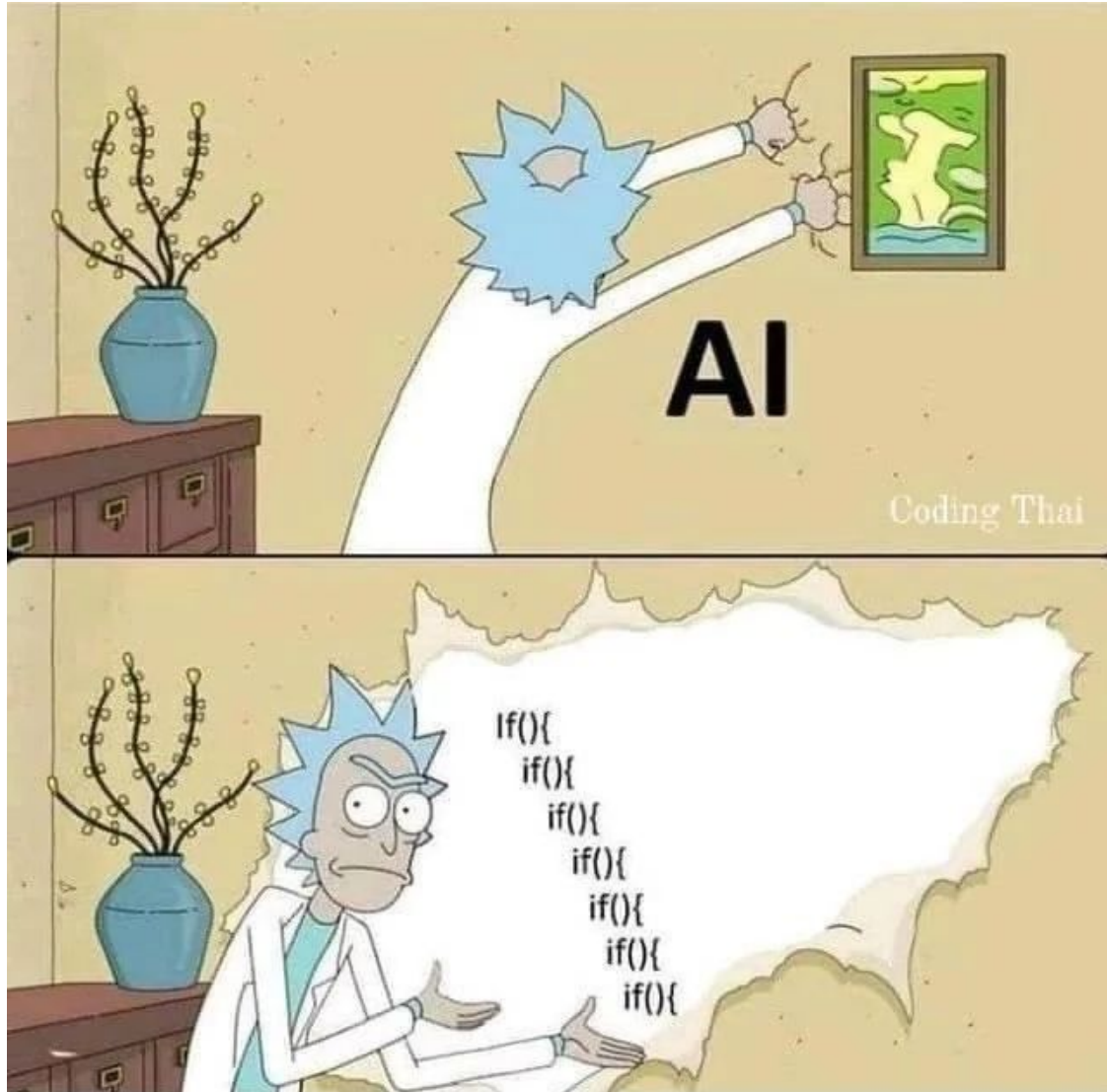
Model Size in Tokens



Deep learning



GNOMON[®]
DIGITAL



=

ChatGPT 4o >

✎

9.11 and 9.9 – which is bigger



9.11 is bigger than 9.9.

subtract them



To subtract 9.9 from 9.11, you perform the following calculation:

$$9.11 - 9.9 = 0.21$$

use python



The result of subtracting 9.9 from 9.11 in Python is `-0.79`. This discrepancy is due to the way floating-point arithmetic is handled in Python, which can lead to small precision errors. The actual intended result should be 0.21. [\[>-\]](#)

Deep learning

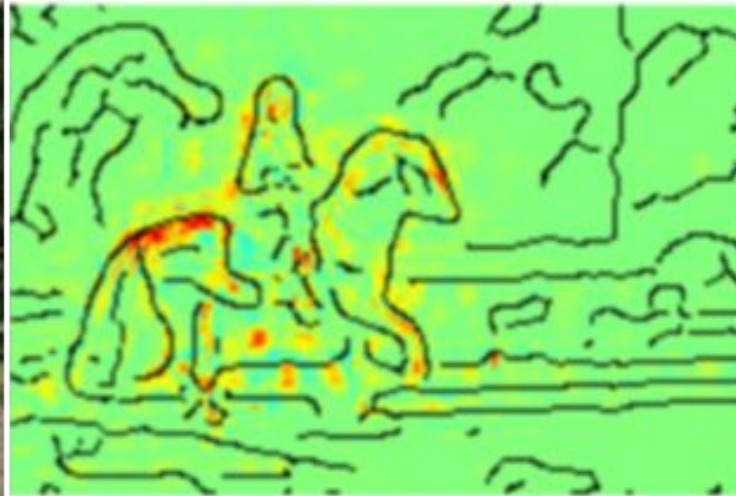


GNOMON[®]
DIGITAL

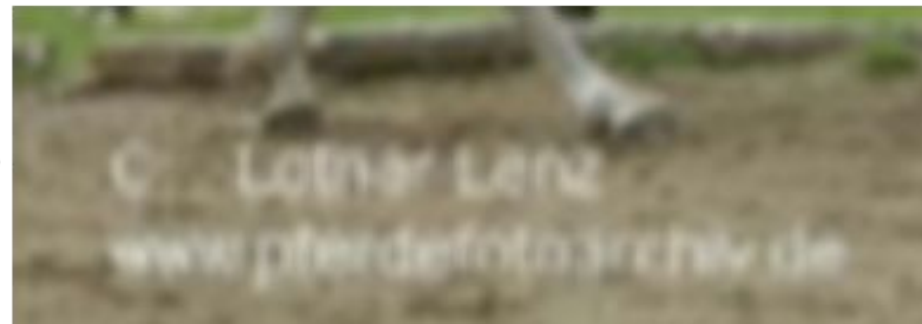
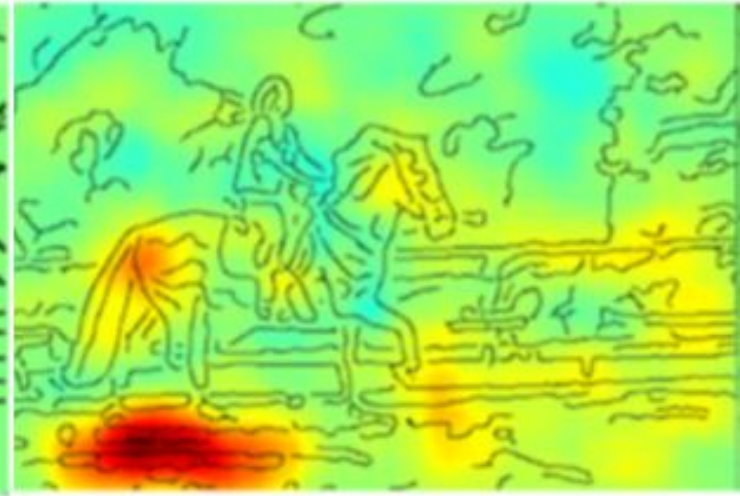
Image



DNN



FV

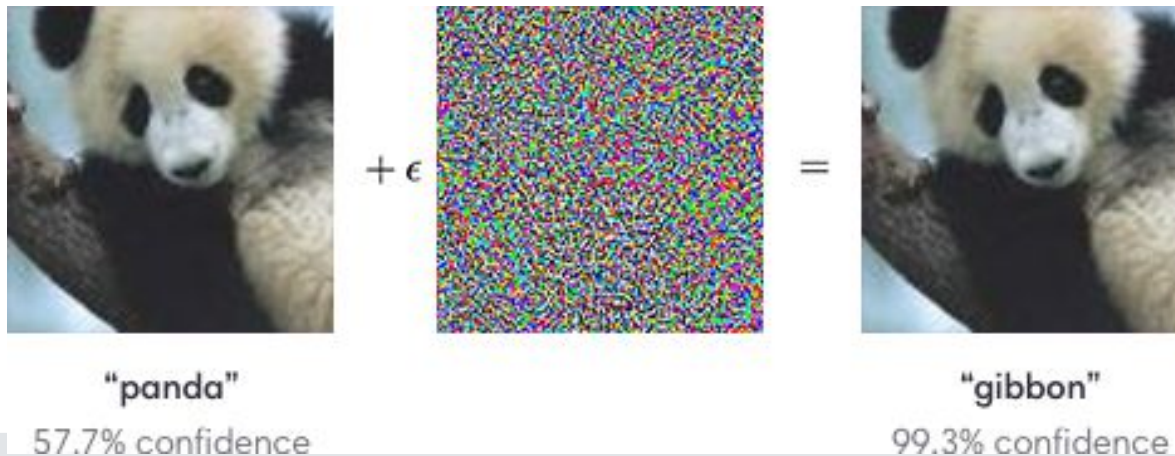


Deep learning

- L'human fais l'erreur:



- Le modèle aussi:

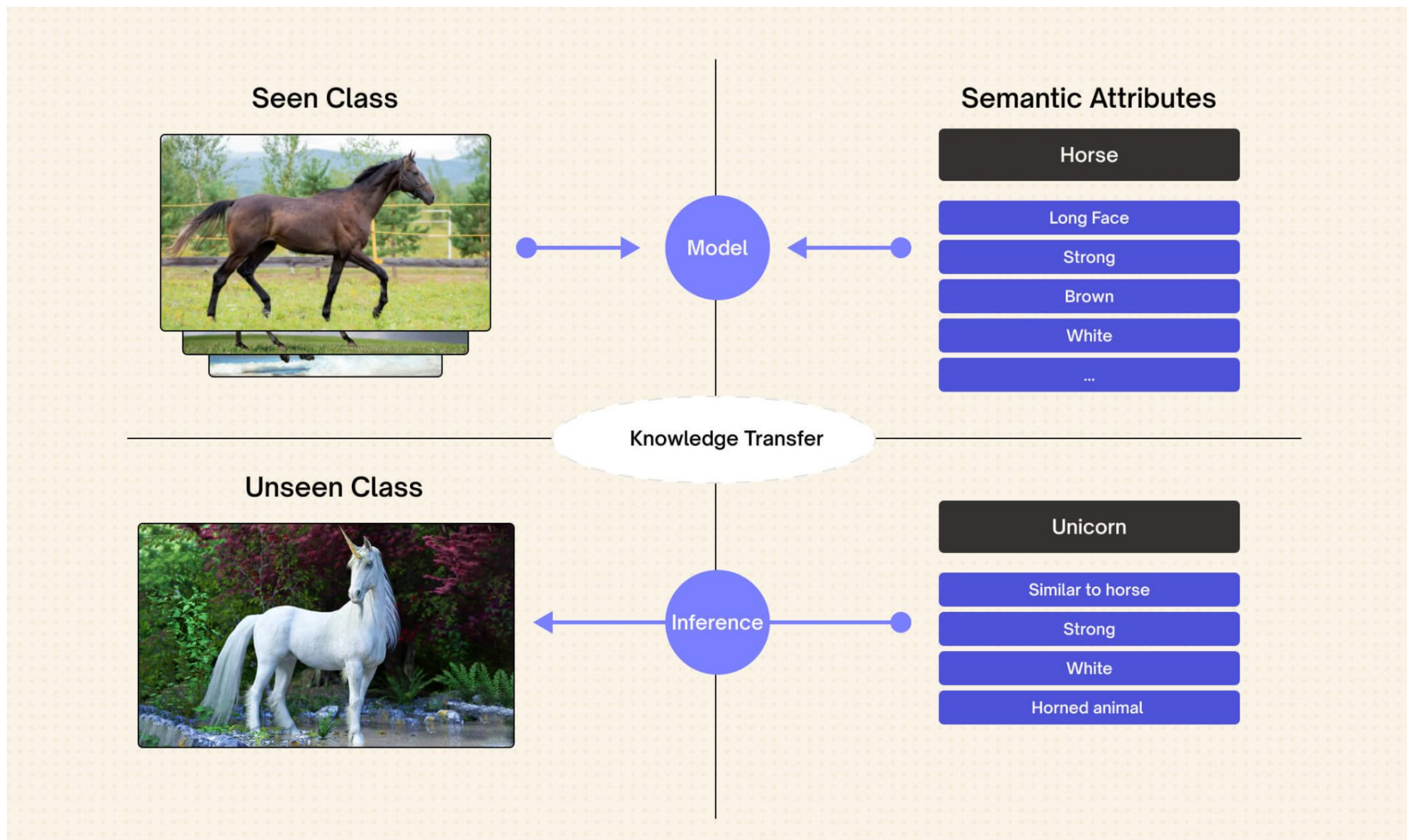


Deep learning



GNOMON[®]
DIGITAL

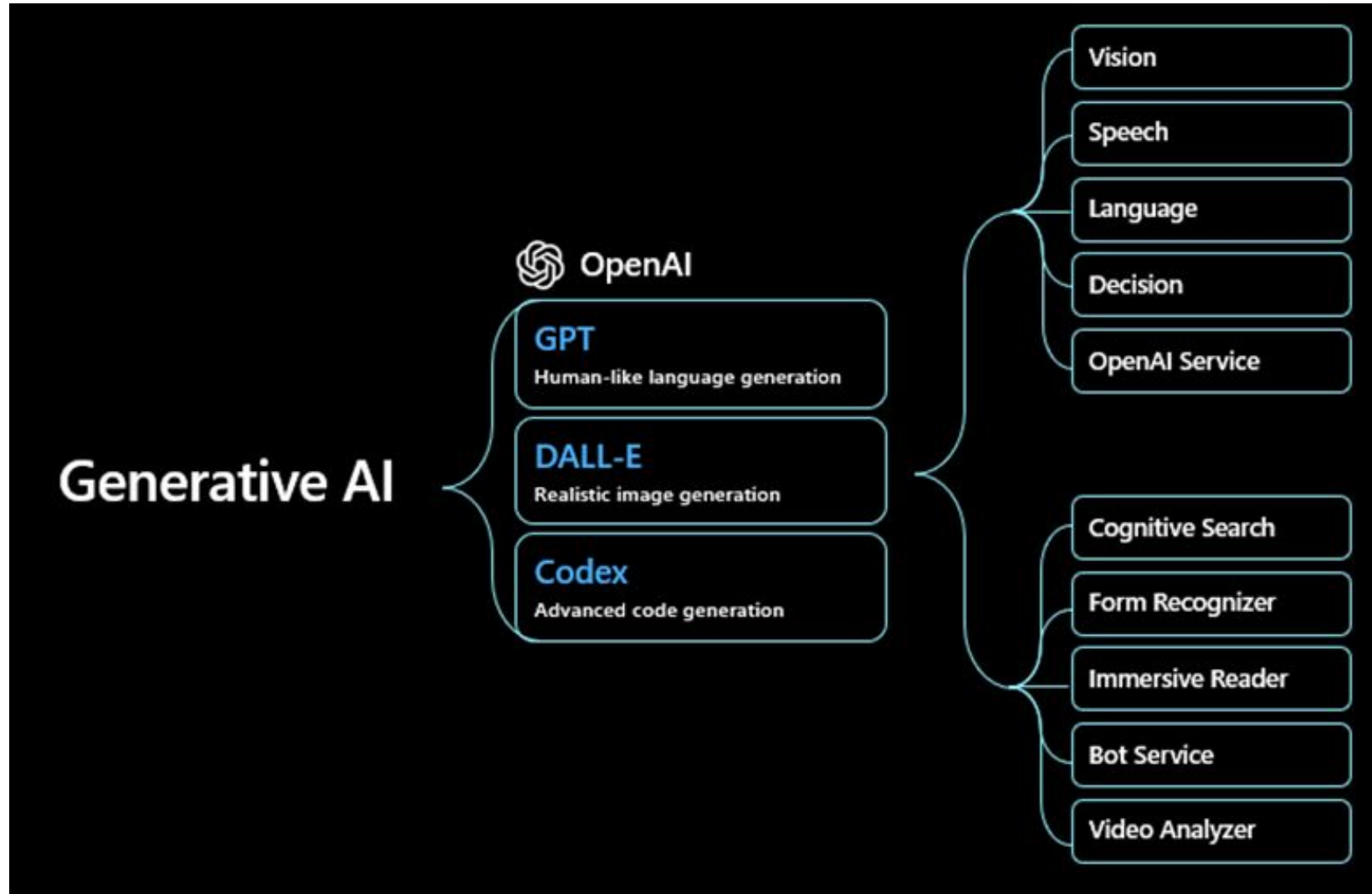
- Few-shot learning
- Zero-shot learning



Deep learning



GNOMON[®]
DIGITAL



Deep learning



GNOMON[®]
DIGITAL

Pourquoi c'est si difficile? -> Distribution change



Training data



Testing data



99.5%



57.5%

Structure d'un modèle deep learning



GNOMON[®]
DIGITAL

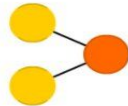
CNN, LSTM ou un LLM sont just des façons différentes de connecter entres des couches des neurons.

A mostly complete chart of Neural Networks

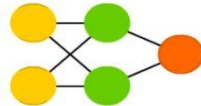
©2016 Fjodor van Veen - asimovinstitute.org

- Backfed Input Cell
- Input Cell
- △ Noisy Input Cell
- Hidden Cell
- Probabilistic Hidden Cell
- △ Spiking Hidden Cell
- Output Cell
- Match Input Output Cell
- Recurrent Cell
- Memory Cell
- △ Different Memory Cell
- Kernel
- Convolution or Pool

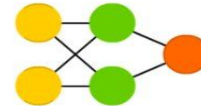
Perceptron (P)



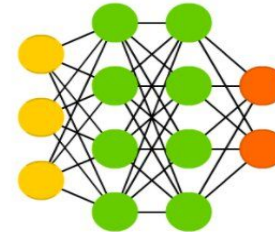
Feed Forward (FF)



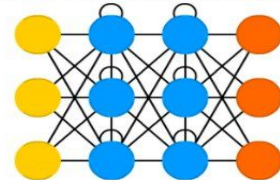
Radial Basis Network (RBF)



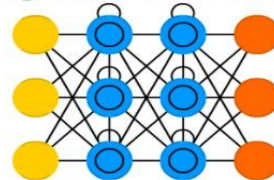
Deep Feed Forward (DFF)



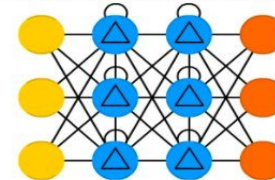
Recurrent Neural Network (RNN)



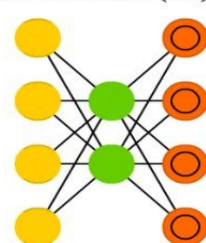
Long / Short Term Memory (LSTM)



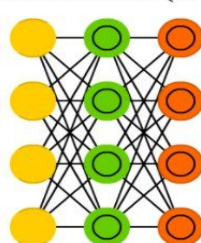
Gated Recurrent Unit (GRU)



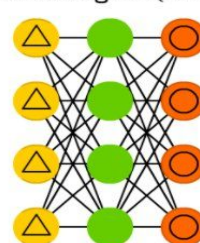
Auto Encoder (AE)



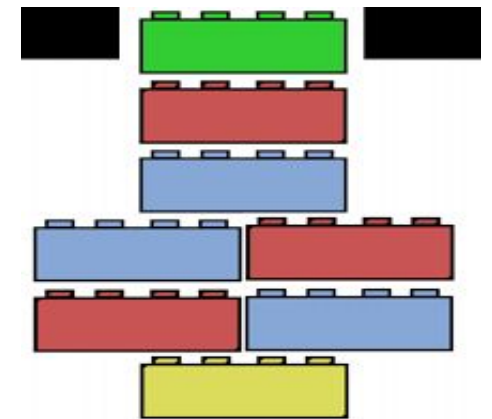
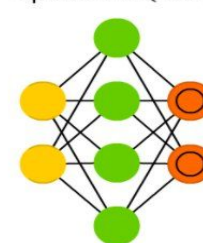
Variational AE (VAE)



Denoising AE (DAE)



Sparse AE (SAE)

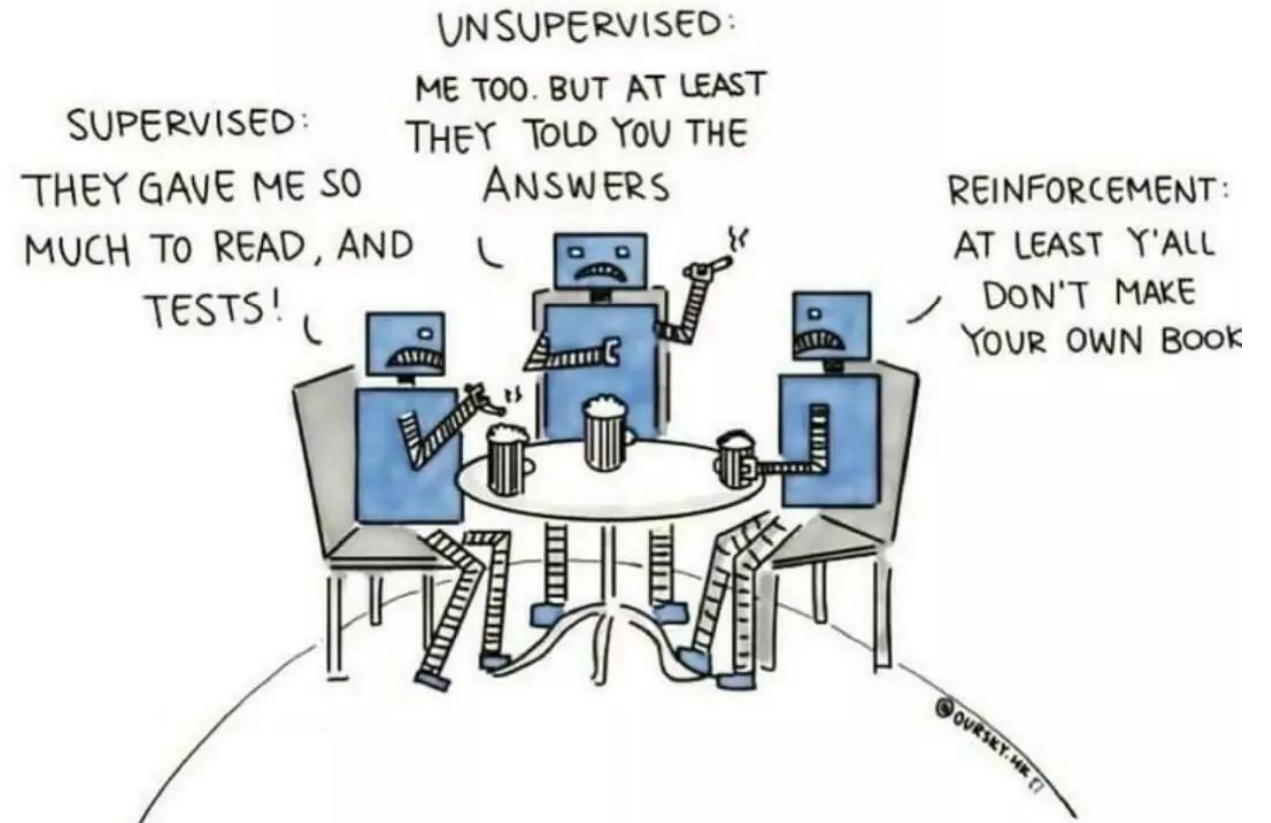


Typologie d'apprentissage



Reinforcement Learning
Apprentissage par renforcement

Three main types of Machine Learning Algorithms

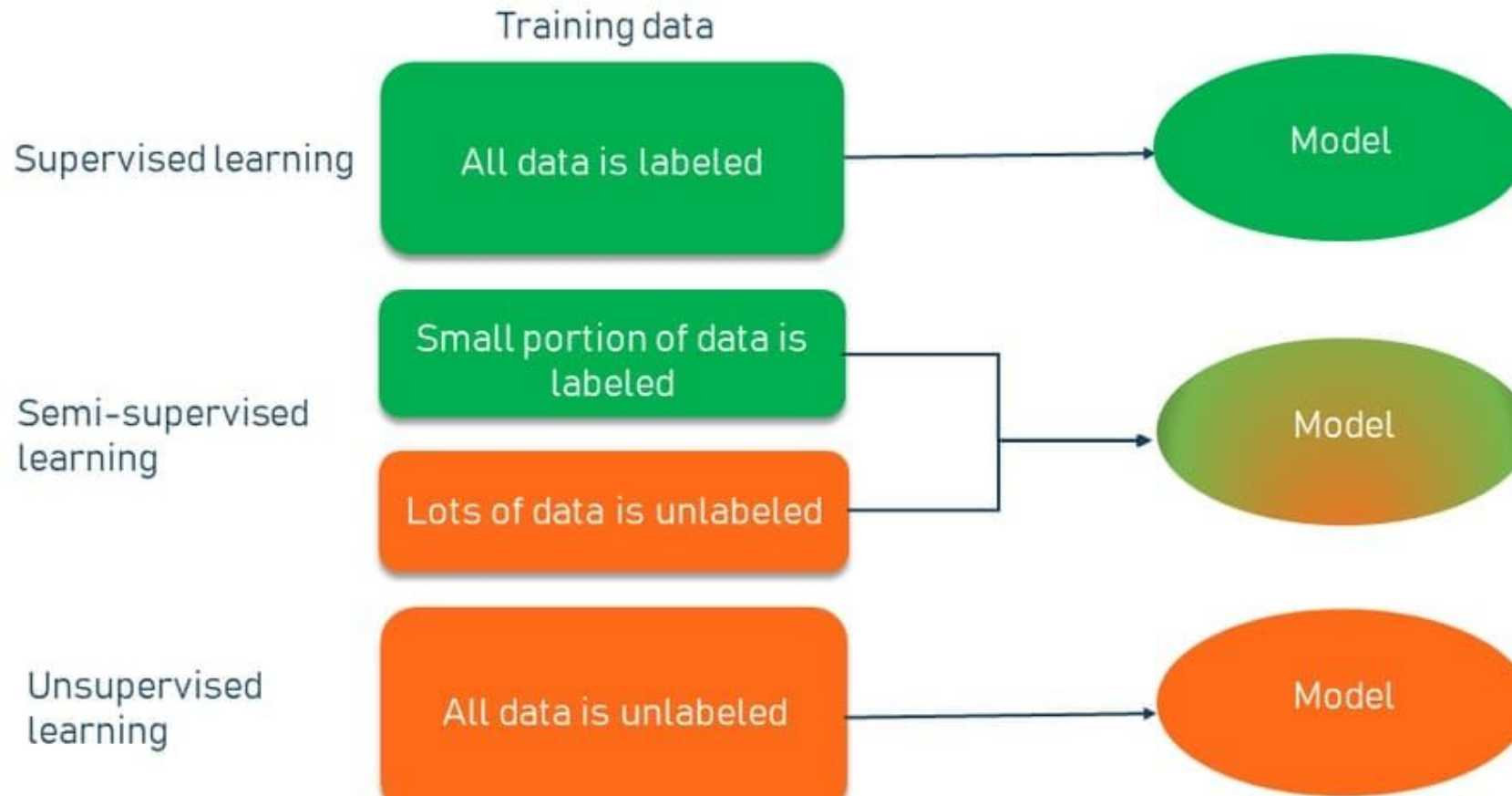


Typologie d'apprentissage



GNOMON[®]
DIGITAL

SUPERVISED LEARNING vs SEMI-SUPERVISED LEARNING vs UNSUPERVISED LEARNING



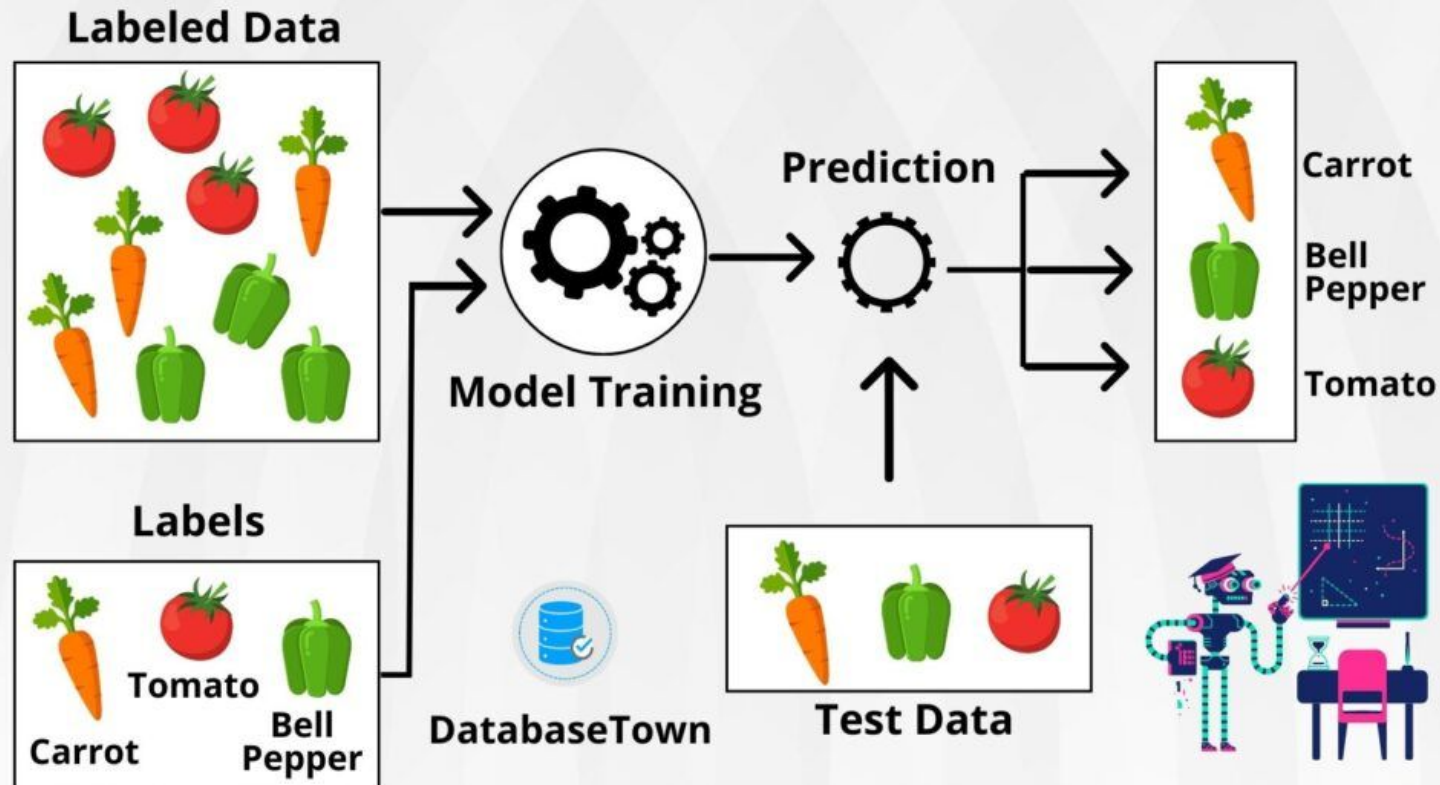
Typologie d'apprentissage



GNOMON[®]
DIGITAL

SUPERVISED LEARNING

Supervised machine learning is a branch of artificial intelligence that focuses on training models to make predictions or decisions based on labeled training data.



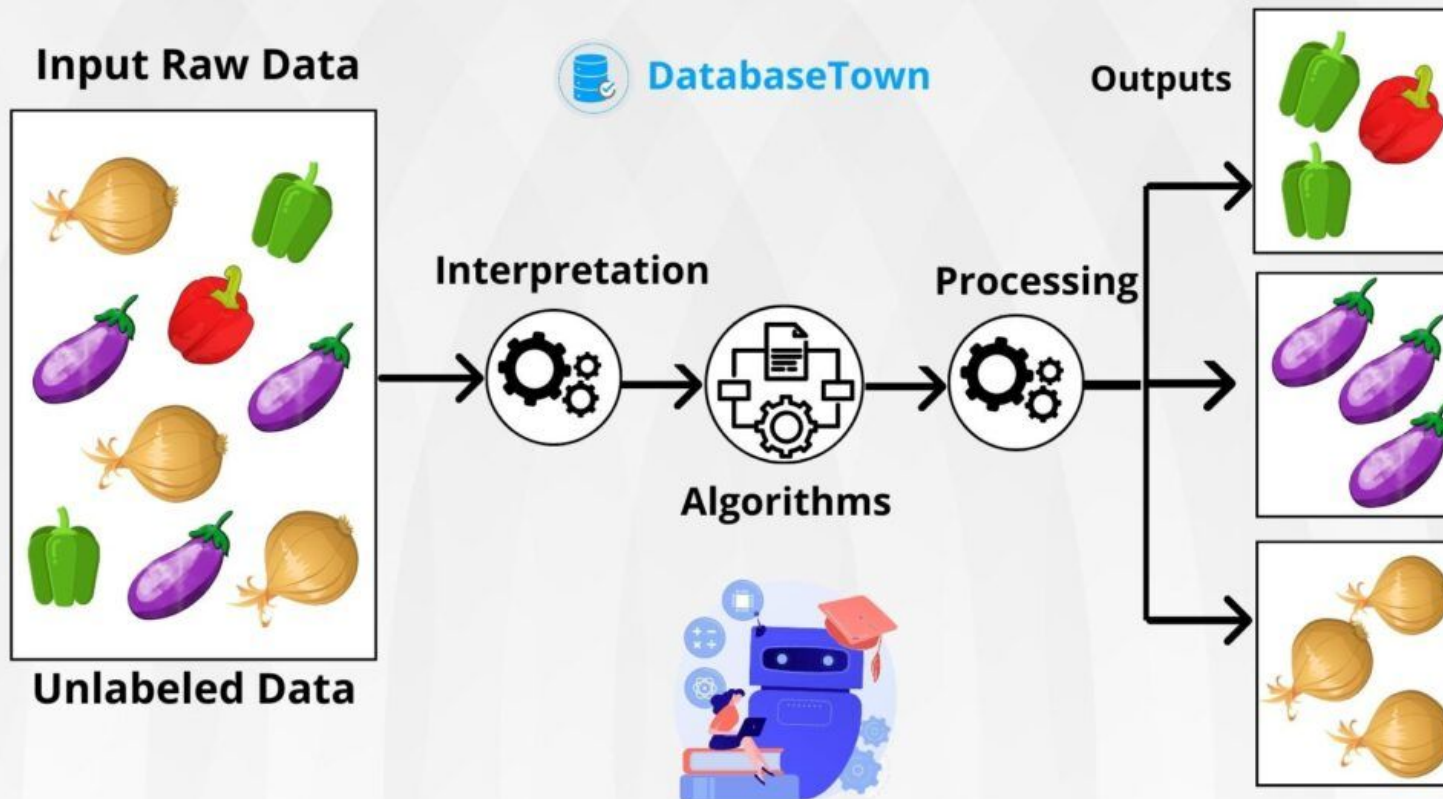
Typologie d'apprentissage



GNOMON[®]
DIGITAL

UNSUPERVISED LEARNING

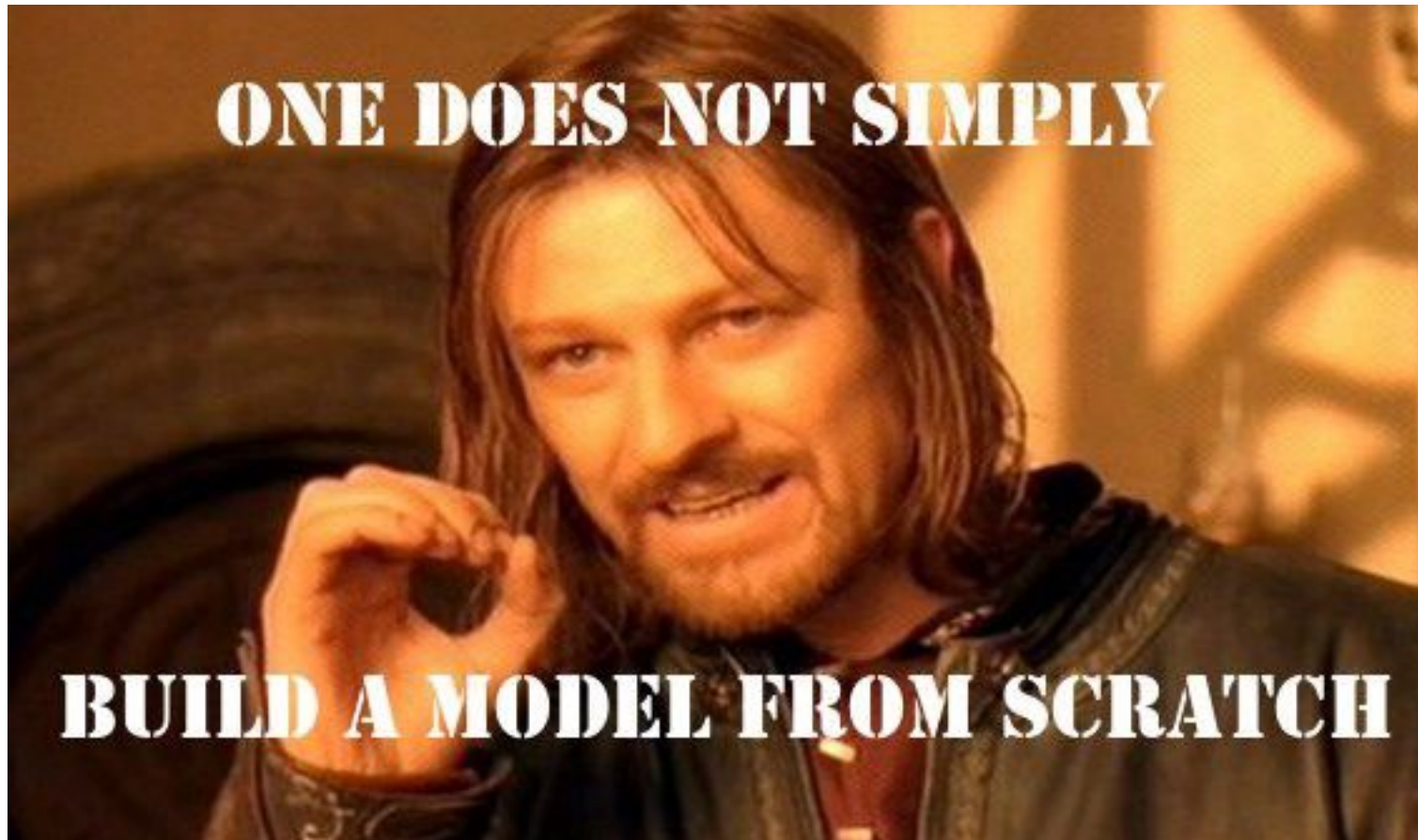
Unsupervised learning is a type of machine learning where the algorithm learns from unlabeled data without any predefined outputs or target variables.



Typologie d'apprentissage

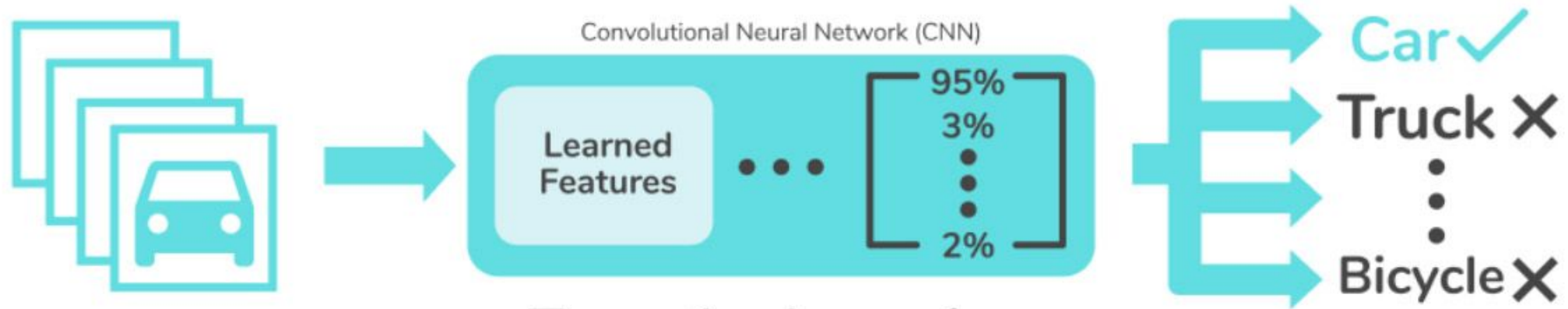


GNOMON[®]
DIGITAL



Typologie d'apprentissage

Training from Scratch



Transfer Learning

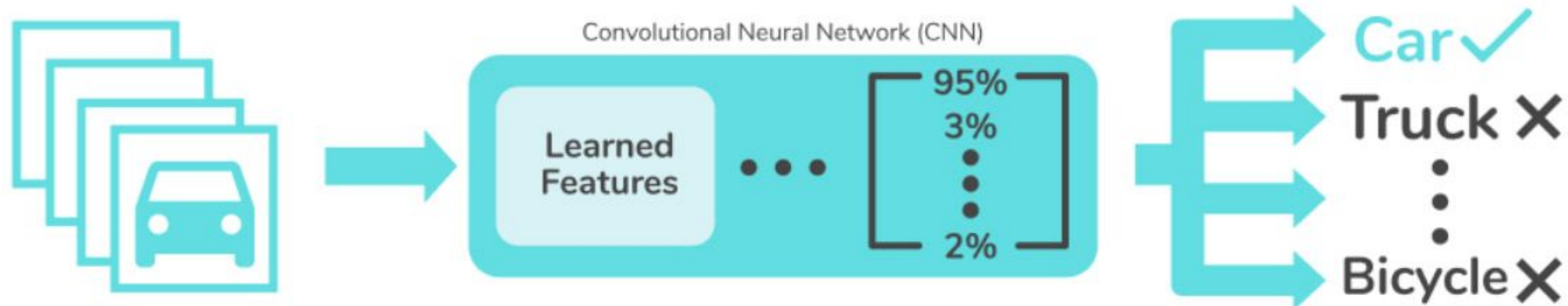


Typologie d'apprentissage



GNOMON[®]
DIGITAL

Training from Scratch



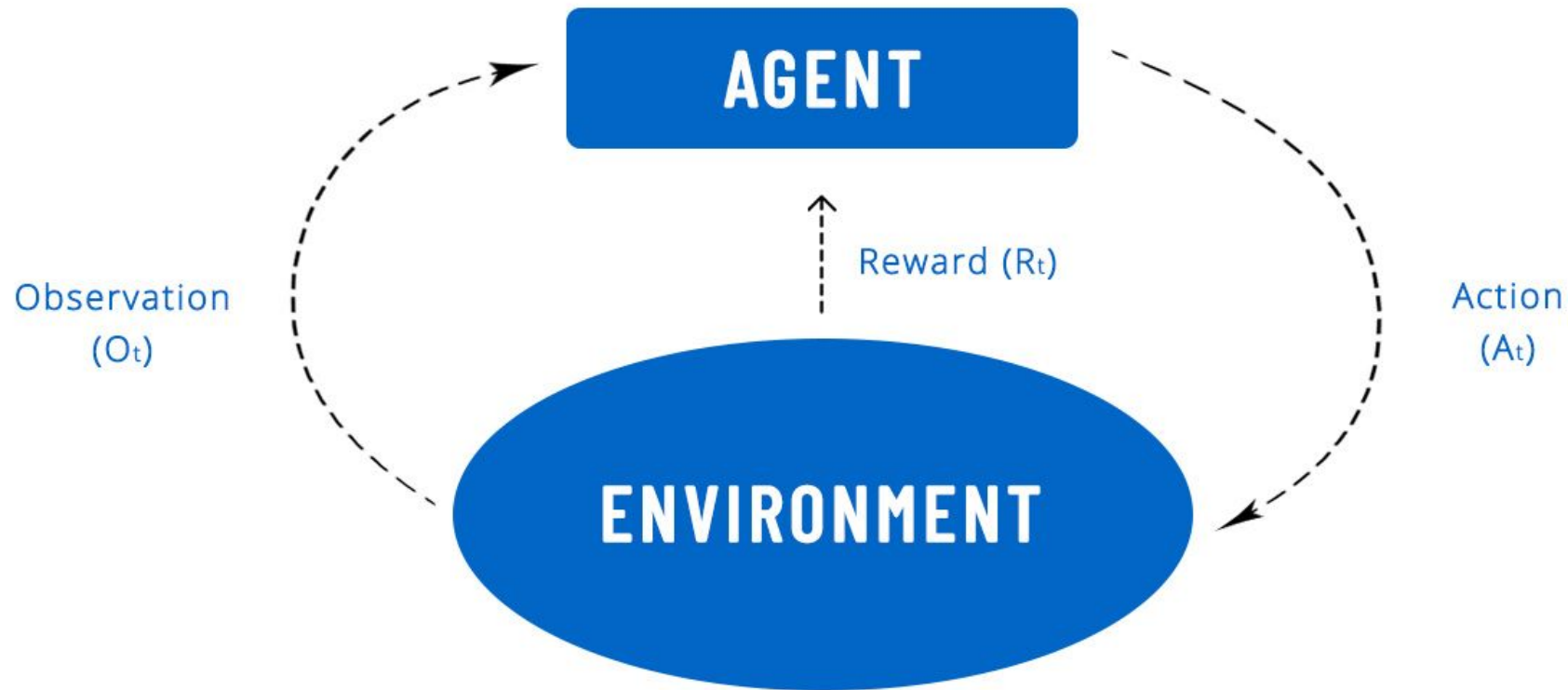
Transfer Learning



Typologie d'apprentissage



GNOMON[®]
DIGITAL



Typologie d'apprentissage

- Supervised:



Next
move:
"5-5"



Next
move:
"3-3"

- Reinforcement Learning

First
move



..... many moves
.....



Win!

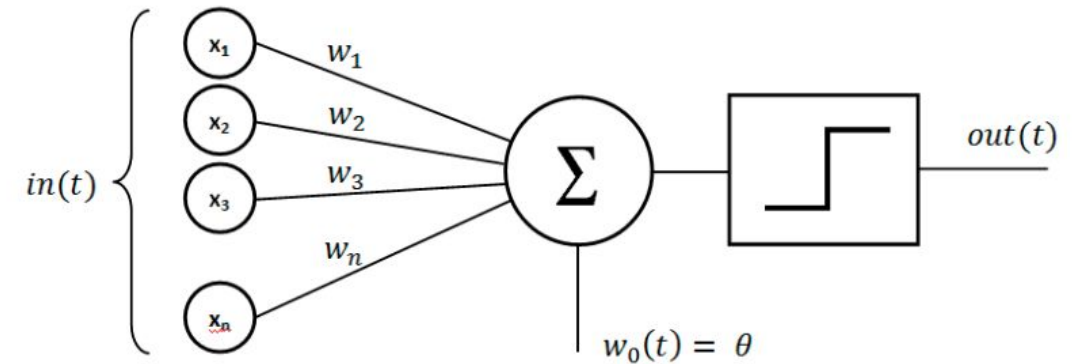
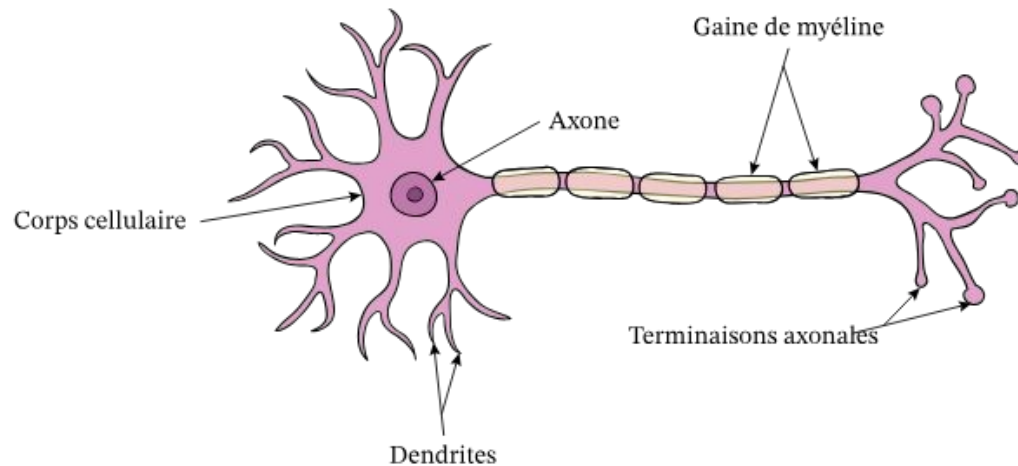


L'architecture classique:

- MLP
- CNN
- Transfer learning
- RNN
- Embedding
- Sequence-to-sequence
- Deep generative model

Introduction réseau de neurones

- Les réseaux de neurones (artificiels) sont des programmes informatiques fondés sur un modèle simplifié du cerveau.

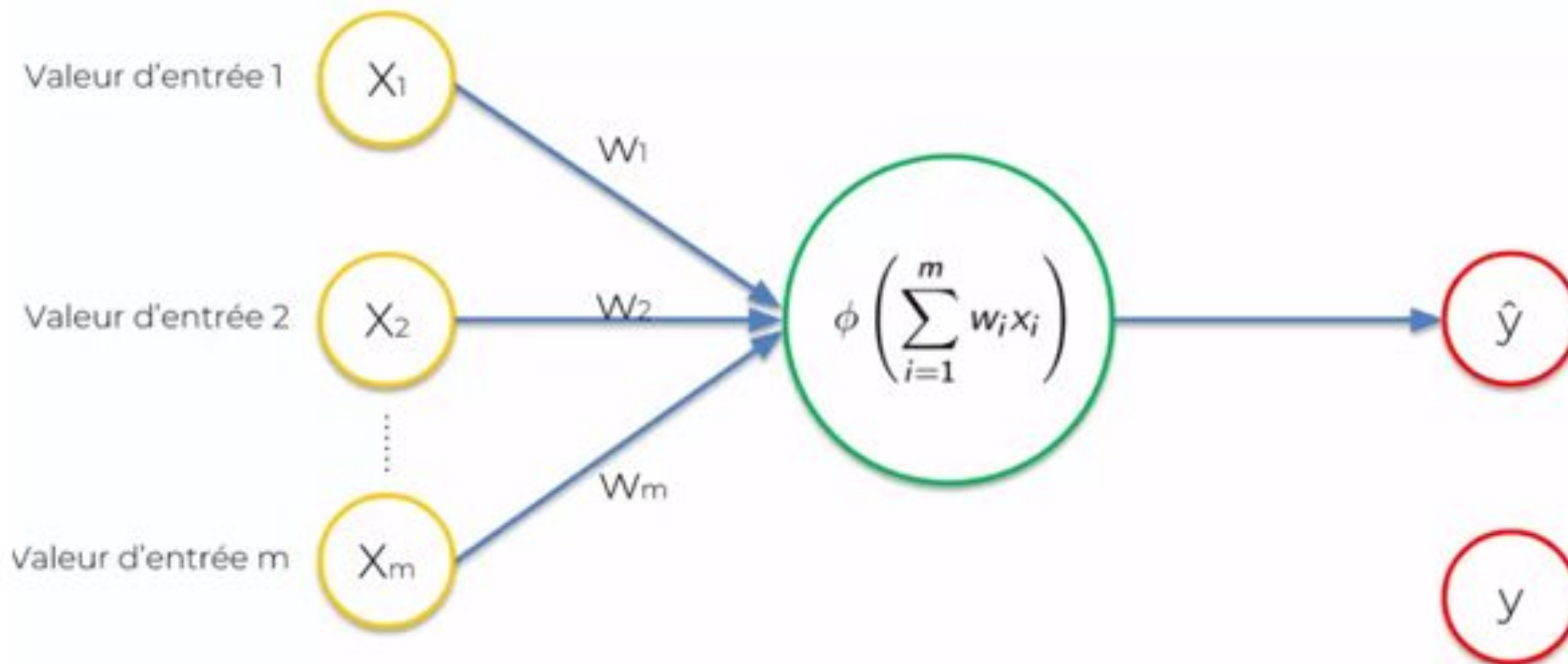


La formule du Perceptron

Réseau de neurone simple



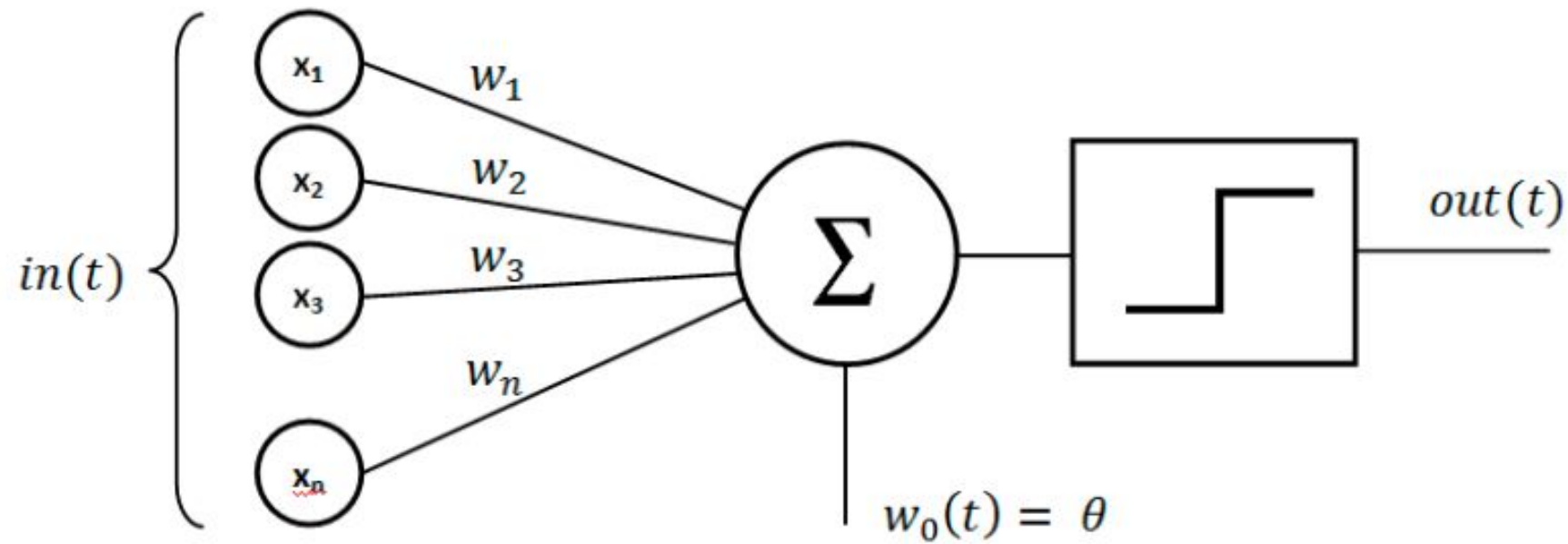
GNOMON[®]
DIGITAL



Perceptron



GNOMON[®]
DIGITAL



La formule du Perceptron

C'est quoi une fonction d'activation ?



GNOMON[®]
DIGITAL

- La fonction d'activation sert **avant tout à modifier** de manière **non-linéaire** les données. Cette non-linéarité permet de **modifier spatialement leur représentation**.
- Dit simplement, la **fonction d'activation** permet de **changer notre manière de voir** une donnée.

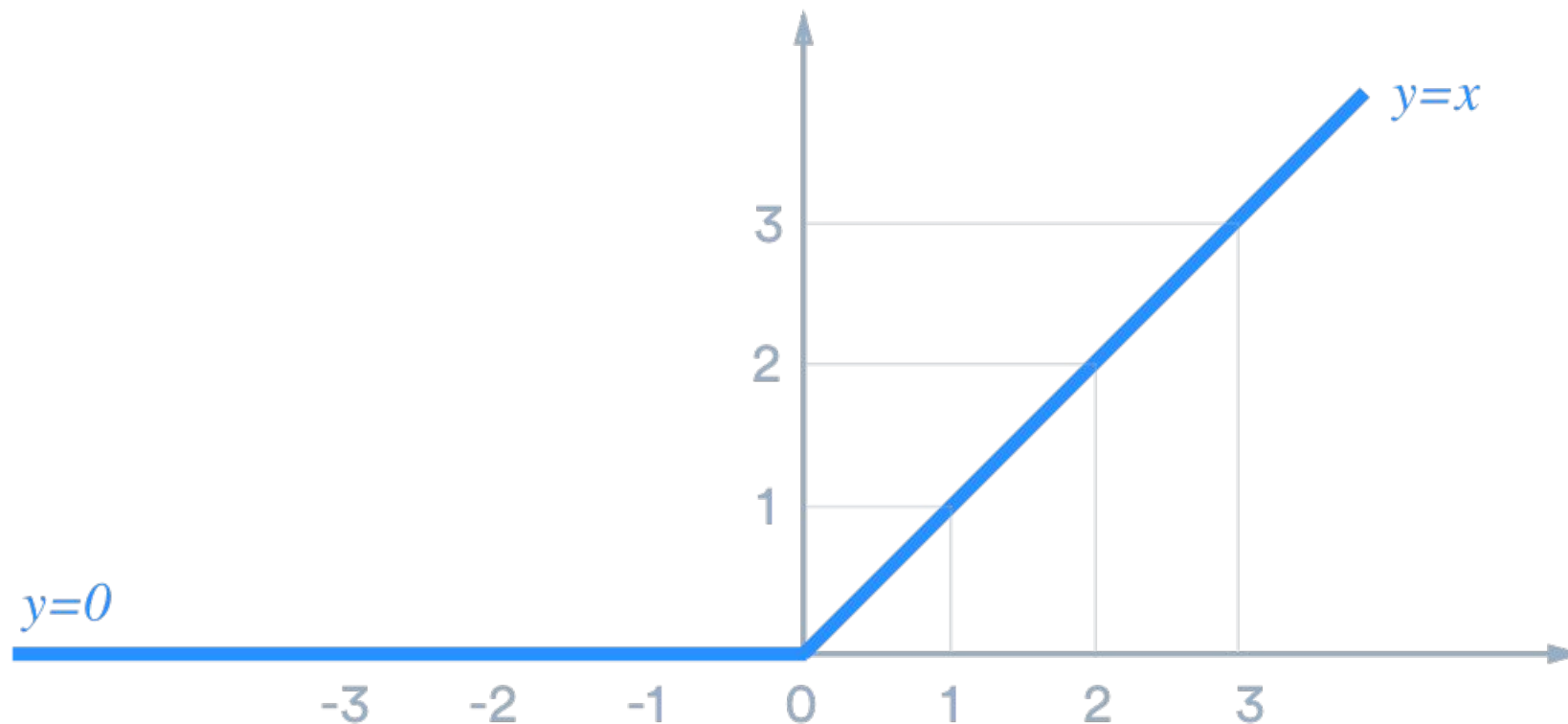
Linear Unit (ReLU)



GNOMON[®]
DIGITAL

La fonction **Linear Unit (ReLU)** est la fonction d'activation **la plus simple et la plus utilisée**.

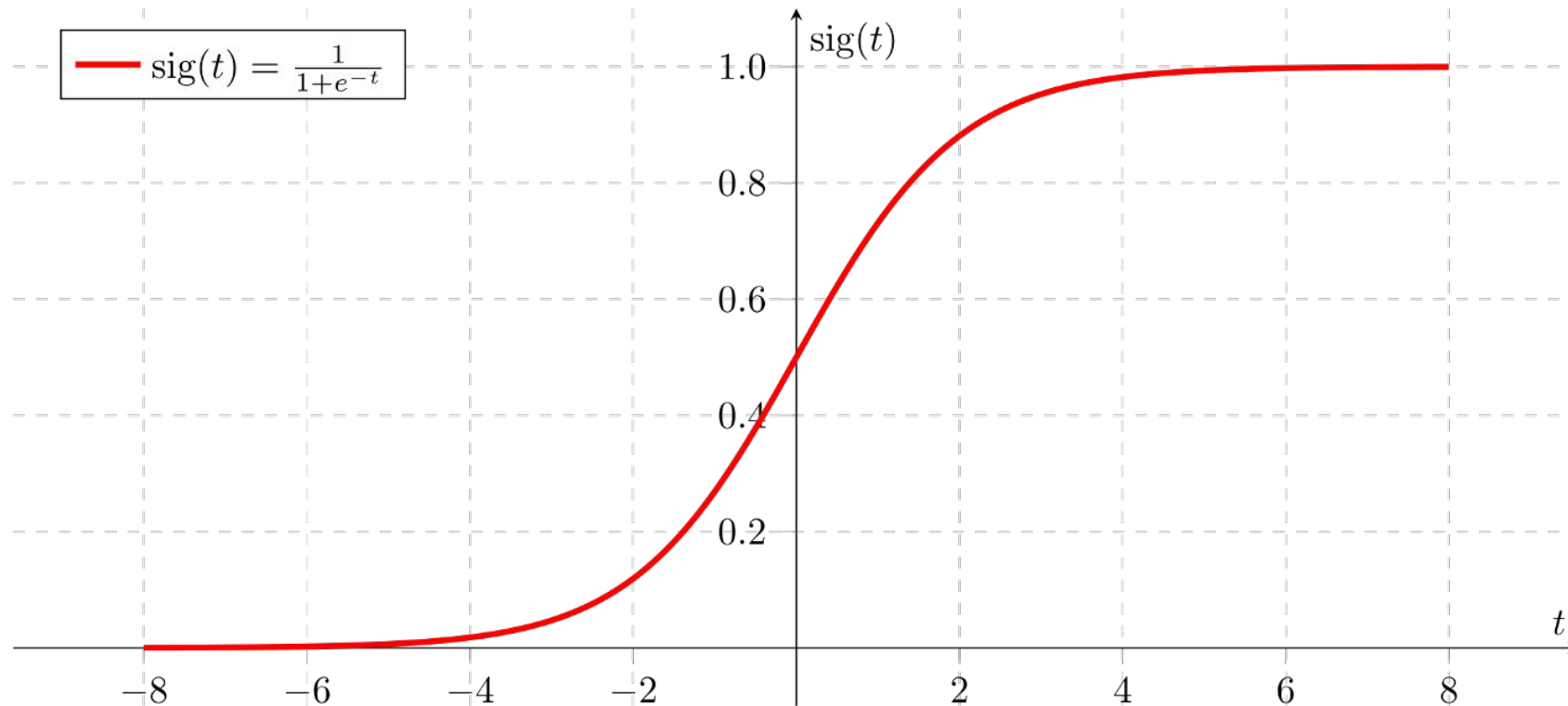
Elle donne **x** si **x** est supérieur à **0**, **0** sinon. Autrement dit, c'est le **maximum entre x et 0** :



Sigmoïde



La fonction **Sigmoïde** donne une valeur entre 0 et 1, une probabilité. Elle est donc très utilisée pour les **classification binaire**, lorsqu'un modèle doit déterminer seulement deux labels.

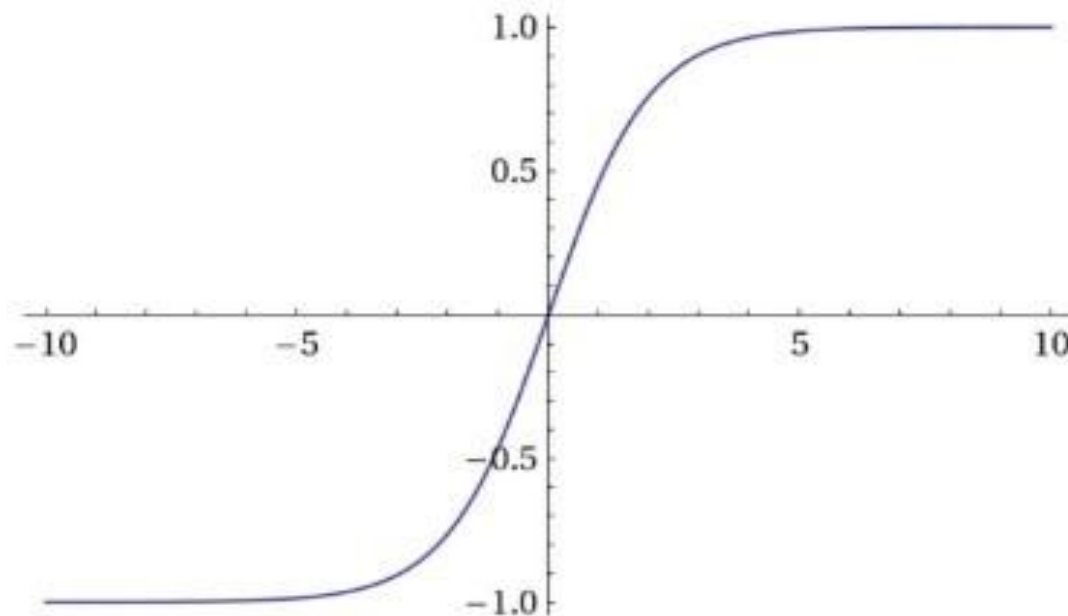


Softmax

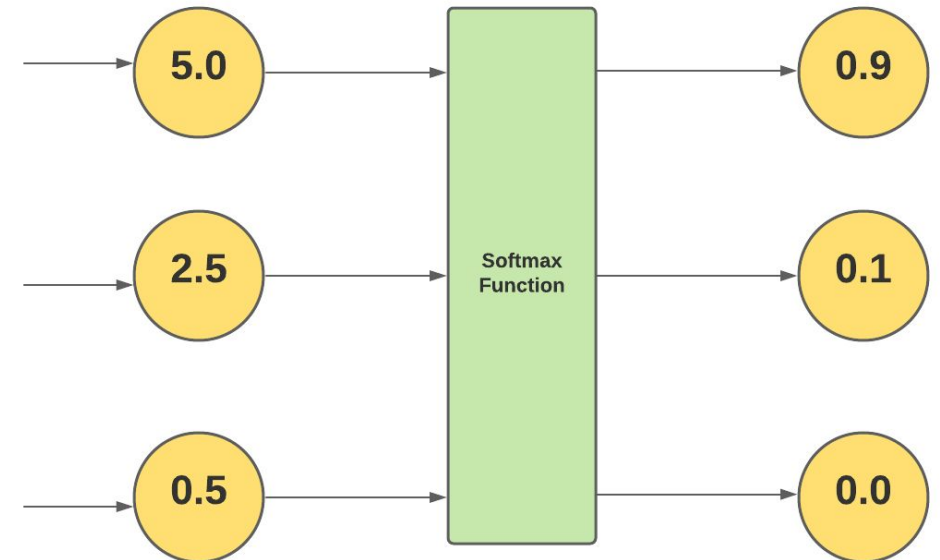


La fonction **Softmax** permet elle de transformer un vecteur réel en **vecteur de probabilité**.
On l'utilise souvent dans la couche finale d'un modèle de classification, notamment pour les **problèmes multiclasse**.

Softmax Activation Function



Model output
for a particular
input



$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Softmax



GNOMON[®]
DIGITAL

$$\sigma(\vec{z})_i = \frac{e^{z_i/T}}{\sum_{j=1}^K e^{z_j/T}}$$

Il est possible d'ajouter un coefficient T (température) sur toutes les sorties.
Quelle est l'impact sur les probabilités prédites?

Trois classes:

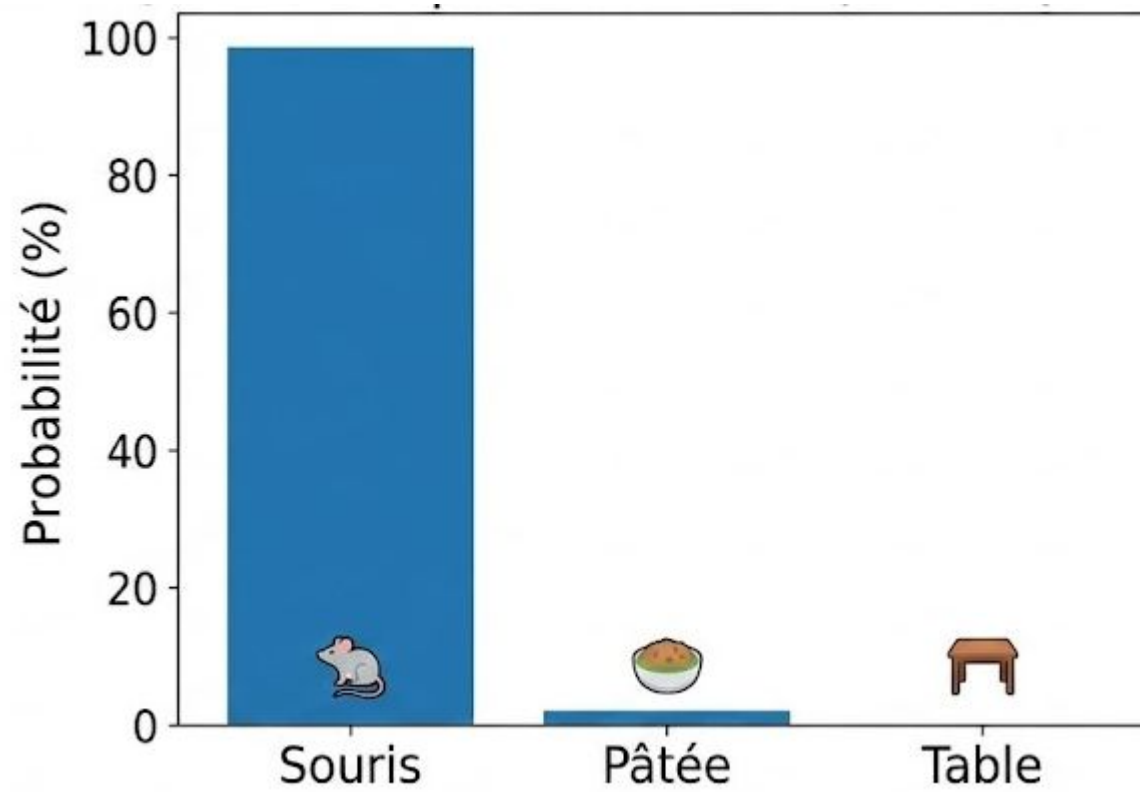
- Souris: Z1= 5.0
- Pâtée: Z2=4.0
- Table: Z3= 1.5

T=0.2/1/2.5

Softmax



GNOMON[®]
DIGITAL

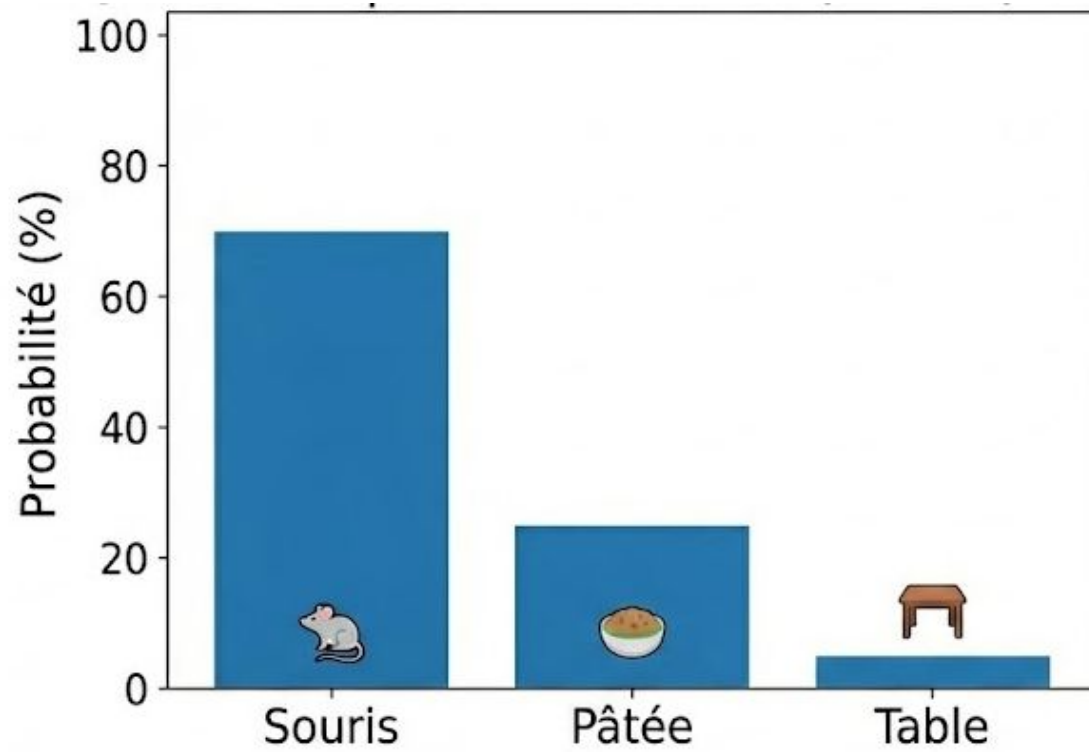


$T=0.2$

Softmax



GNOMON[®]
DIGITAL

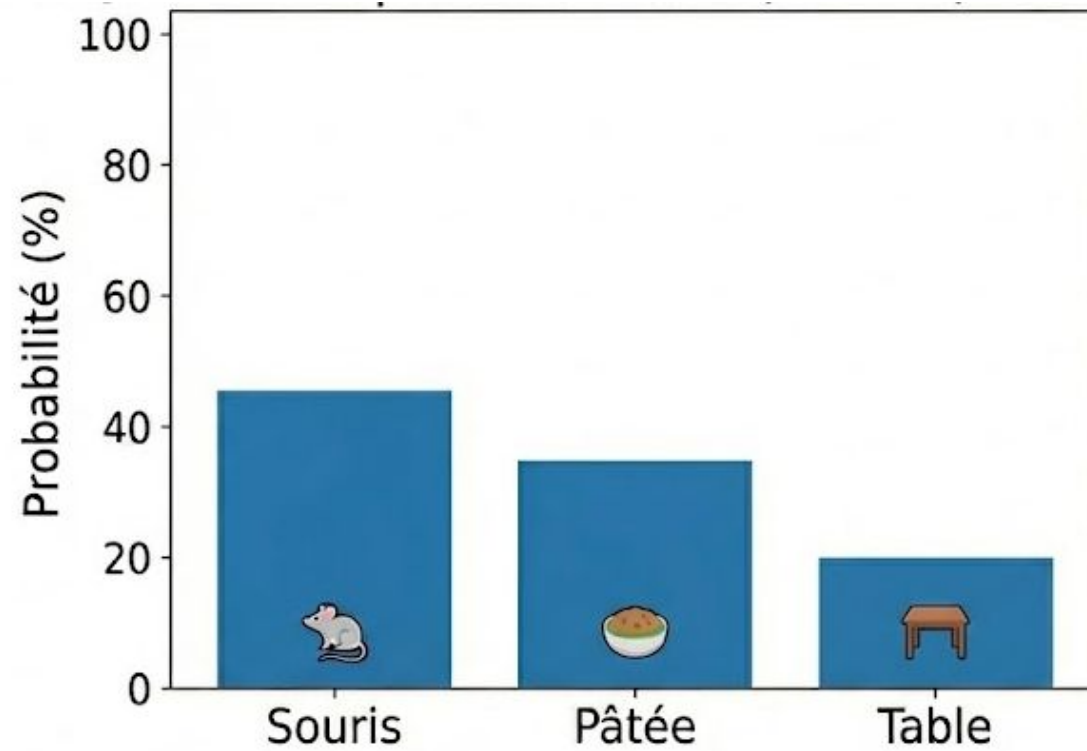


T=1

Softmax



GNOMON[®]
DIGITAL



$T=2.5$

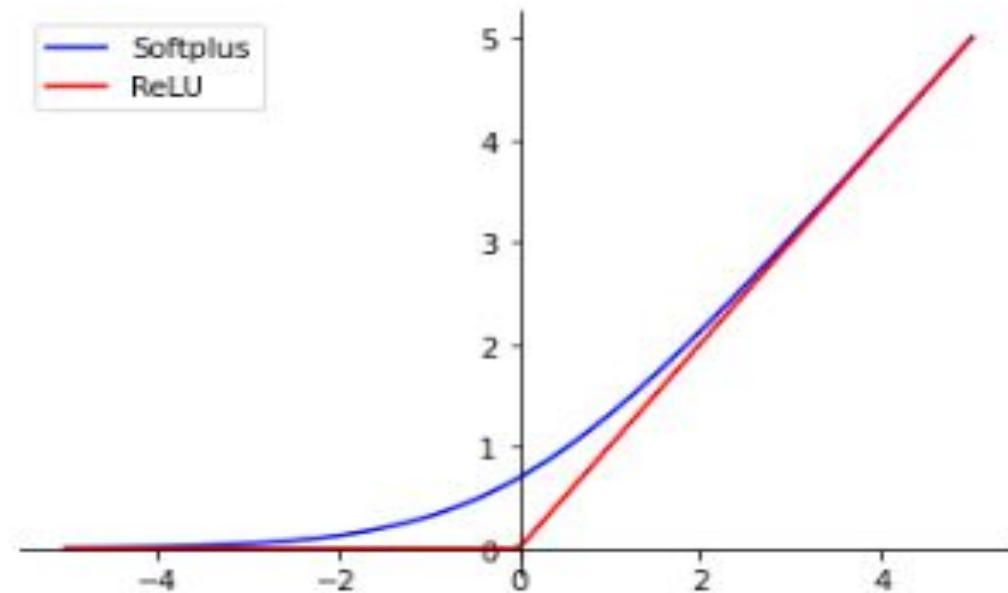
Softplus



GNOMON[®]
DIGITAL

La fonction **Softplus** est une **approximation** 'lisse' de la fonction **ReLU**. Cet aspect 'lisse' (ou soft) implique que la fonction est **différentiable**.

En fait, cette fonction est intéressante par sa dérivée. Quand on dérive **Softplus**, on obtient la **fonction logistique** $f(x) = 1/(1+\exp(-x))$. On rappelle que la dérivée est utilisée lors de la **Back-propagation** pour mettre à jour les poids



Fonction Softplus

Autres fonctions



GNOMON[®]
DIGITAL

Softsign : La fonction **Softsign** est utile pour **normaliser nos données** car elle permet d'avoir un **résultat entre -1 et 1** et garde en mémoire le **signe des données (positif ou négatif)**.
Autrement dit, les données sont **recentrées sur zéro** et **bornées par -1 et 1**.

Tanh : La fonction **tanh** est simplement la fonction de la **tangente hyperbolique**.

ELU : La fonction **Exponential Linear Unit (ELU)** est une **amélioration de ReLU** car elle permet d'avoir des **valeurs lisses** lorsque $x < 0$.

SELU : La fonction **Scaled Exponential Linear Unit (SELU)** est une optimisation de **ELU**.

Quelle fonction à utiliser ?



GNOMON[®]
DIGITAL

Type de problème	Fonction d'activation de la dernière couche
Classification Binaire	sigmoid
Classification multiclasse, à label unique	softmax
Classification multiclasse, multilabel	sigmoid
Régression	Linear

Architecture des réseaux de neurones artificiels



GNOMON[®]
DIGITAL

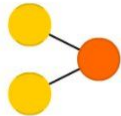
A mostly complete chart of

Neural Networks

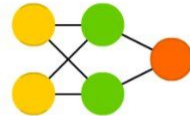
©2016 Fjodor van Veen - asimovinstitute.org

- Backfed Input Cell
- Input Cell
- Noisy Input Cell
- Hidden Cell
- Probabilistic Hidden Cell
- Spiking Hidden Cell
- Output Cell
- Match Input Output Cell
- Recurrent Cell
- Memory Cell
- Different Memory Cell
- Kernel
- Convolution or Pool

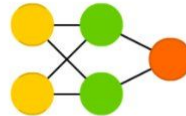
Perceptron (P)



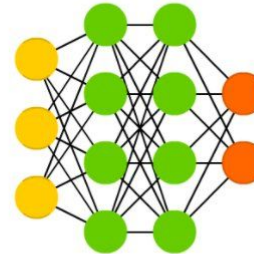
Feed Forward (FF)



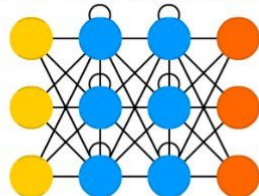
Radial Basis Network (RBF)



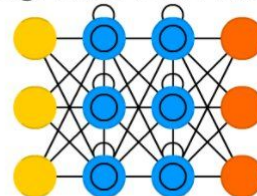
Deep Feed Forward (DFF)



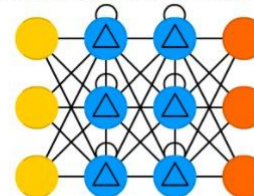
Recurrent Neural Network (RNN)



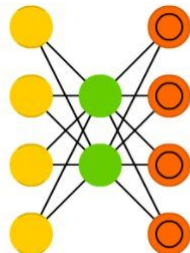
Long / Short Term Memory (LSTM)



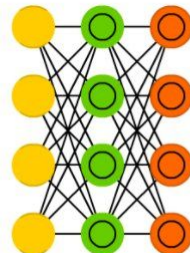
Gated Recurrent Unit (GRU)



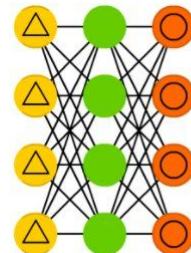
Auto Encoder (AE)



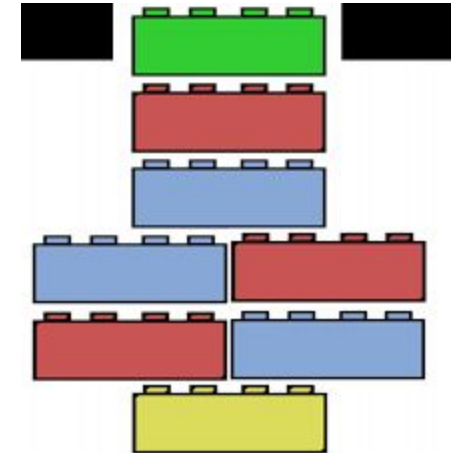
Variational AE (VAE)



Denoising AE (DAE)



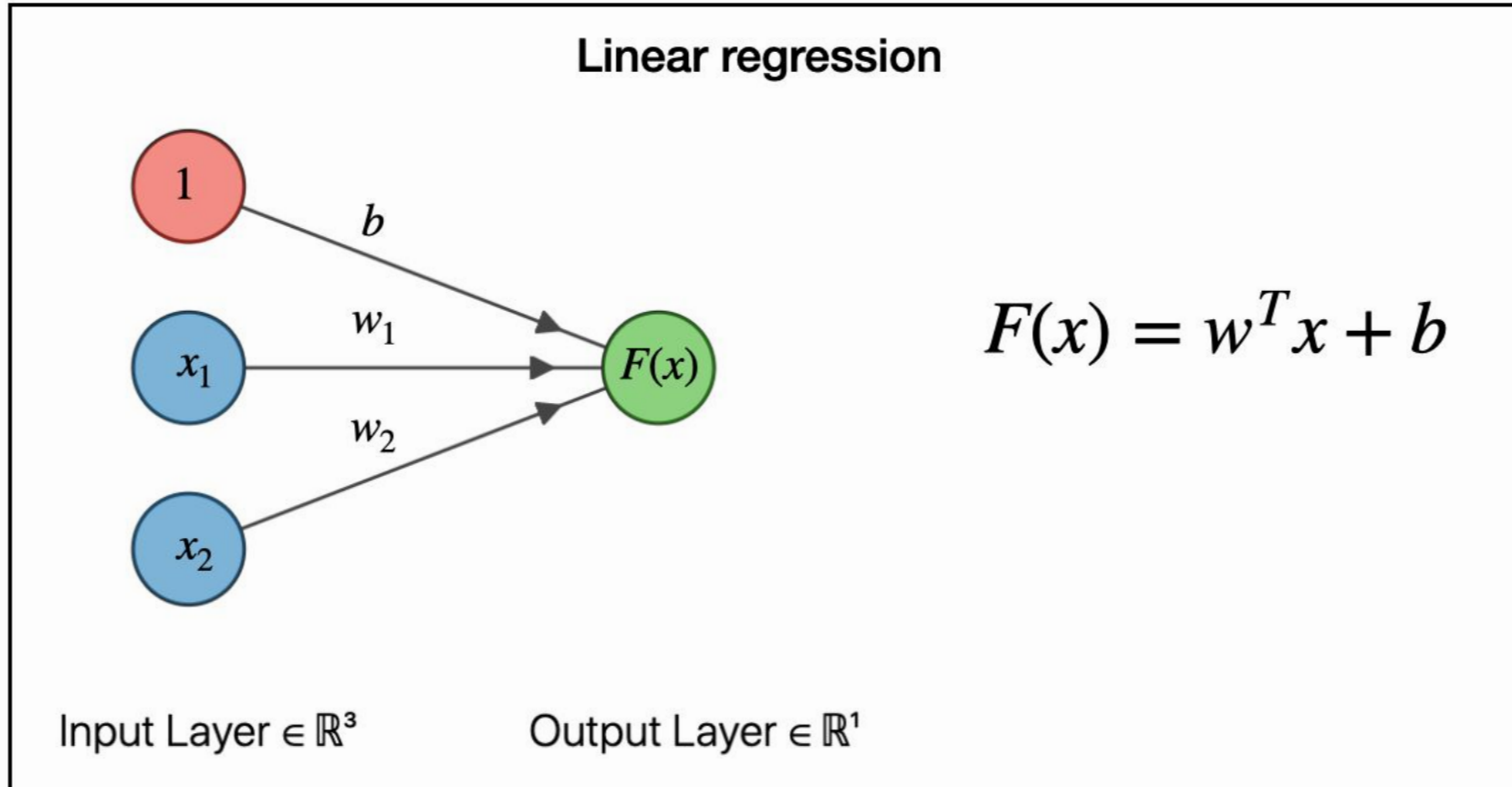
Sparse AE (SAE)



Lien avec le modèle de régression



GNOMON[®]
DIGITAL



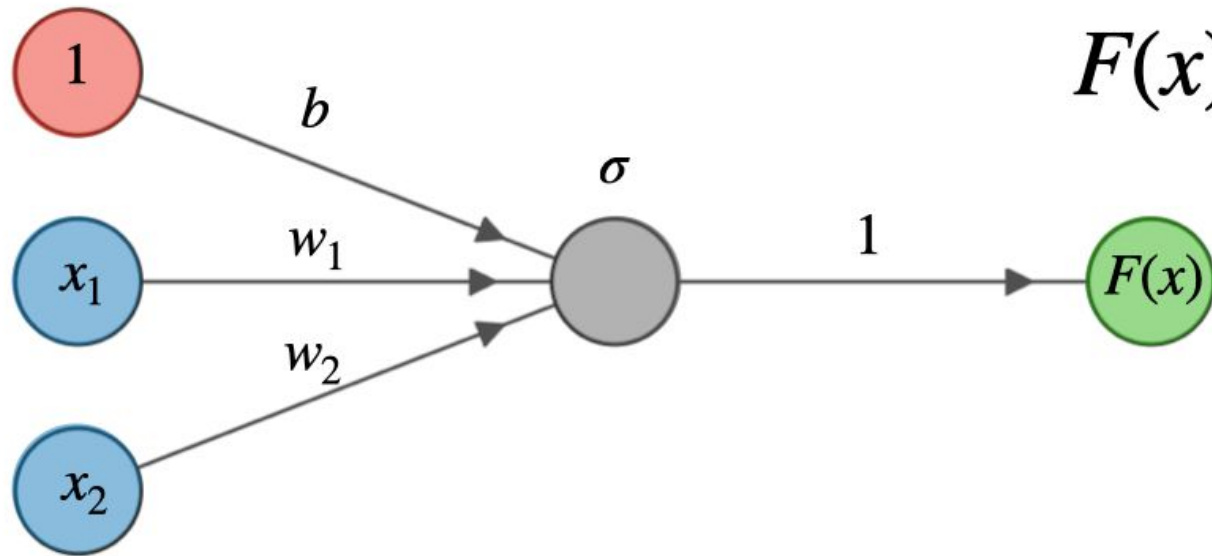
$$F(x) = w_1 \cdot x_1 + w_2 \cdot x_2 + 1 \cdot b$$

Perceptron



GNOMON[®]
DIGITAL

Logistic regression (generalized linear model)



$$F(x) = \sigma(w^T x + b)$$

$$\sigma(h) = \frac{1}{1 + e^{-h}}$$

Input Layer $\in \mathbb{R}^3$

Hidden Layer $\in \mathbb{R}^1$

Output Layer $\in \mathbb{R}^1$

La Descente de Gradient



La Descente de Gradient est un algorithme d'optimisation qui permet de trouver le minimum de n'importe quelle fonction convexe en convergeant progressivement vers celui-ci.

Vous êtes ici

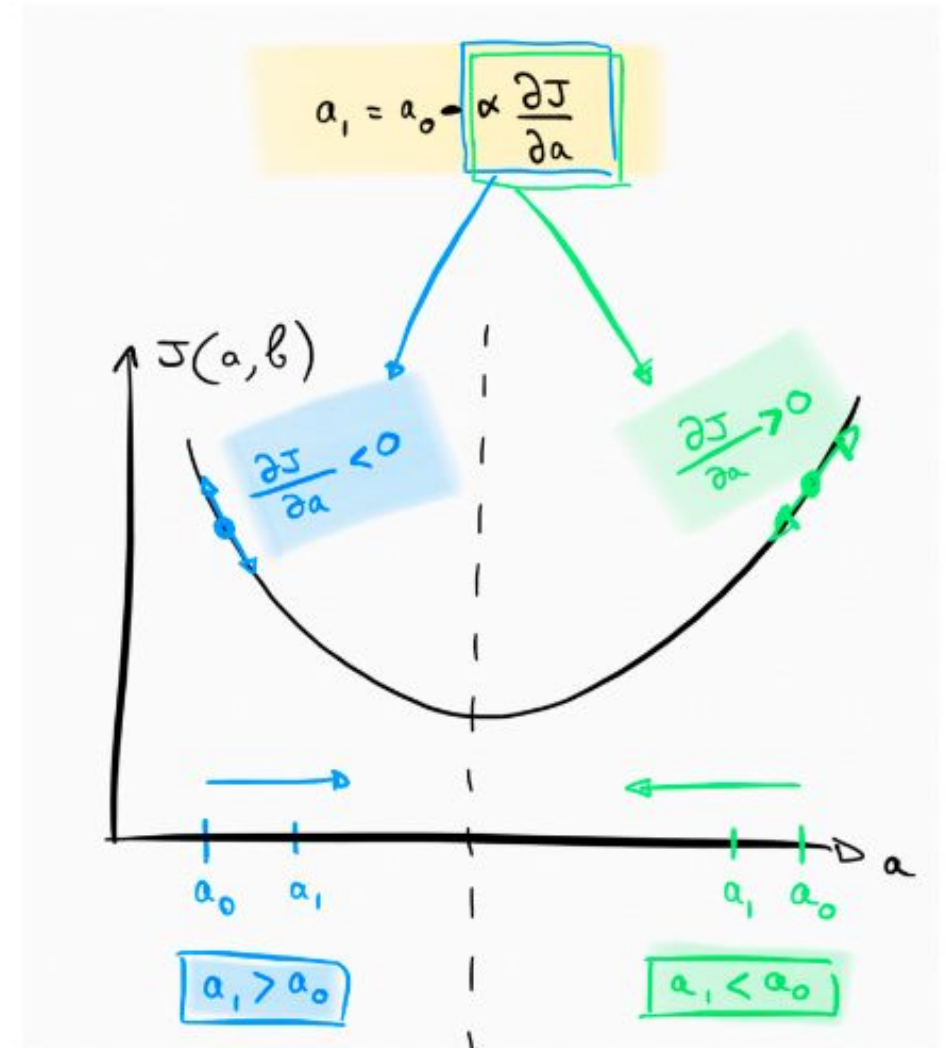


Descente de gradient



GNOMON[®]
DIGITAL

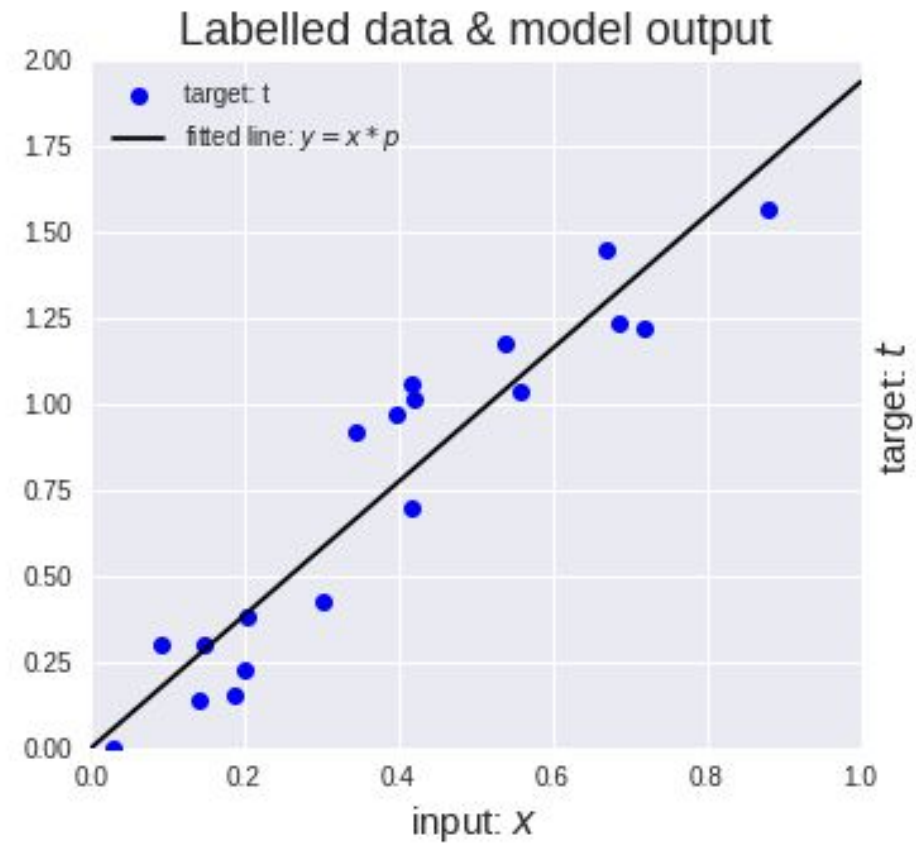
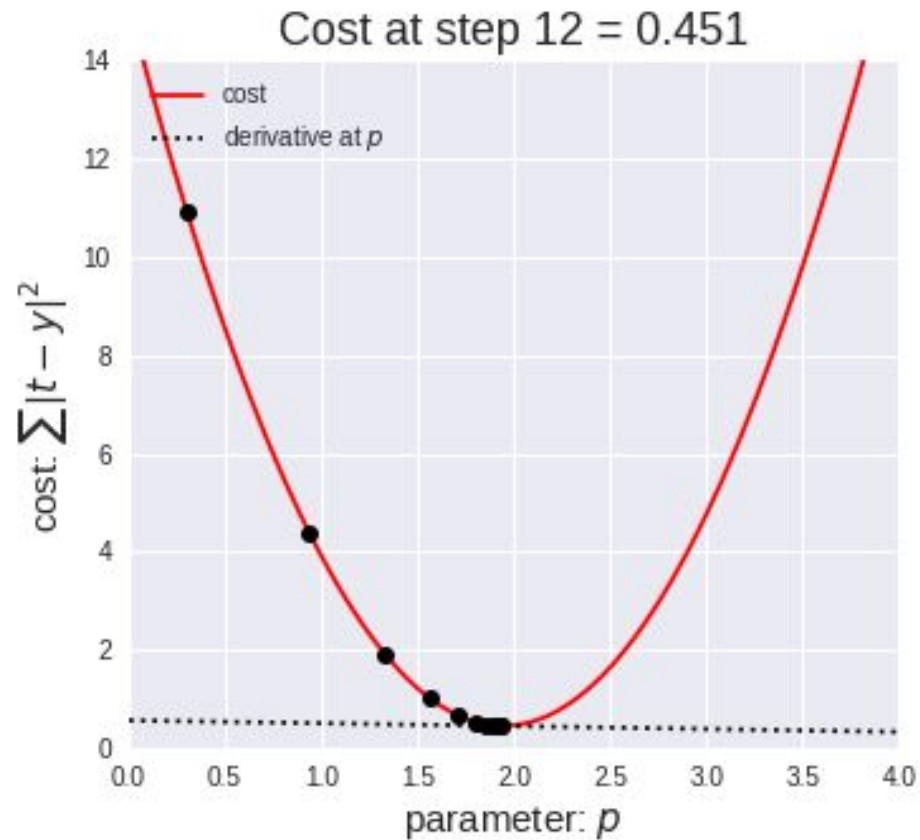
- Étape 1 : Calcul de la dérivée de la Fonction Coût
- Étape 2 : Mise à jour des paramètres du modèle
- Étape 3 : Répéter l'étape 1



Descente de gradient



GNOMON[®]
DIGITAL



Les hyperparameters



GNOMON[®]
DIGITAL

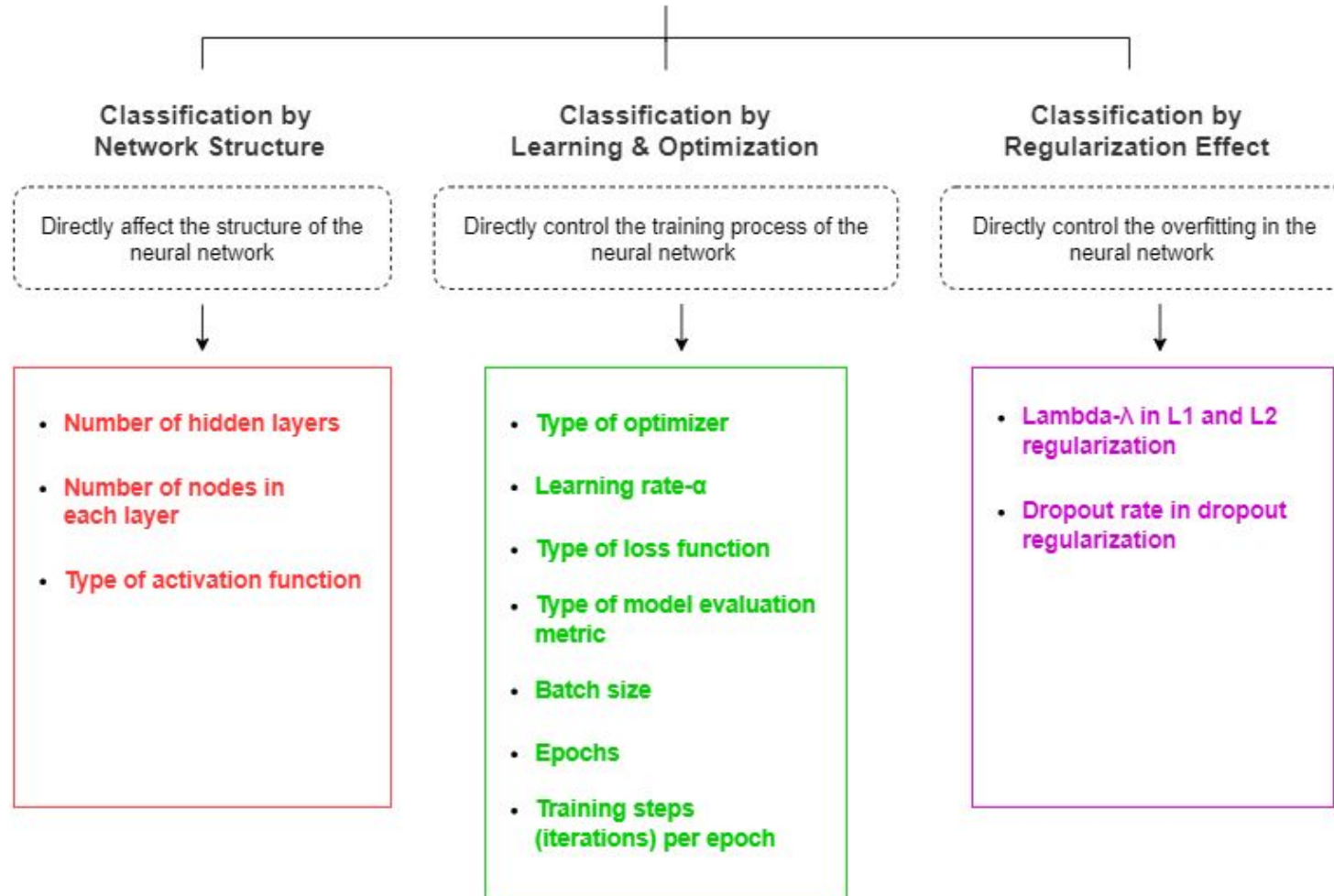
ML Algorithms	Hyperparameters	Definition	Defined Parameters
SVM (support vector machines)	Kernel type	the kernel function	RBF
	C	the penalty parameter	0.01–100
	σ	the bandwidth parameter	0.01–100
Cubist (regression tree)	committees	the number of model trees	1–100
	neighbors	the number of nearest neighbors	0–9
XGBoost (extreme gradient boosting)	booster	the type of model	gbtree
	max_depth	the depth of tree	3–10
	min_child_weight	the minimum sum of weights of all observations	0–5
	colsample_bytree	the number of variables supplied to a tree	0.5–1
	subsample	the number of samples supplied to a tree	0.5–1
	eta	learning rate	0.01–0.5
RF (random forest)	Mtry	the number of input variables	1–30
	Ntree	the number of trees	100–3000
ANN (artificial neural networks)	decay	learning rate	0.001–0.05
	size	the number of neurons in the hidden layer	1–10
DNN	hidden	the number of hidden layers	2–10

Les hyperparameters



GNOMON[®]
DIGITAL

Classification of Neural Network Hyperparameters

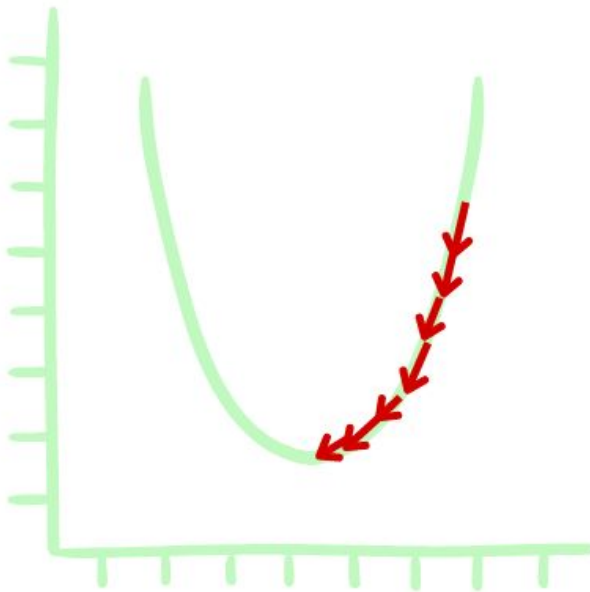


Taux d'apprentissage (Learning rate)



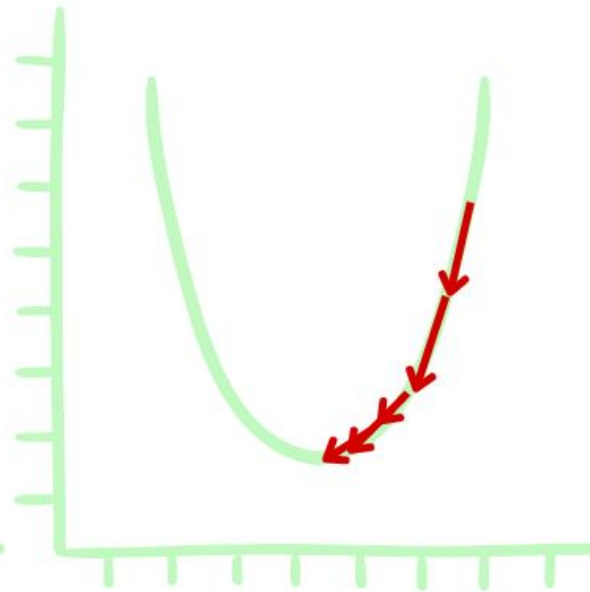
GNOMON[®]
DIGITAL

Too low



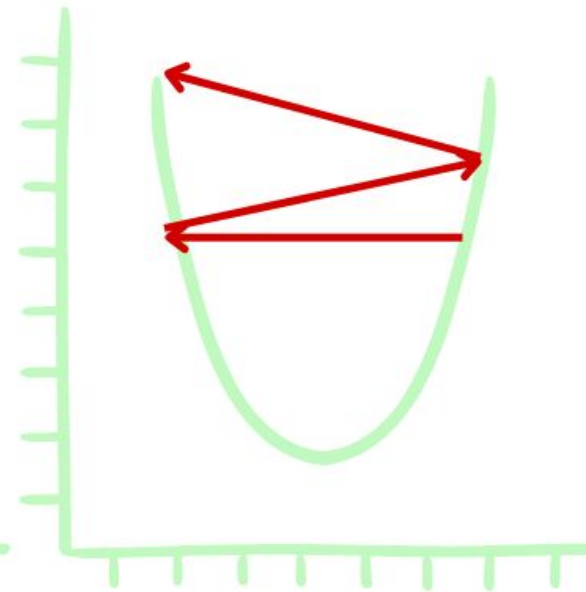
Smaller learning rate requires many updates, hence learning would be very slow

Just Right



Optimal learning rate smoothly reaches to the minima

Too High



Larger learning rate may lead to drastic updates and GD may diverge from the minima.

Fonction d'objectif (Loss Functions)



GNOMON[®]
DIGITAL

- Il définit la qualité d'apprentissage
- Il faut définir la fonction en fonction du problème
- Ex: Régression

Mean squared error

$$\text{MSE} = \frac{1}{n} \sum_{t=1}^n e_t^2$$

Root mean squared error

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{t=1}^n e_t^2}$$

Mean absolute error

$$\text{MAE} = \frac{1}{n} \sum_{t=1}^n |e_t|$$

Mean absolute percentage error

$$\text{MAPE} = \frac{100\%}{n} \sum_{t=1}^n \left| \frac{e_t}{y_t} \right|$$

Fonction d'objectif (Loss Functions)



GNOMON[®]
DIGITAL

Problem Type	Output Type	Final Activation Function	Loss Function
Regression	Numerical value	Linear	Mean Squared Error (MSE)
Classification	Binary outcome	Sigmoid	Binary Cross Entropy
Classification	Single label, multiple classes	Softmax	Cross Entropy
Classification	Multiple labels, multiple classes	Sigmoid	Binary Cross Entropy

$$Entropy = -p_i \log_b(p_i)$$

$$CrossEntropy = - \sum_{i=1}^{i=n} Y_i \log_b(p_i)$$

Fonction de coût (Loss Functions)



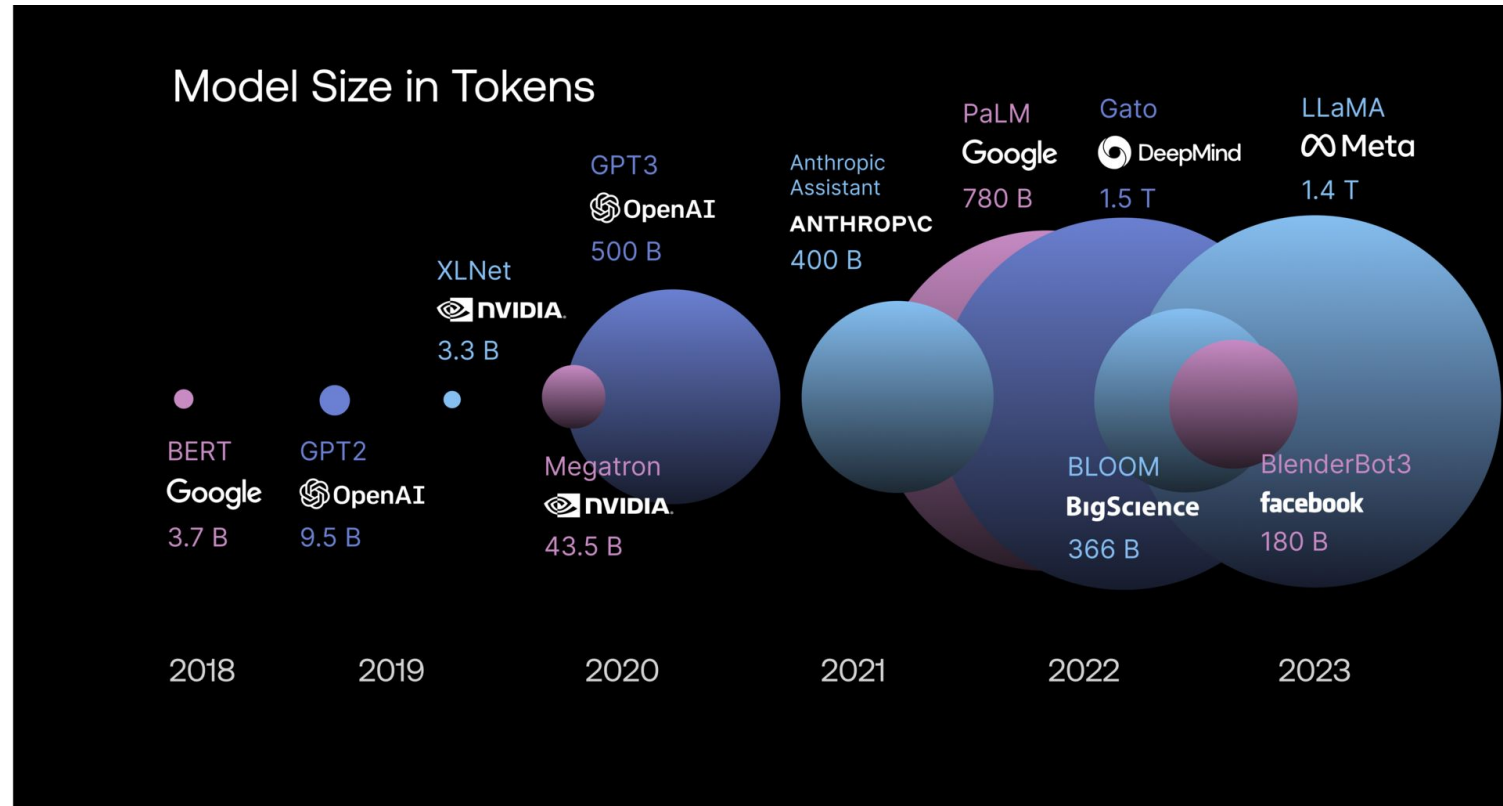
GNOMON[®]
DIGITAL

- **Binary Cross-Entropy** : Fonction la plus connue, définissant comme mesure de la différence entre deux probabilité de distribution pour une variable prise aléatoirement.
- **Categorical Cross-Entropy** : Utilisé pour de la classification multi-classes, mais en souhaitant un seul label en sortie.
- **Weighted Cross-Entropy** : C'est une variante de la Binary Cross-Entropy. Elle permet de rectifier la faiblesse de la BCE pour les datasets déséquilibrés, en ajoutant des poids selon des coefficients, pour les exemples positifs.
- **Balanced Cross-Entropy** : Elle est similaire à la Weighted Cross-Entropy. A la différence prêt, qu'elle pénalise en plus des exemples positives, les exemples négatifs

Epoch vs batch size



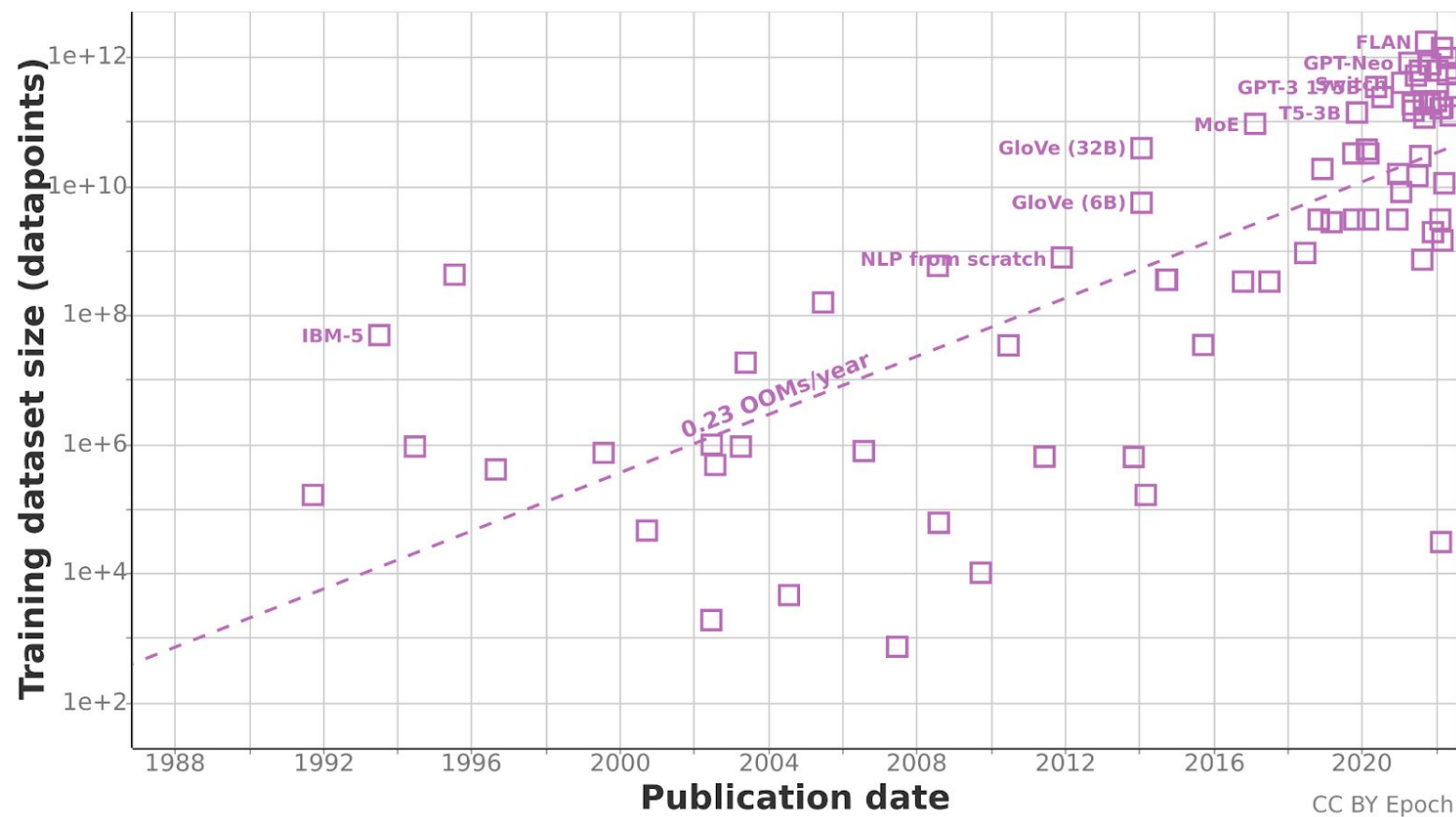
GNOMON[®]
DIGITAL



Epoch vs batch size



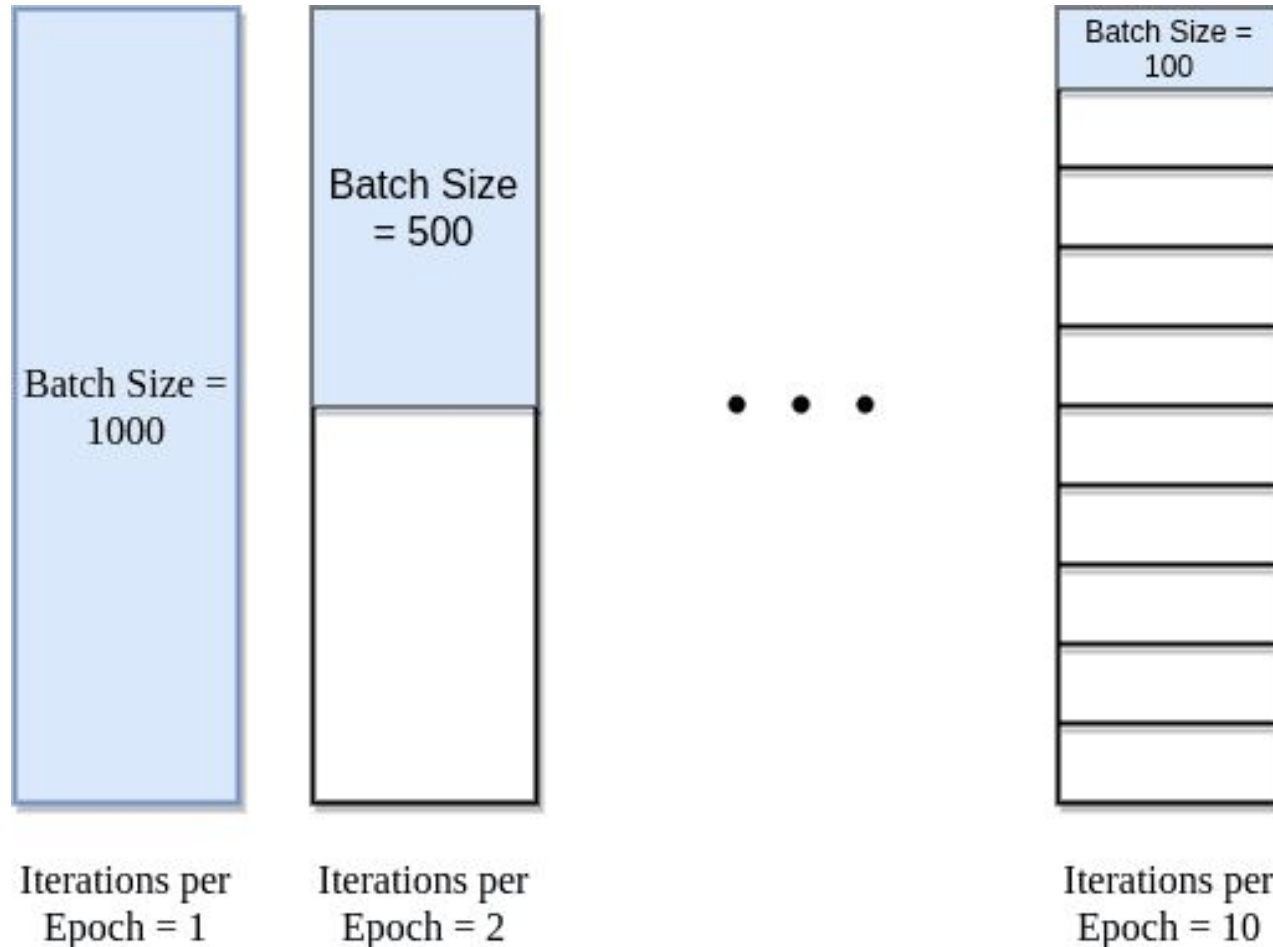
GNOMON[®]
DIGITAL



Epoch vs batch size



GNOMON[®]
DIGITAL



Epoch vs batch size



GNOMON[®]
DIGITAL

- L'epoch est un hyperparamètre qui définit le nombre de fois que l'algorithme d'apprentissage s'entraînera sur l'ensemble des données d'apprentissage.
- Une epoch signifie que chaque échantillon de l'ensemble de données d'apprentissage a eu l'occasion de mettre à jour les paramètres du modèle interne.
- Une epoch est composée d'un ou plusieurs lots ou batch size.
- On prend généralement une epoch de 25, 50, 100 ou 200. mais ce nombre peut varier grandement suivant la problématique ou la taille du dataset.

- Le batch size est un hyperparamètre qui définit le nombre d'échantillons à traiter avant de mettre à jour les paramètres du modèle interne.
- L'utilisation de batch permet de consommer moins de mémoire lors des phases d'entraînement.
- les tailles de lots populaires incluent 32, 64 et 128 échantillons

Typologie de modèles



- Il existe deux types de modèles : les modèles **discriminants** et **génératifs**.
- Les modèles génératifs et discriminants sont deux approches différentes qui sont largement étudiées dans la tâche de classification. Ils suivent un itinéraire différent les uns des autres pour atteindre le résultat final.
- Les modèles discriminants sont très populaires et sont utilisés de manière plus comparative pour effectuer la tâche car ils donnent de meilleurs résultats lorsqu'ils sont fournis avec une bonne quantité de données.

Modèle discriminant



- La tâche d'un modèle discriminant est simple, si on lui montre des données de différentes classes, il devrait être capable de les discriminer.
- Par exemple, si je montre au modèle un ensemble d'images de chien et de chat, il devrait pouvoir dire ce qu'est un chien et ce qu'est un chat en utilisant des caractéristiques discriminantes telles que la forme des yeux, la taille des oreilles, etc.

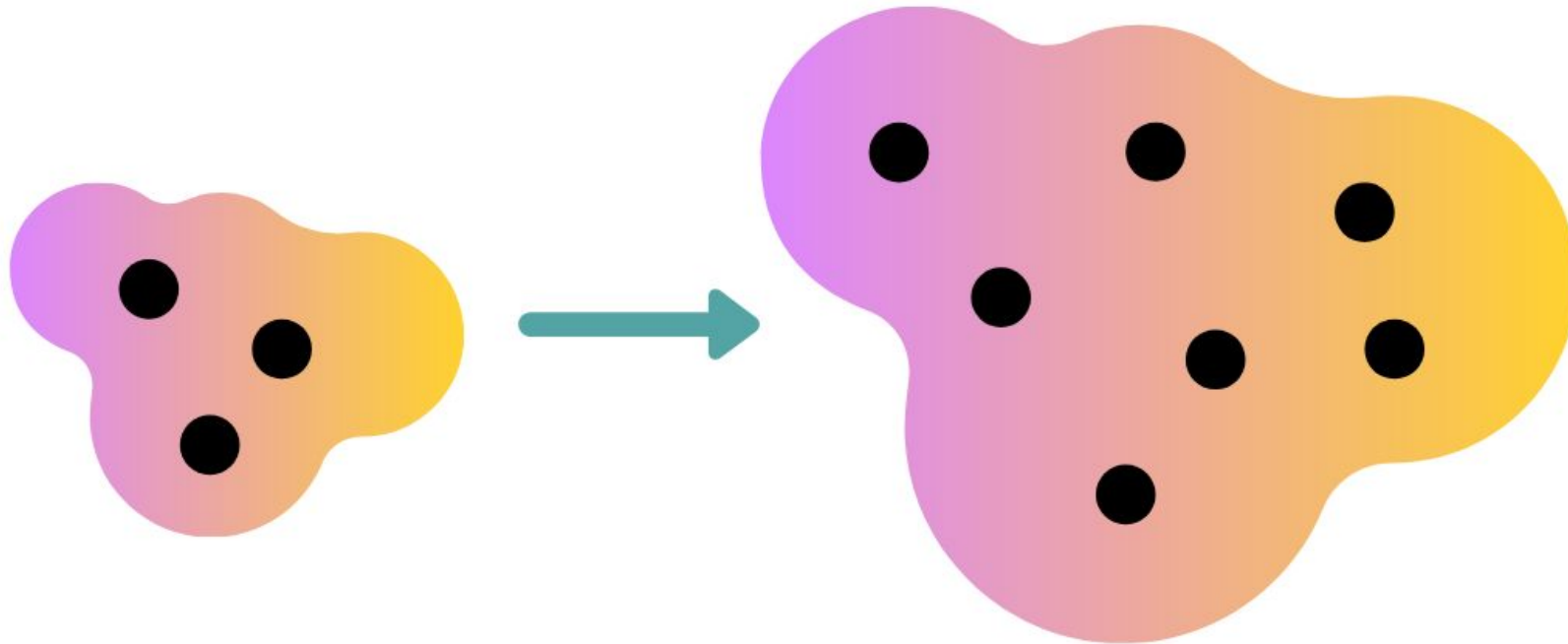
Generative Models



GNOMON[®]
DIGITAL



ChatGPT



Generating new data using past data

Generative Models



GNOMON[®]
DIGITAL



Generative Models



GNOMON[®]
DIGITAL

- Les modèles génératifs représentent une catégorie à part. Contrairement aux modèles discriminatifs qui associent les valeurs d'entrée à des étiquettes de sortie, ils ont pour objectif de générer de nouvelles données suivant des règles et des conditions spécifiques.
- Une explication simplifiée serait que les modèles génératifs sont capables d'apprendre les caractéristiques communes d'une collection de données et ensuite, de générer des données similaires



Deep
learning
framework

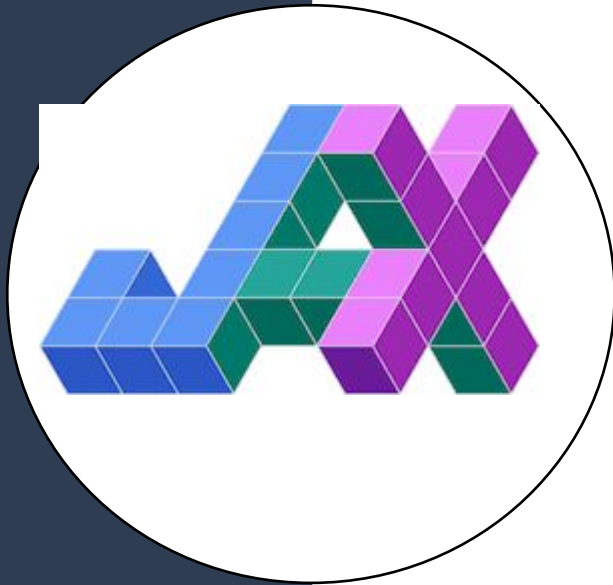




- PyTorch est une bibliothèque logicielle Python open source d'apprentissage machine qui s'appuie sur Torch développée par Meta
- PyTorch permet d'effectuer des calculs tensoriels nécessaires notamment pour du deep learning. Ces calculs sont optimisés et effectués soit par CPU ou GPU



- TensorFlow est un outil open source d'apprentissage automatique développé par Google.
- TensorFlow peut être lancé sur plusieurs CPU et GPU (avec des extensions optionnelles telles que CUDA ou SYCL)



- Jax est un outil open source d'apprentissage automatique développé par Google.
- Il est conçu pour suivre au plus près la structure et le flux de travail de NumPy et fonctionne avec divers frameworks existants tels que TensorFlow et PyTorch .

Ne confondre pas
avec ce Jax



Que choisir?



GNOMON[®]
DIGITAL

- TensorFlow et PyTorch offrent tous deux des abstractions utiles qui facilitent le développement de modèles en réduisant le code passe-partout
- PyTorch a une approche plus "pythonique" et est orienté objet, tandis que TensorFlow offre une variété d'options.
- TensorFlow offre une meilleure visualisation, ce qui permet aux développeurs de mieux déboguer
- Dans le domaine du parallélisme des données, PyTorch obtient des performances optimales en s'appuyant sur un support natif pour l'exécution asynchrone via Python.

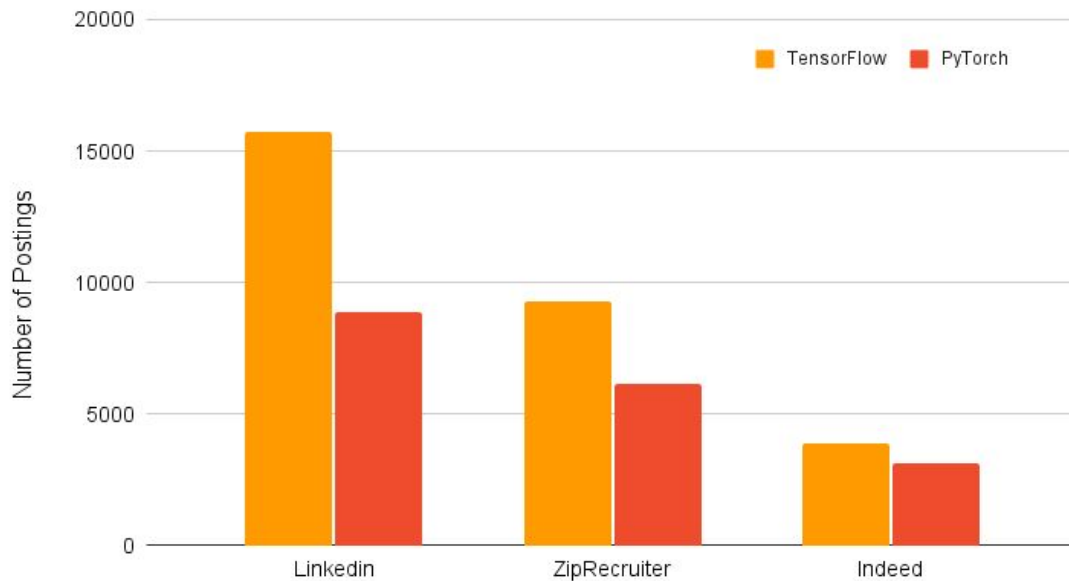


PyTorch vs TF

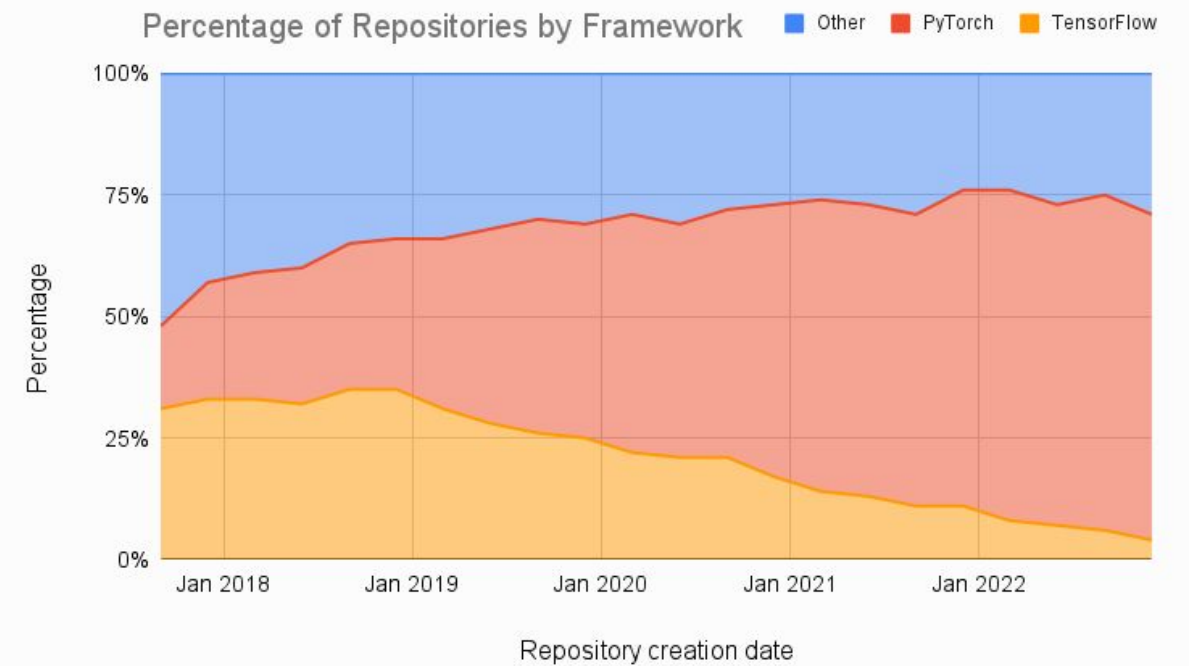


GNOMON[®]
DIGITAL

Number of Job Postings by Framework



Percentage of Repositories by Framework

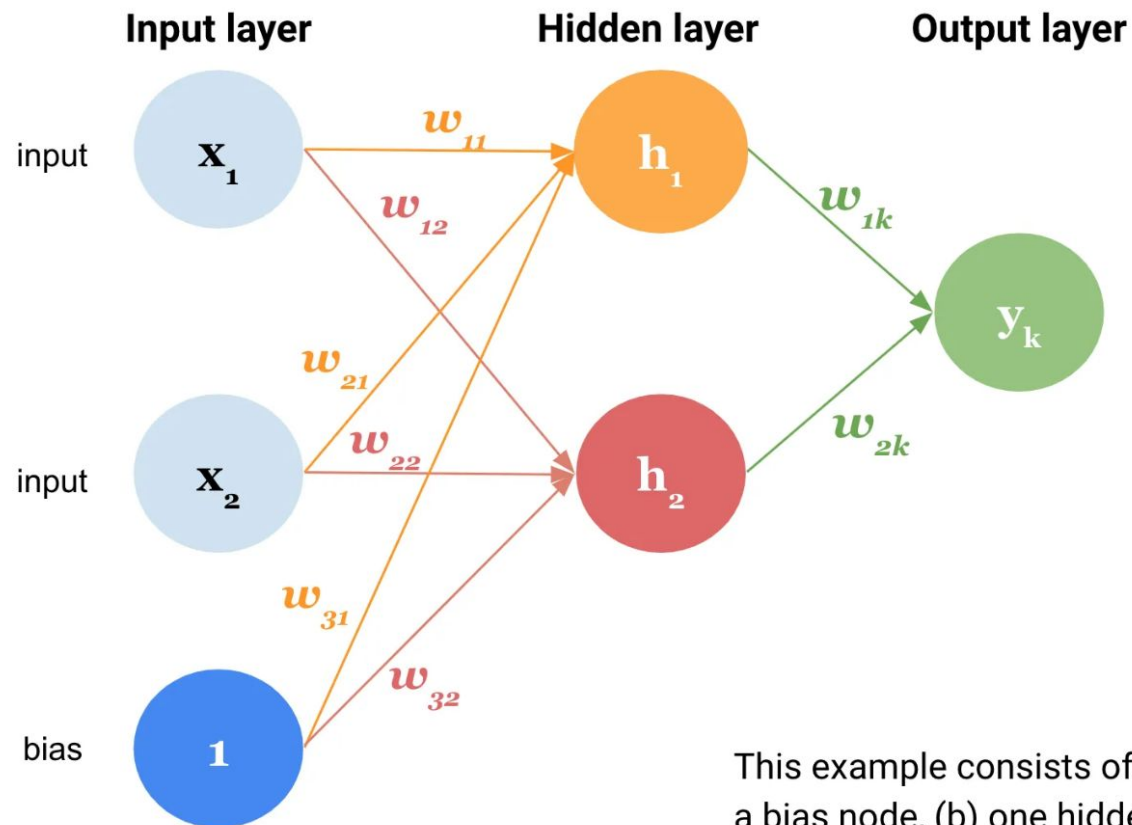




L'architecture classique:

- MLP
- CNN
- Transfer learning
- RNN
- Embedding
- Sequence-to-sequence
- Deep generative model

Illustrative example of Multilayer perceptron, a Feedforward neural network



x_1, x_2 : input data features
 w_{ij} : weights of the network
 h_1, h_2 : nodes in the hidden layer
 y_k : output variable

© AIML.com Research

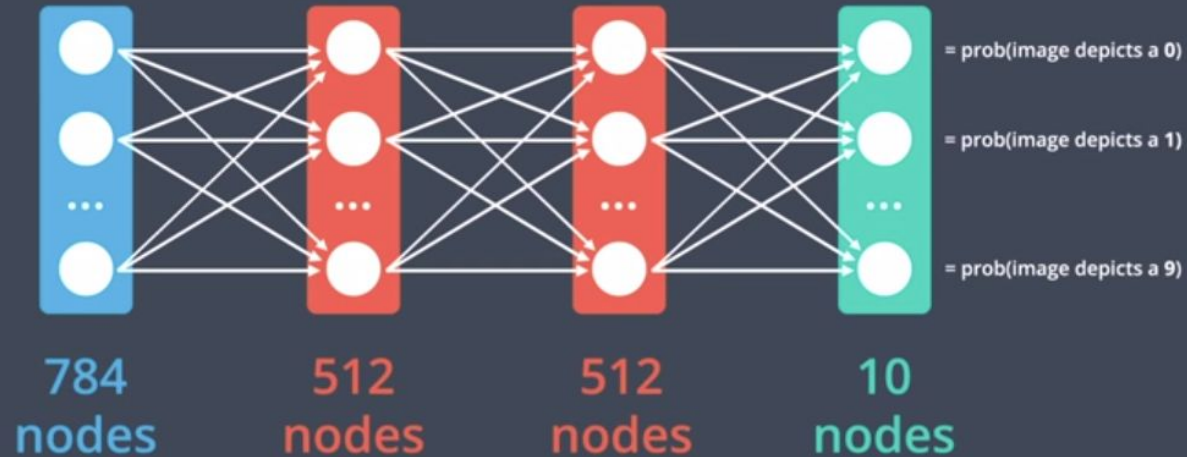
This example consists of: (a) an input layer with two input nodes and a bias node, (b) one hidden layer with two neurons, and (c) an output layer with one neuron

MLP



GNOMON[®]
DIGITAL

Input Layer Hidden Layers Output Layer



```
from keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten

# define the model
model = Sequential()
model.add(Flatten(input_shape=X_train.shape[1:]))
model.add(Dense(512, activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(512, activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(10, activation='softmax'))

# summarize the model
model.summary()
```

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 784)	0
dense_1 (Dense)	(None, 512)	401920
dropout_1 (Dropout)	(None, 512)	0
dense_2 (Dense)	(None, 512)	262656
dropout_2 (Dropout)	(None, 512)	0
dense_3 (Dense)	(None, 10)	5130

Total params: 669,706.0

Trainable params: 669,706.0

Non-trainable params: 0.0

MLP



One-Hot Label
→ 0001000000

Model Prediction 1:

→ 000.1 0000.9 0

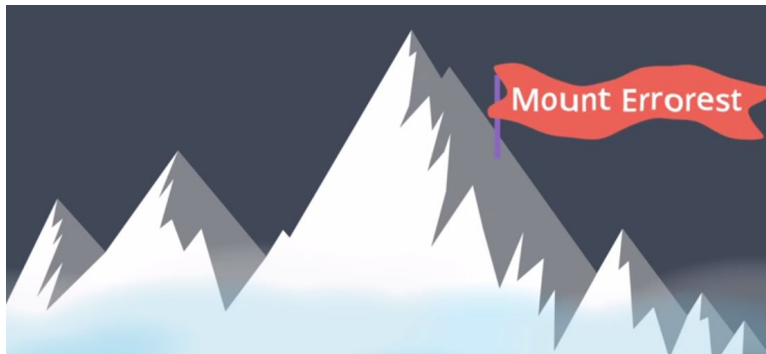
loss = $-\log(.1) = 2.30$

Model Prediction 2:

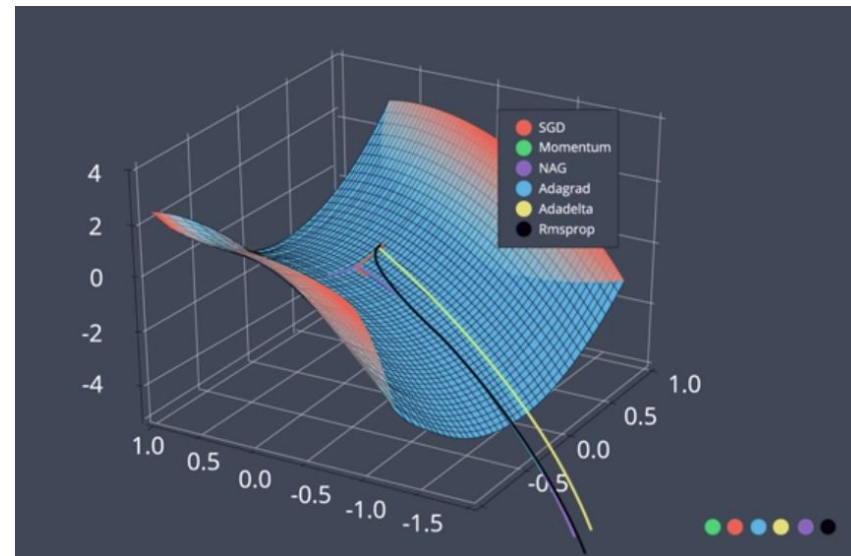
000.9 0000.1 0

loss = $-\log(.9) = .105$

LOSS = $\begin{cases} \text{LOWER :) } & \text{When label and prediction } \textit{agree} \\ \text{HIGHER :(} & \text{When label and prediction } \textit{disagree} \end{cases}$



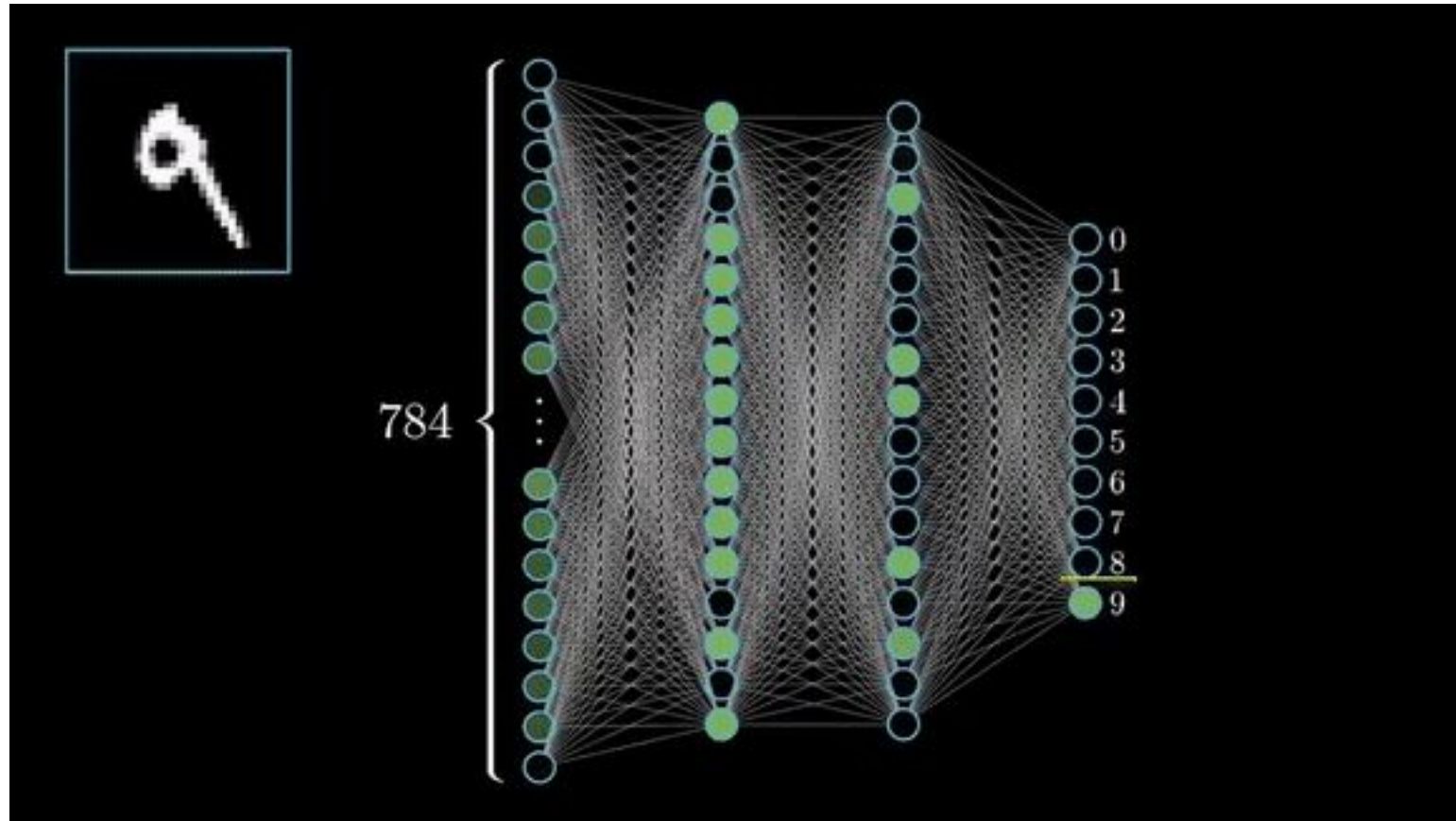
Categorical Cross-Entropy



MLP



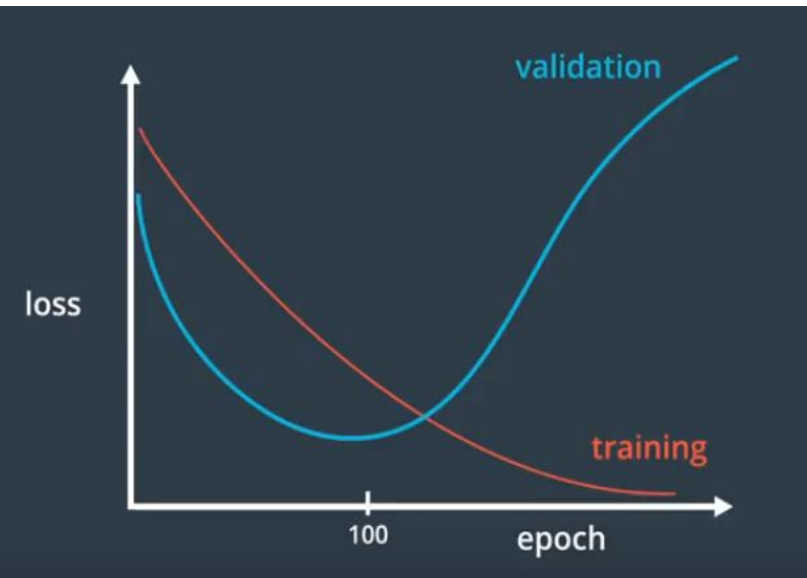
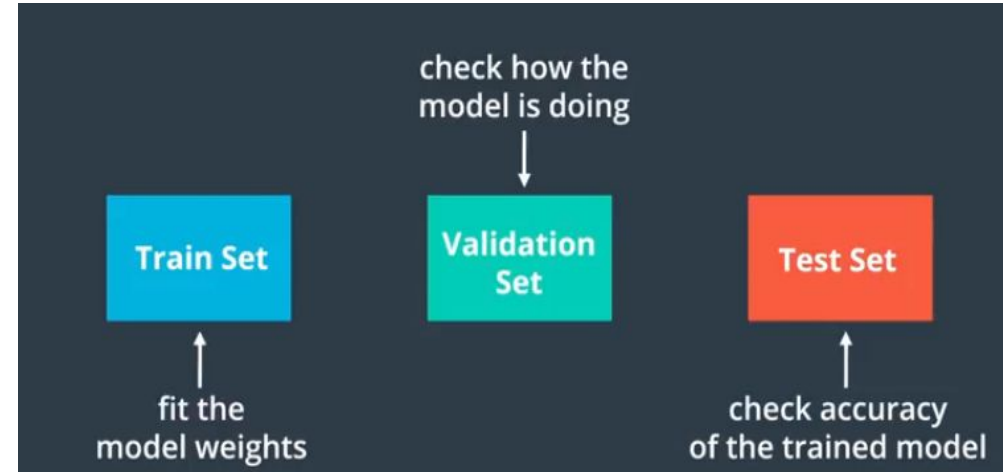
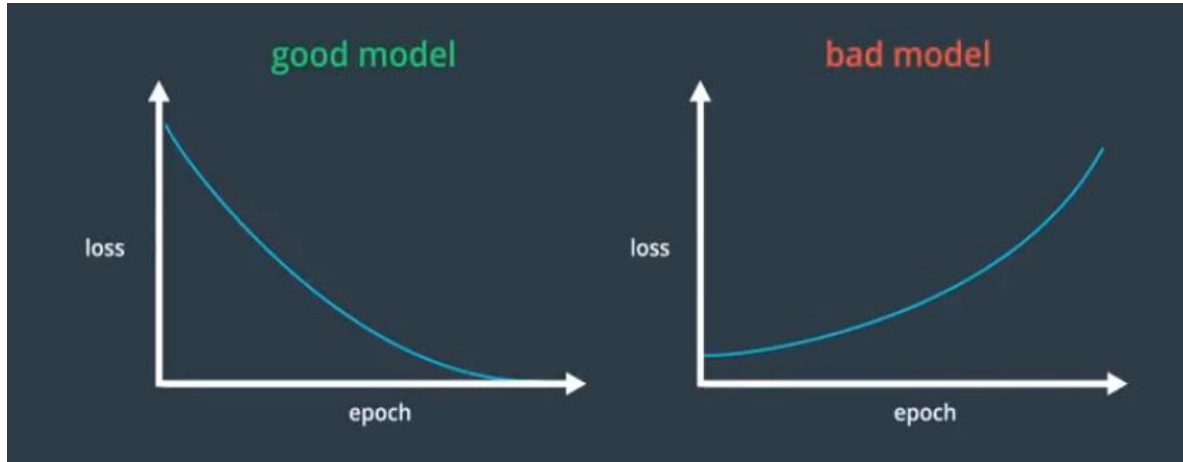
GNOMON[®]
DIGITAL



MLP



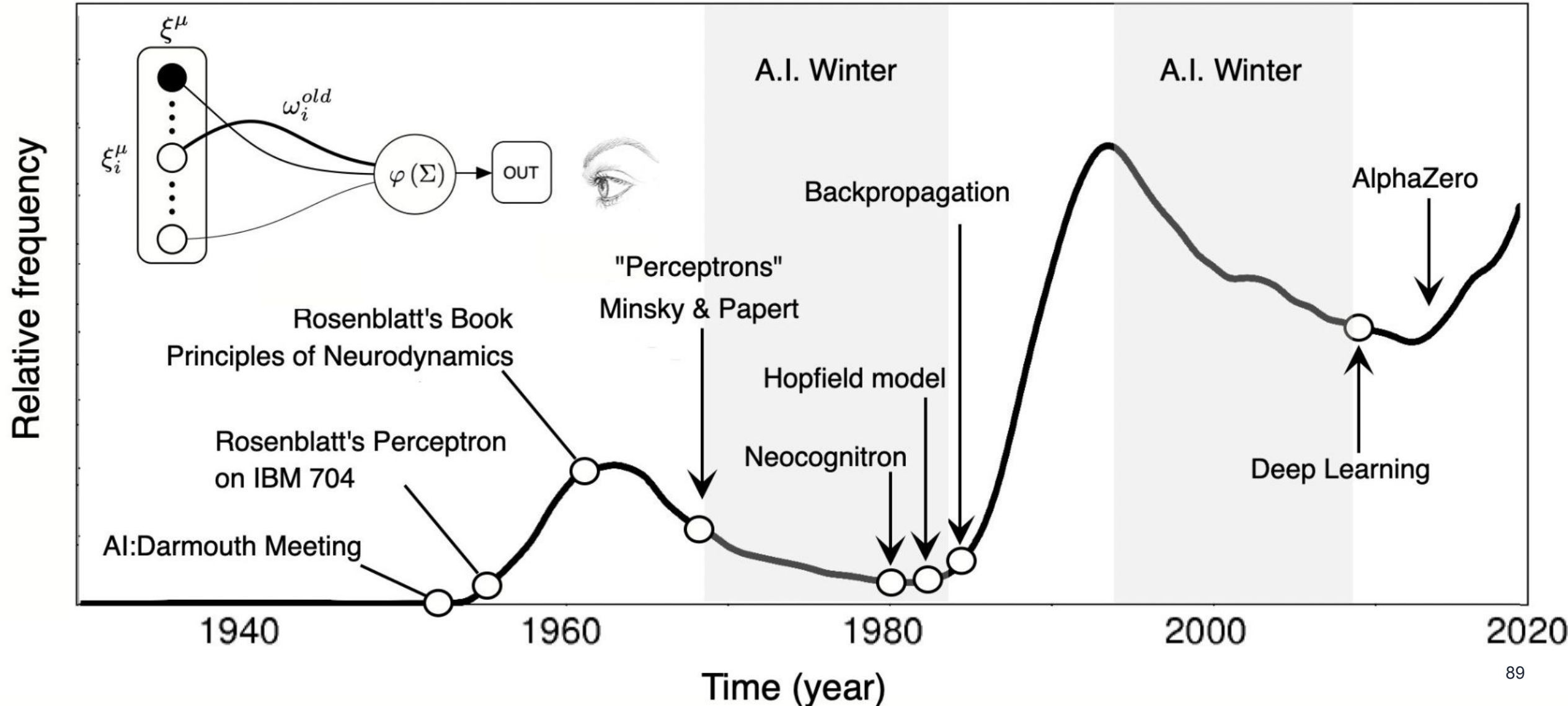
GNOMON[®]
DIGITAL



CNN: Convolutional Neural Network



GNOMON[®]
DIGITAL



CNN: Convolutional Neural Network

pixel image



imread



					Blue
				Green	255 134 93 22
			Red	255 134 202 22	2
255	231	42	22	4	30
123	94	83	2	92	124
34	44	187	92	34	142
34	76	232	124	04	
67	83	194	202		



reshaped image vector

$\begin{pmatrix} 255 \\ 231 \\ 42 \\ 22 \\ 123 \\ 94 \\ \vdots \\ \vdots \\ 92 \\ 142 \end{pmatrix}$



Données non tabular sont difficiles à comprendre pour la machine

CNN: Convolutional Neural Network



MLP ne fonctionne bien sur la classification d'images

THE MNIST DATABASE of handwritten digits

[Yann LeCun](#), Courant Institute, NYU

[Corinna Cortes](#), Google Labs, New York

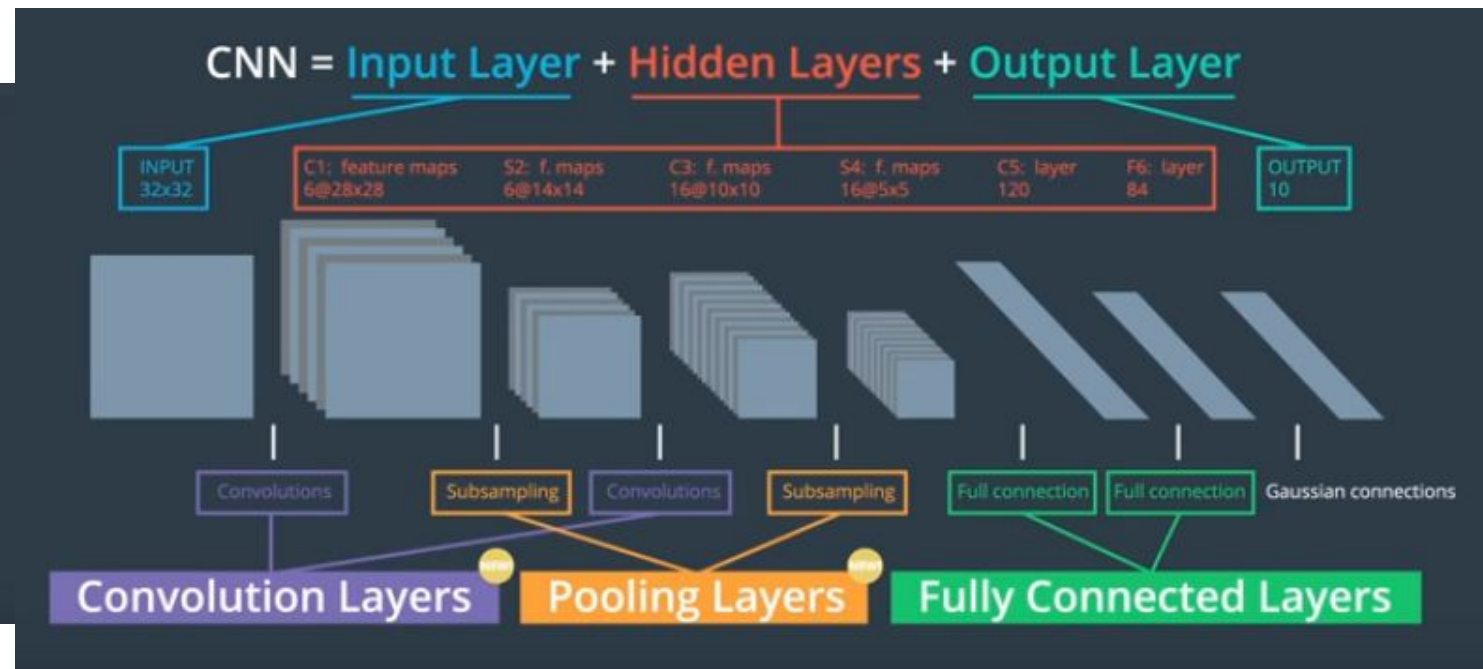
[Christopher J.C. Burges](#), Microsoft Research, Redmond

The MNIST database of handwritten digits, available from this page, has a training set of 60,000 examples, and a test set of 10,000 examples. It is a subset of a larger set available from NIST. The digits have been size-normalized and centered in a fixed-size image.

It is a good database for people who want to try learning techniques and pattern recognition methods on real-world data while spending minimal efforts on preprocessing and formatting.

Four files are available on this site:

`train-images-idx3-ubyte.gz`: training set images (9912422 bytes)



CNN: Convolutional Neural Network



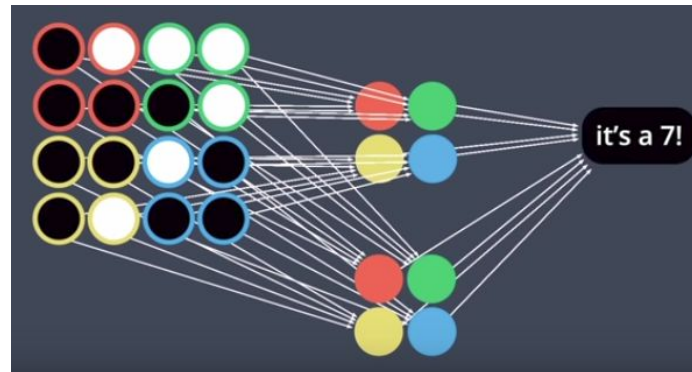
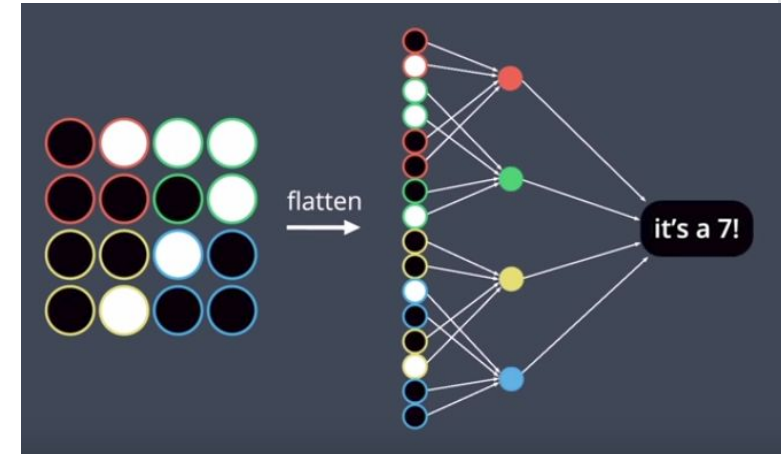
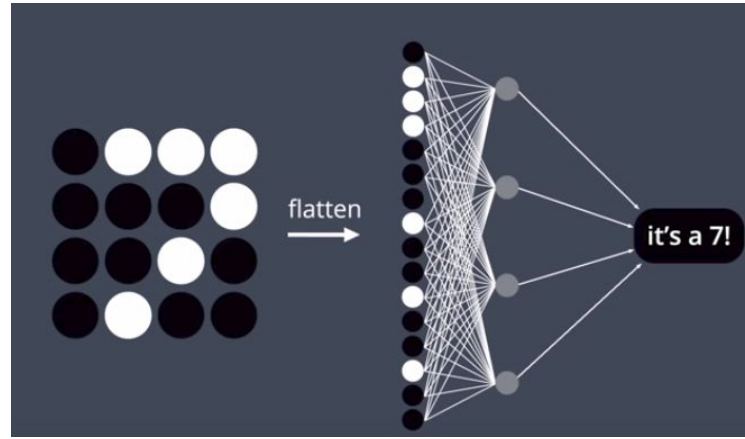
GNOMON[®]
DIGITAL

MLPs

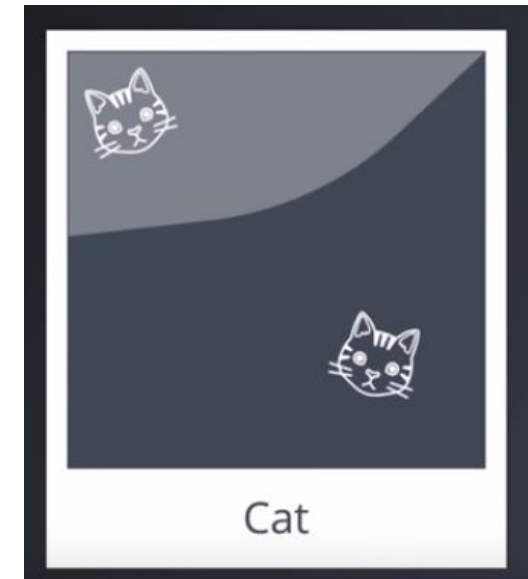
- Only use **fully** connected layers
- Only accept **vectors** as input

CNNs

- Also use **sparsely** connected layers
- Also accept **matrices** as input



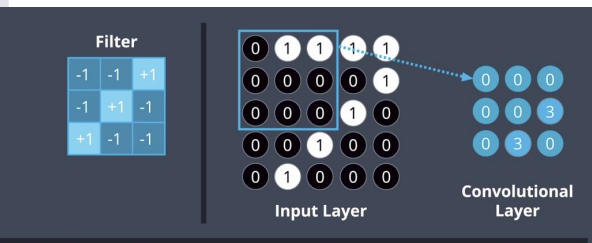
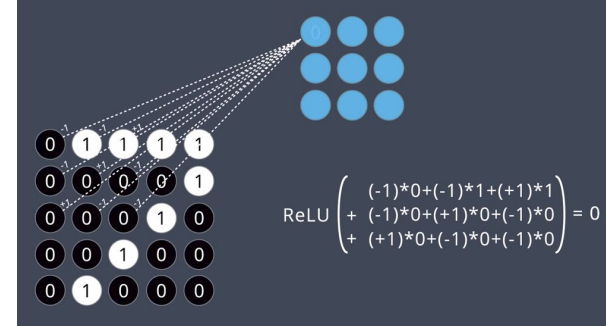
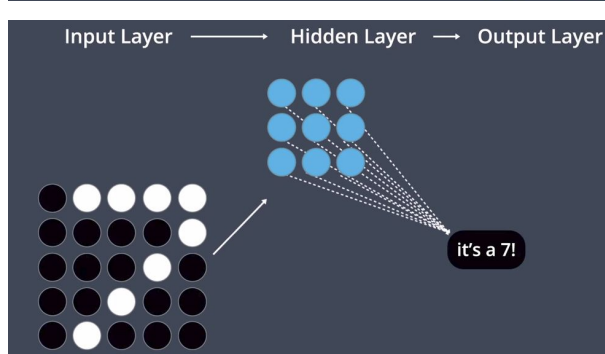
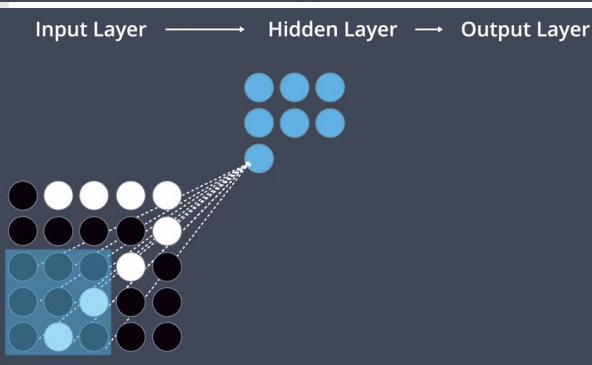
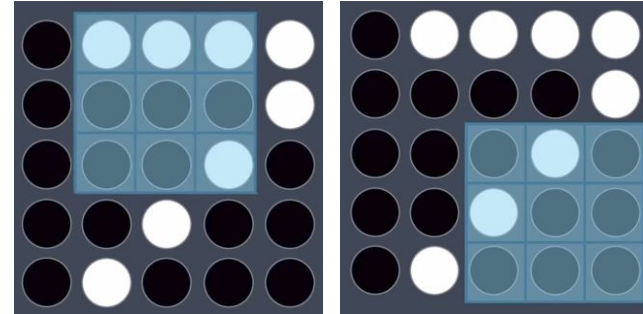
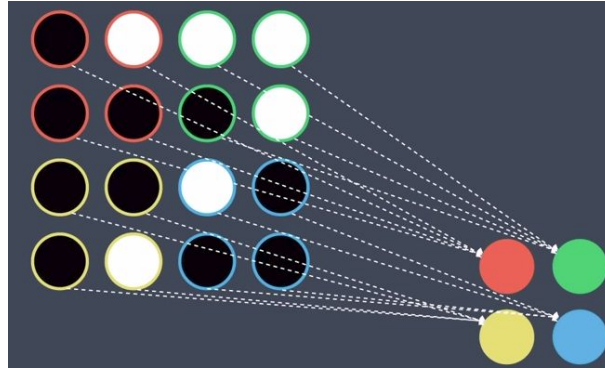
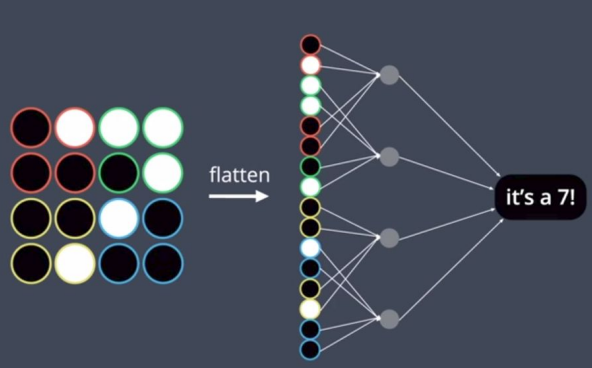
Local Connectivity



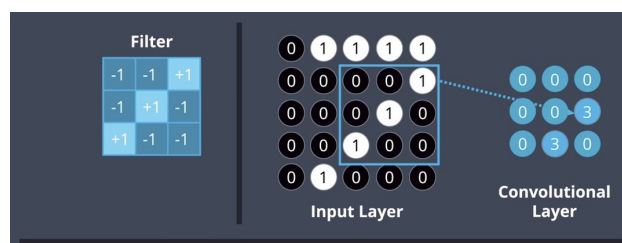
CNN: Convolutional Neural Network



GNOMON[®]
DIGITAL



$$\text{ReLU} \left(\text{SUM} \left(\begin{array}{ccc} * & * & * \\ * & * & * \\ * & * & * \end{array} \right) \right) = 0$$

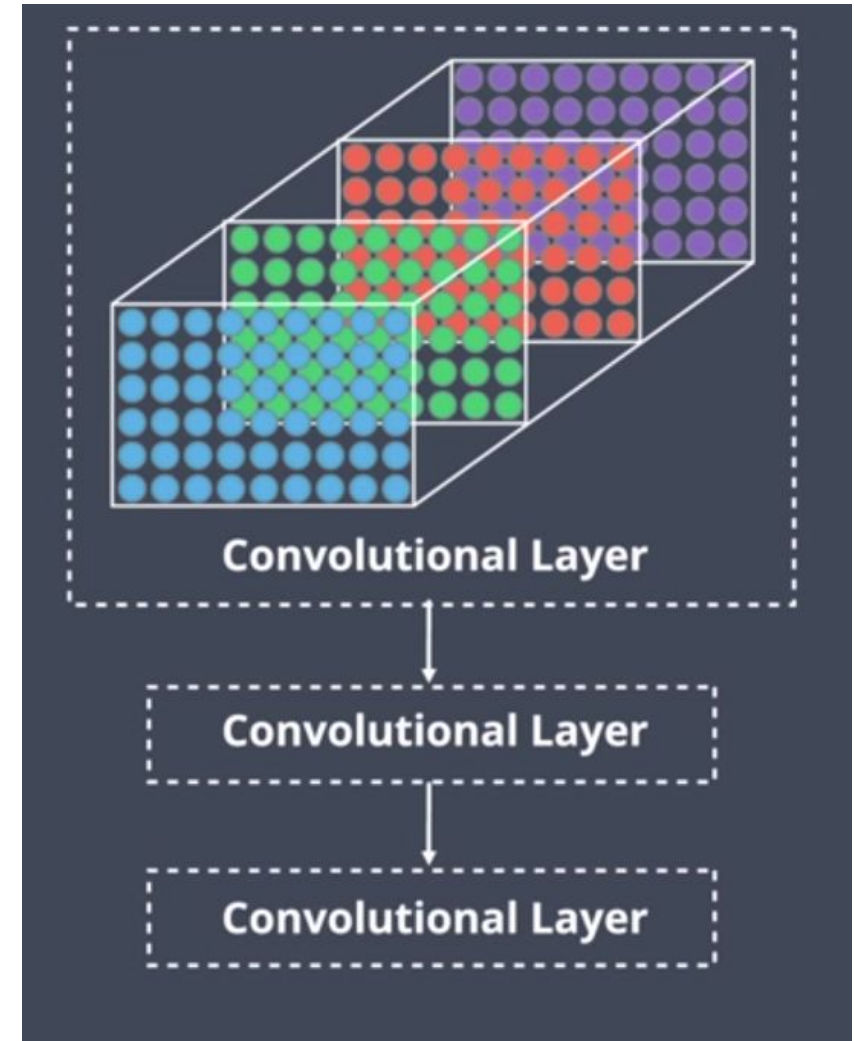
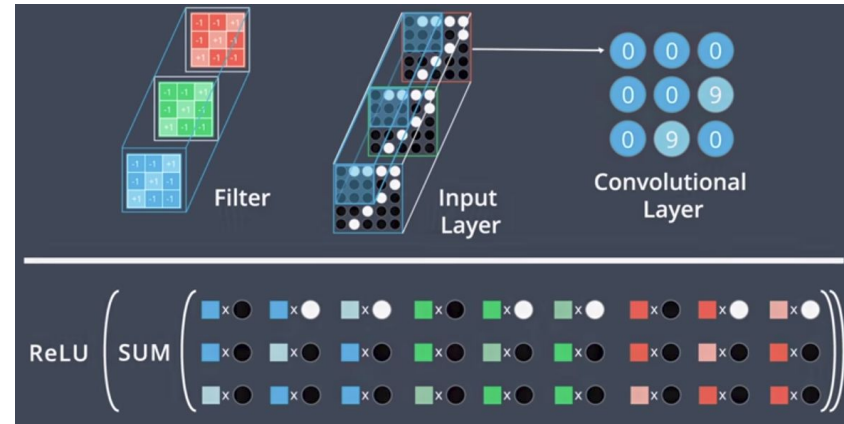
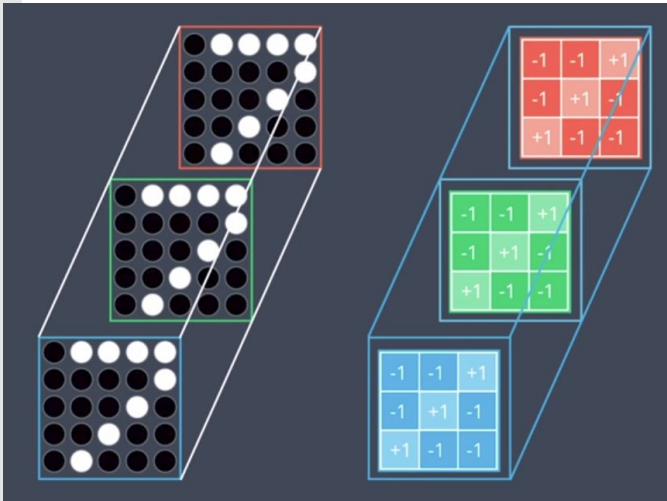


$$\text{ReLU} \left(\text{SUM} \left(\begin{array}{ccc} -1 * 0 & -1 * 0 & -1 * 1 \\ -1 * 0 & -1 * 1 & -1 * 0 \\ +1 * 1 & -1 * 0 & -1 * 0 \end{array} \right) \right) = 3$$

CNN: Convolutional Neural Network



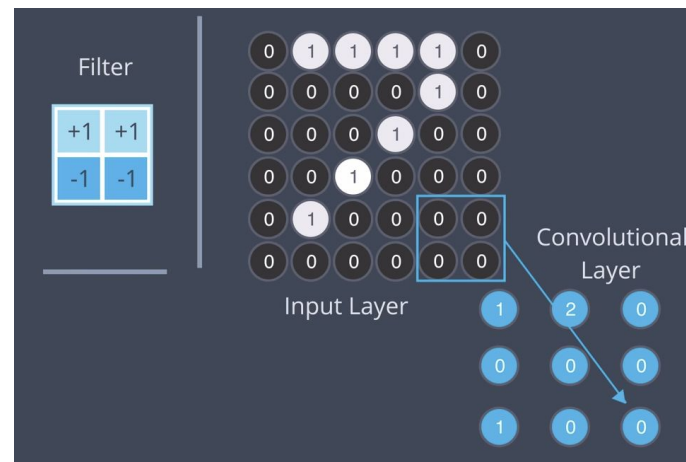
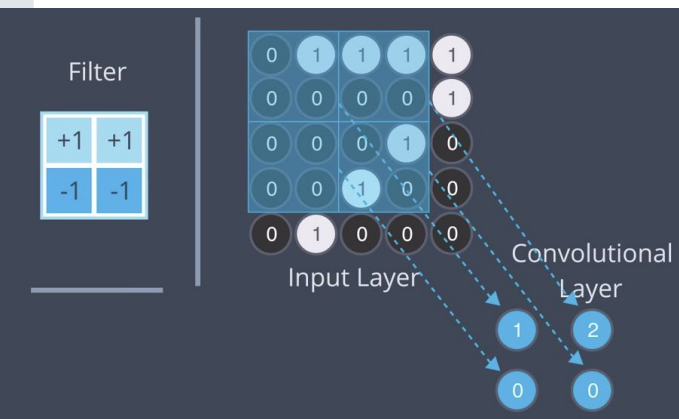
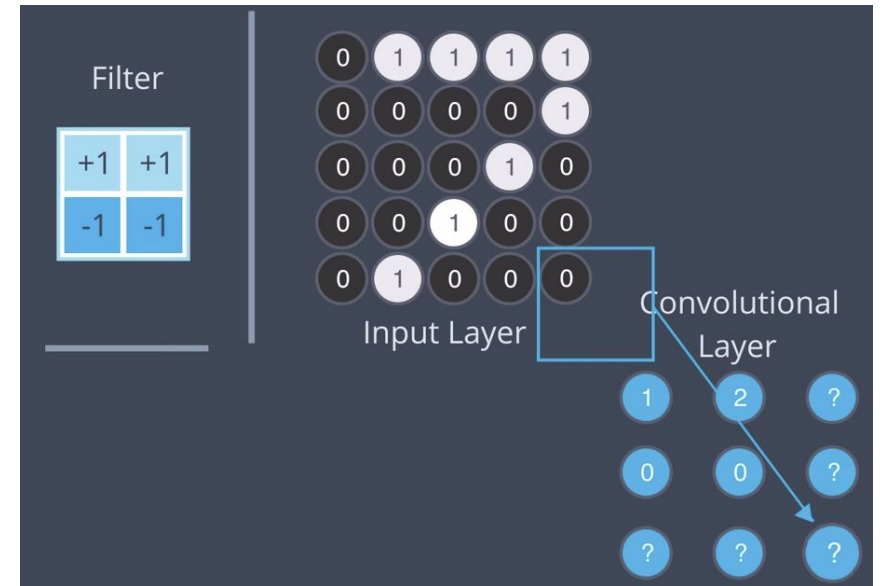
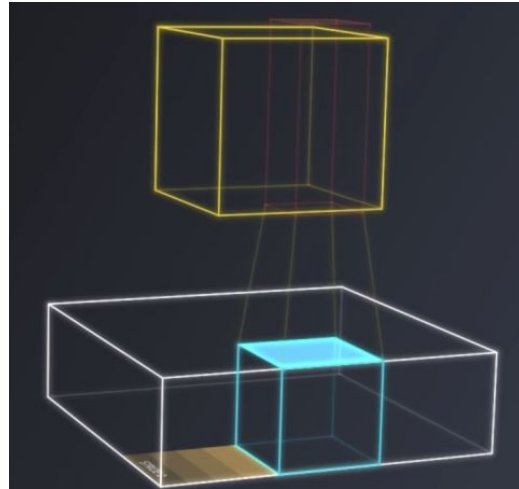
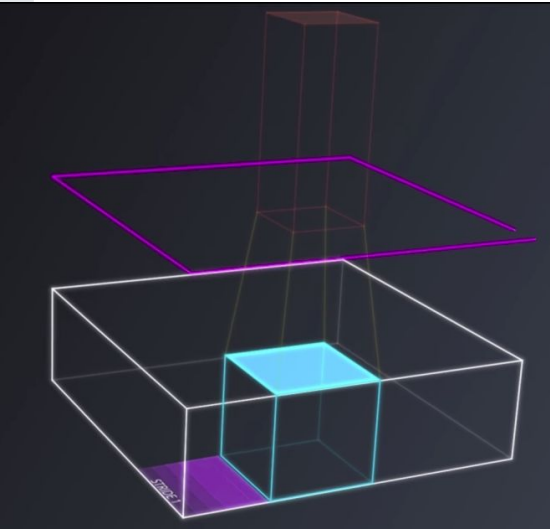
GNOMON[®]
DIGITAL



CNN: Convolutional Neural Network

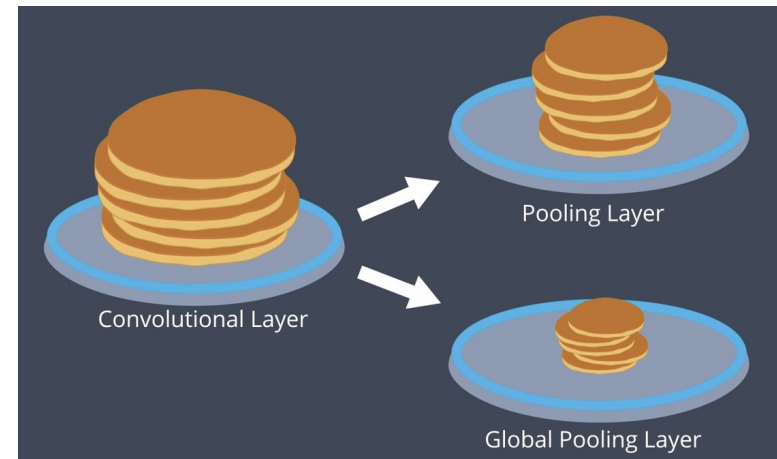
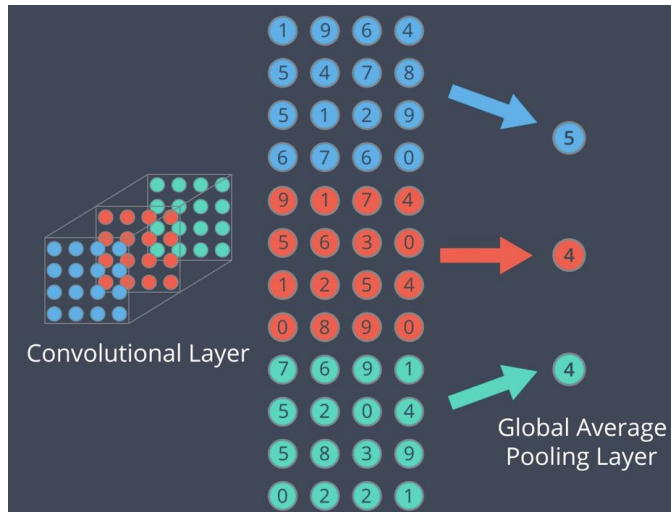
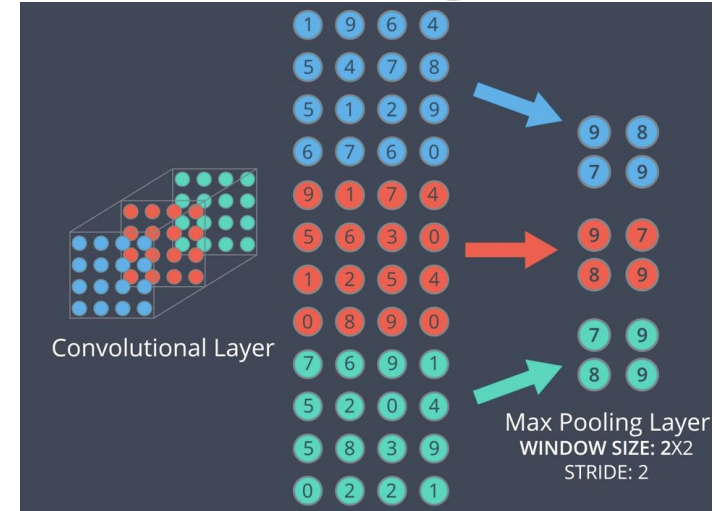
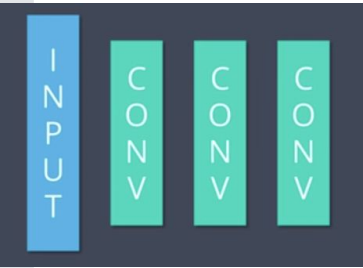


GNOMON[®]
DIGITAL



Stride and Padding

CNN: Convolutional Neural Network



CNN: Convolutional Neural Network



GNOMON[®]
DIGITAL

MaxPooling2D(pool_size, stride, padding)

INPUT

CONV

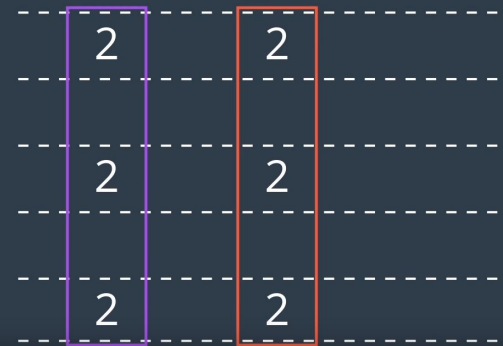
POOL

CONV

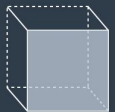
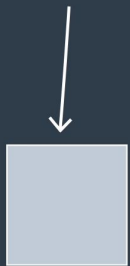
POOL

CONV

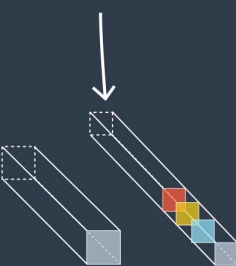
POOL



Contains a ton
of spatial information



Contains no
spatial information



Wheels? No
Eyes? Yes
Legs? Yes

```
from keras.models import Sequential
from keras.layers import Convolution2D, MaxPooling2D

model = Sequential()
model.add(Convolution2D(filters=16, kernel_size=2, padding='same', activation='relu',
                        input_shape=(32, 32, 3)))
model.add(MaxPooling2D(pool_size=2))
model.add(Convolution2D(filters=32, kernel_size=2, padding='same', activation='relu'))
model.add(MaxPooling2D(pool_size=2))
model.add(Convolution2D(filters=64, kernel_size=2, padding='same', activation='relu'))
model.add(MaxPooling2D(pool_size=2))

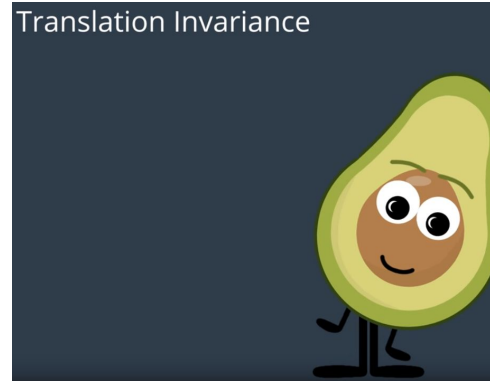
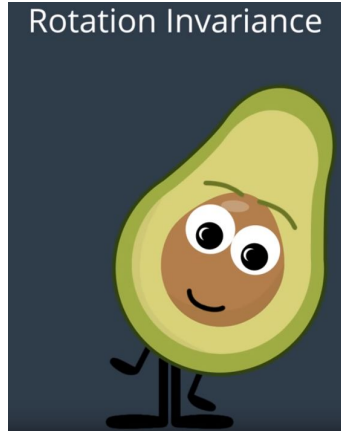
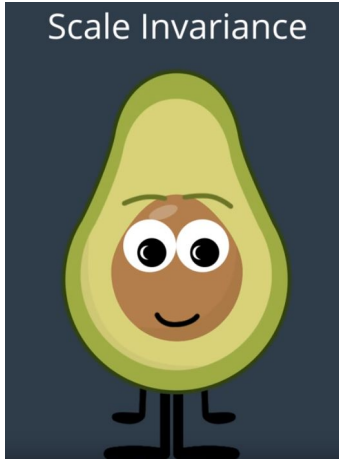
model.summary()
```

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 32, 32, 16)	208
max_pooling2d_1 (MaxPooling2D)	(None, 16, 16, 16)	0
conv2d_5 (Conv2D)	(None, 16, 16, 32)	2080
max_pooling2d_2 (MaxPooling2D)	(None, 8, 8, 32)	0
conv2d_6 (Conv2D)	(None, 8, 8, 64)	8256
max_pooling2d_3 (MaxPooling2D)	(None, 4, 4, 64)	0
Total params: 10,544.0		
Trainable params: 10,544.0		
Non-trainable params: 0.0		

CNN: Convolutional Neural Network



GNOMON[®]
DIGITAL



What we see

```
08 02 22 97 38 15 00 41 00 78 04 05 07 78 82 12 50 77 91 08
49 49 99 40 17 81 18 57 60 87 17 40 98 43 69 18 04 56 62 00
81 49 31 73 55 79 14 29 93 71 40 67 53 88 30 03 49 13 36 65
52 70 95 23 04 60 11 42 69 24 68 56 01 32 56 71 37 02 36 91
22 31 16 71 51 67 63 89 41 92 36 54 22 40 40 28 66 33 13 80
24 47 32 60 99 03 45 02 44 75 33 53 78 36 04 20 35 17 12 50
08 02 22 97 38 15 00 41 00 78 04 05 07 78 82 12 50 77 91 08
49 49 99 40 17 81 18 57 60 87 17 40 98 43 69 18 04 56 62 00
81 49 31 73 55 79 14 29 93 71 40 67 53 88 30 03 49 13 36 65
52 70 95 23 04 60 11 42 69 24 68 56 01 32 56 71 37 02 36 91
22 31 16 71 51 67 63 89 41 92 36 54 22 40 40 28 66 33 13 80
24 47 32 60 99 03 45 02 44 75 33 53 78 36 04 20 35 17 12 50
08 02 22 97 38 15 00 41 00 78 04 05 07 78 82 12 50 77 91 08
49 49 99 40 17 81 18 57 60 87 17 40 98 43 69 18 04 56 62 00
```

What computers see



Image Augmentation

CNN: Convolutional Neural Network



Data Augmentation



6. Visualize original and augmented images

```
import matplotlib.pyplot as plt

# take subset of training data
x_train_subset = x_train[:12]

# visualize subset of training data
fig = plt.figure(figsize=(20,2))
for i in range(0, len(x_train_subset)):
    ax = fig.add_subplot(1, 12, i+1)
    ax.imshow(x_train_subset[i])
fig.suptitle('Subset of Original Training Images', fontsize=20)

# visualize augmented images
fig = plt.figure(figsize=(20,2))
for x_batch in datagen.flow(x_train_subset, batch_size=12):
    for i in range(0, 12):
        ax = fig.add_subplot(1, 12, i+1)
        ax.imshow(x_batch[i])
    fig.suptitle('Augmented Images', fontsize=20)
    plt.show()
    break;
```

Subset of original training images



Augmented images



5. Create and configure augmented image generator

```
from keras.preprocessing.image import ImageDataGenerator

# create and configure augmented image generator
datagen = ImageDataGenerator(
    width_shift_range=0.1, # randomly shift images horizontally (10% of total width)
    height_shift_range=0.1, # randomly shift images vertically (10% of total height)
    horizontal_flip=True) # randomly flip images horizontally

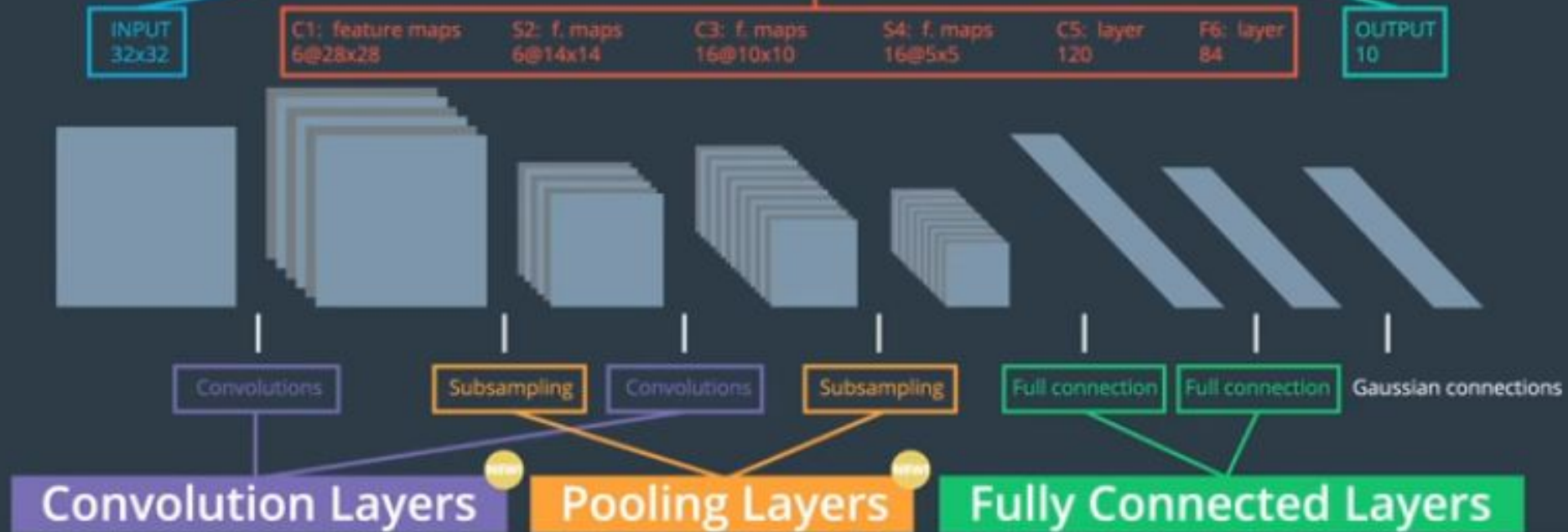
# fit augmented image generator on data
datagen.fit(x_train)
```


CNN: Convolutional Neural Network



GNOMON[®]
DIGITAL

CNN = **Input Layer** + **Hidden Layers** + **Output Layer**



x N fois

CNN: Variances



GNOMON[®]
DIGITAL

ImageNet Large Scale Visual Recognition Competition

AlexNet Architecture

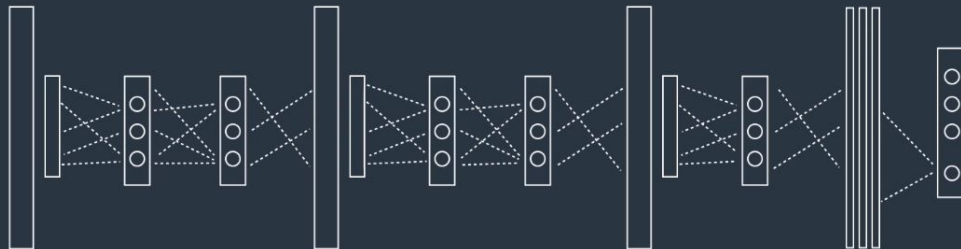


Pioneered the use of the ReLU activation function and dropout as a technique for avoiding overfitting

VGG Architecture

VGG 16

VGG 19

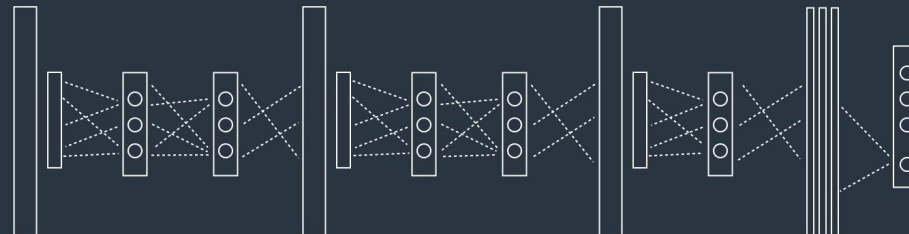


VGG Architecture

3x3 convolutions

Broken up by 2x2 pooling layers

Finished with 3 fully connected layers

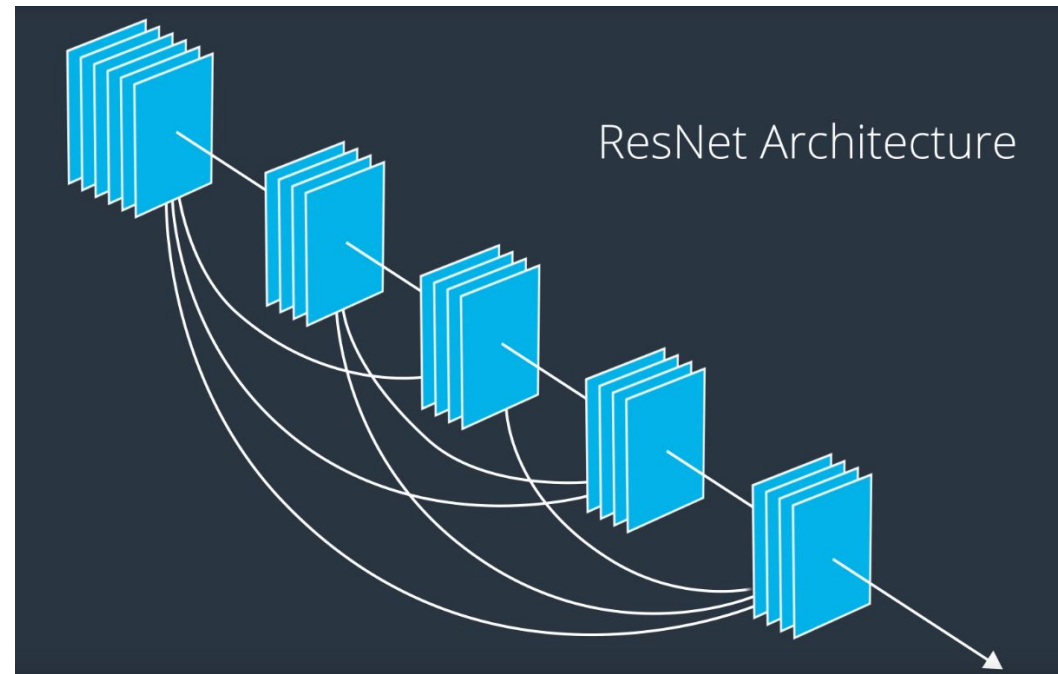
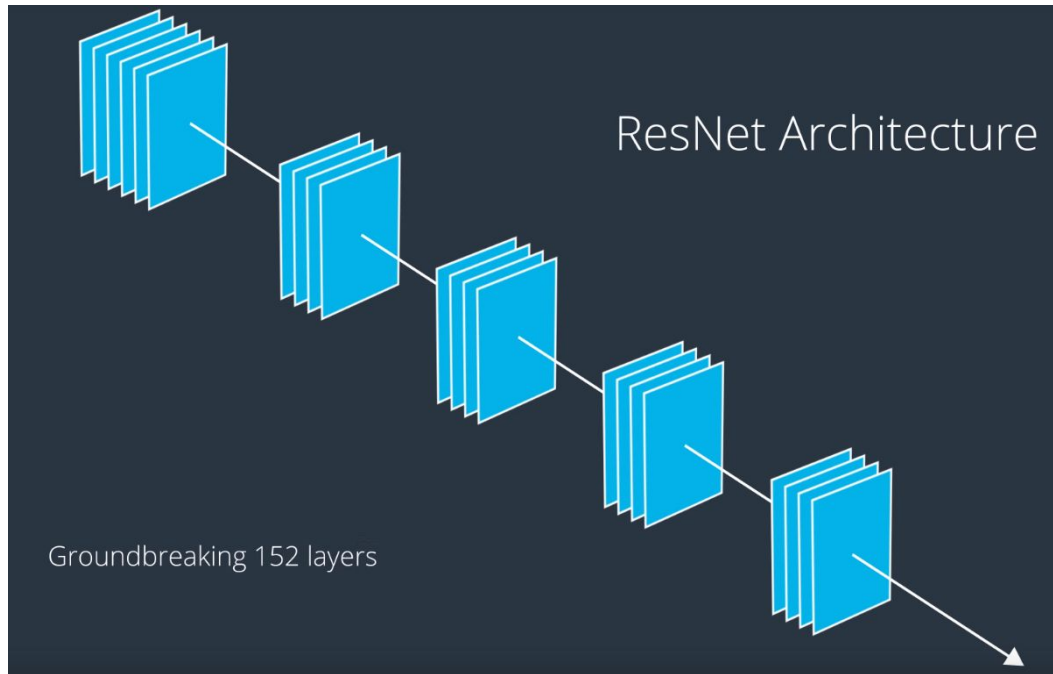


Pioneered the exclusive use of small 3x3 convolution windows

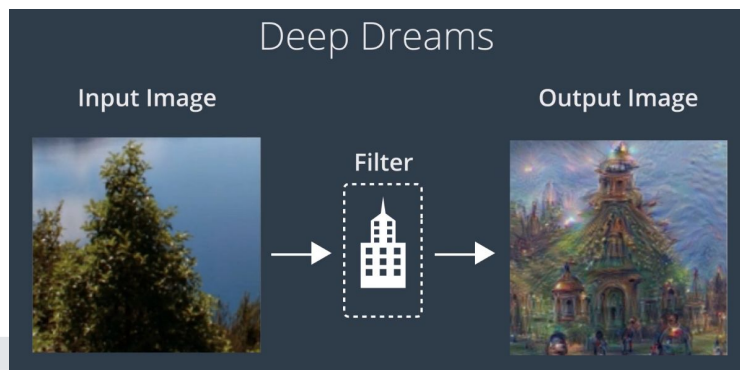
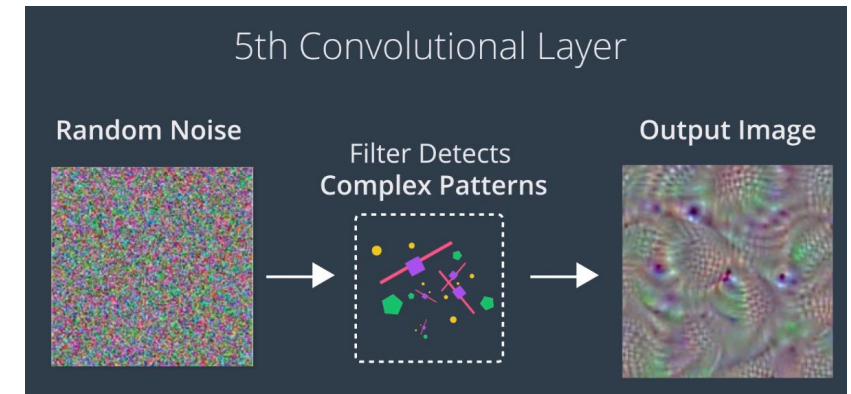
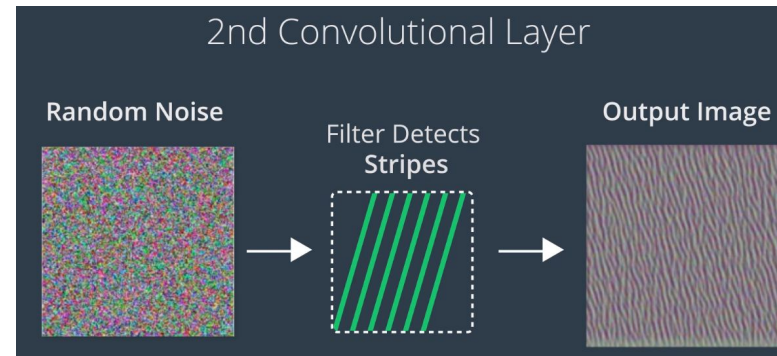
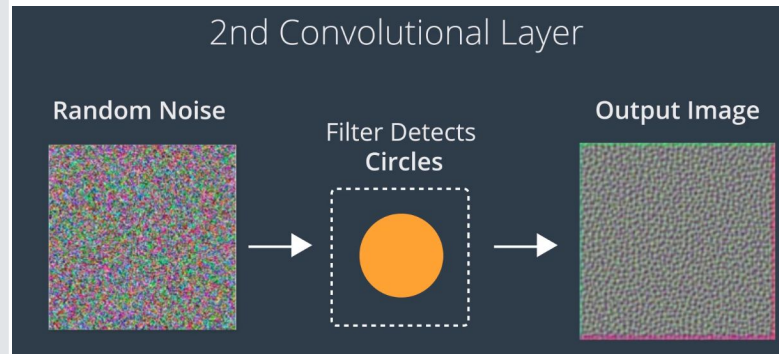
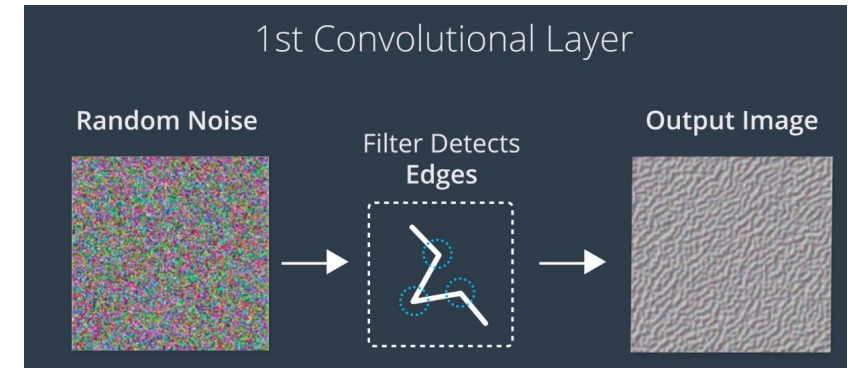
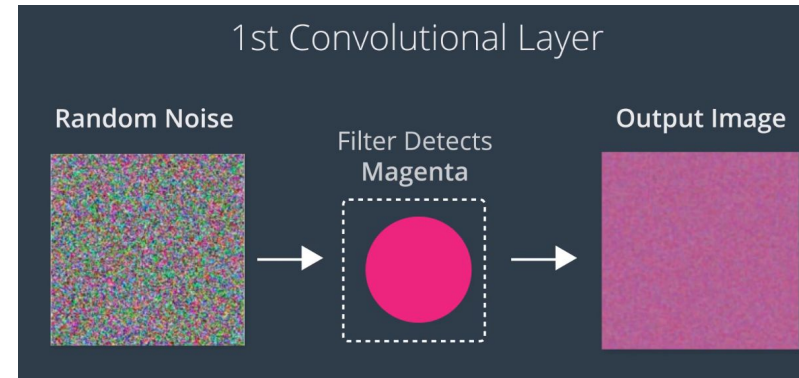
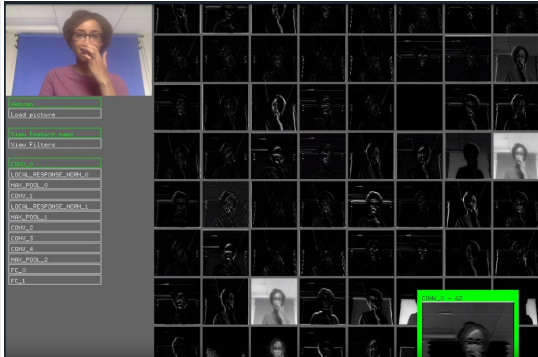
CNN: Variances



GNOMON[®]
DIGITAL



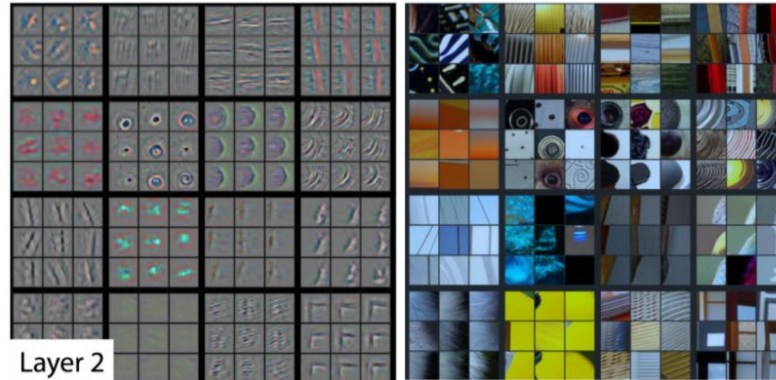
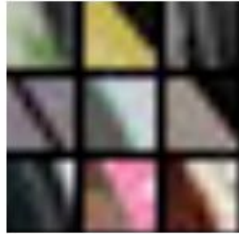
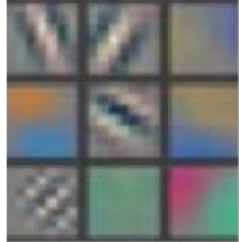
CNN



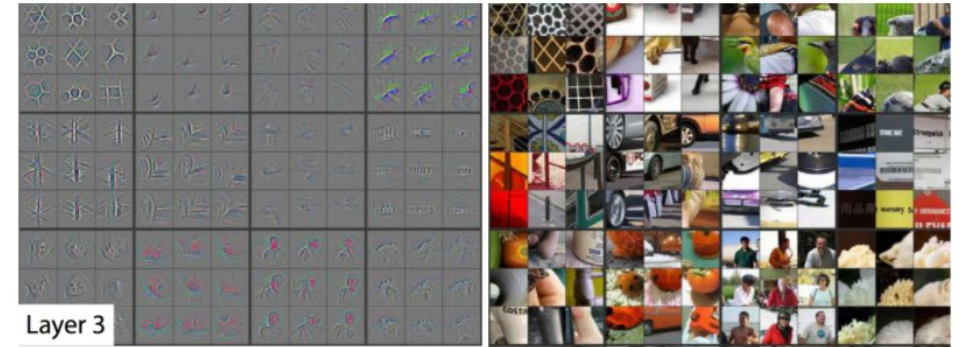
CNN



GNOMON[®]
DIGITAL

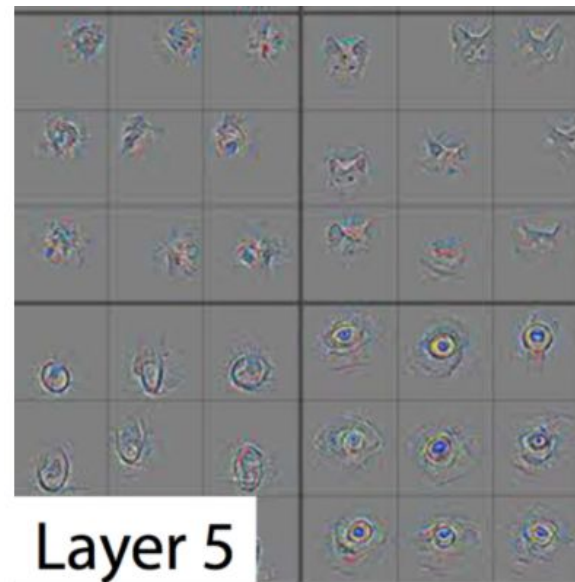
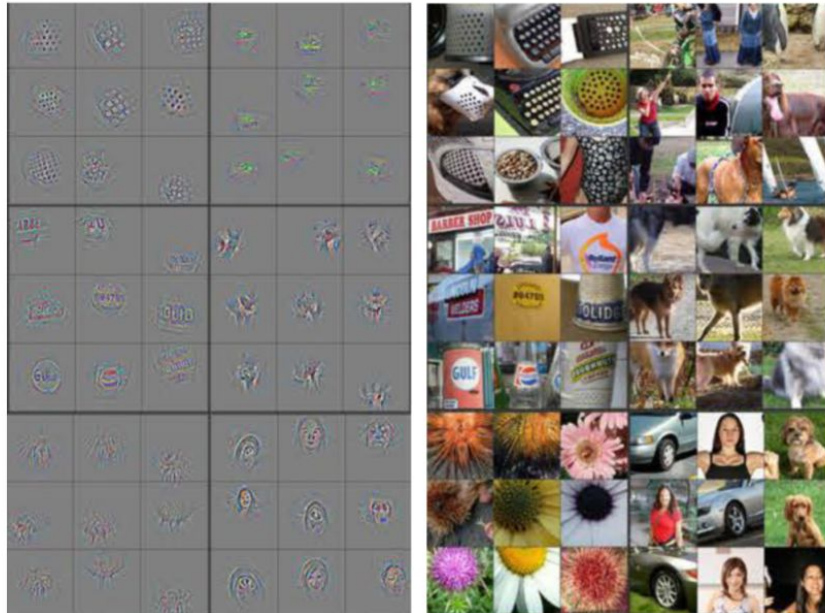


Layer 2

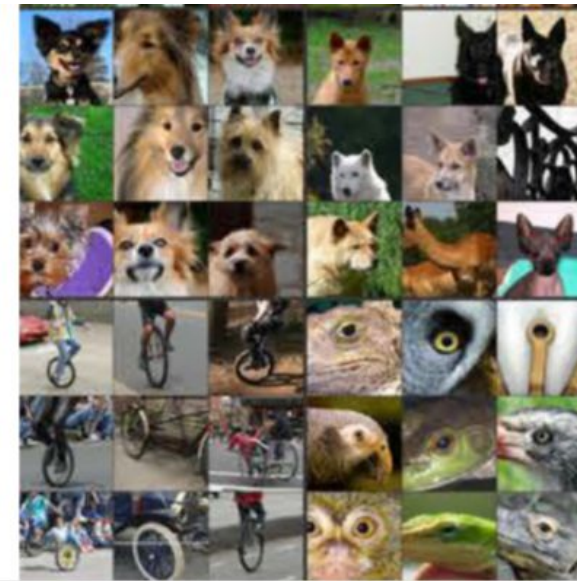


Layer 3

Layer 5



Layer 5



On peut déjà générer des images avec un modèle CNN

tjwhitaker/a-neural-
algorithm-of-artistic-...

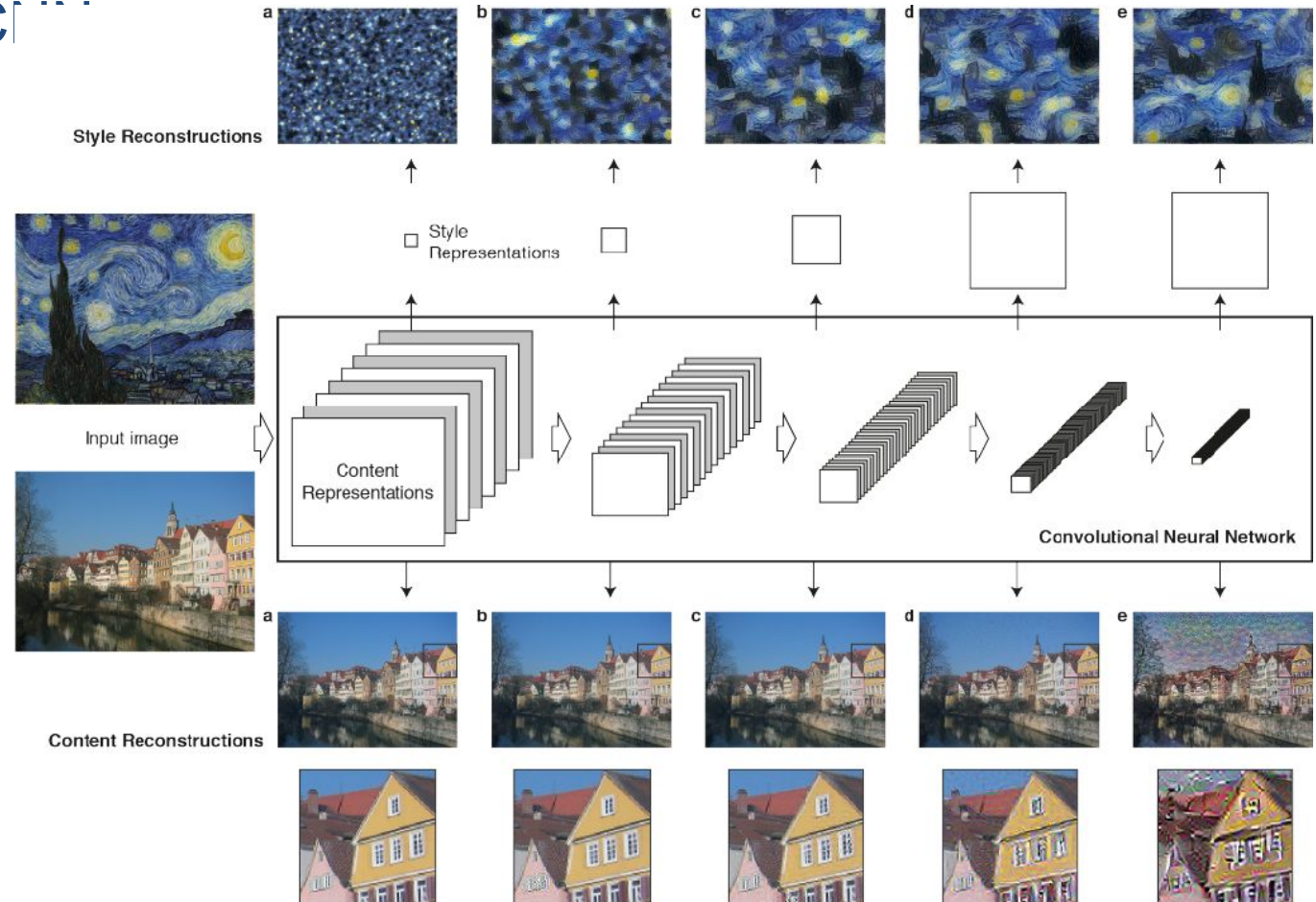
Neural Style Transfer



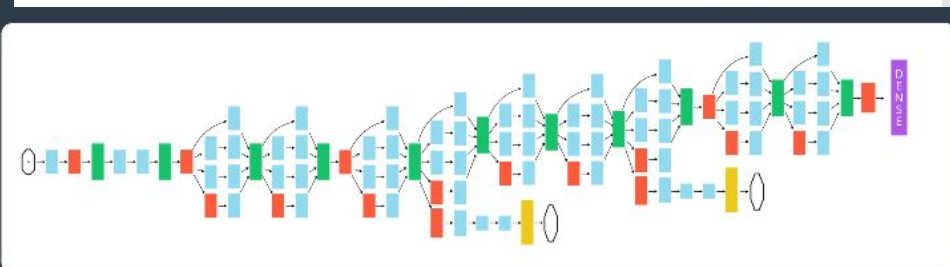
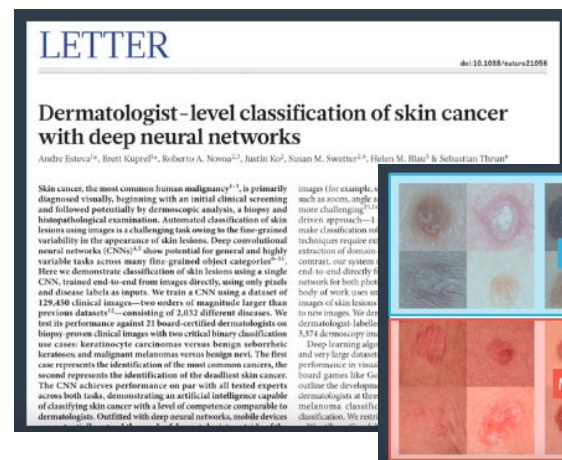
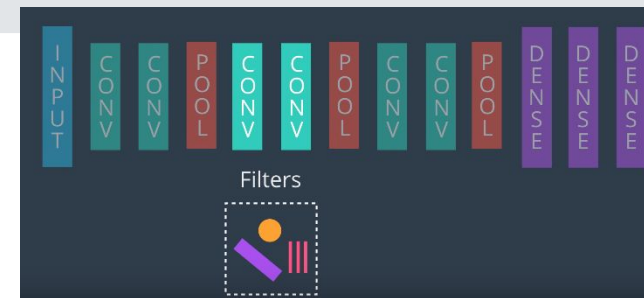
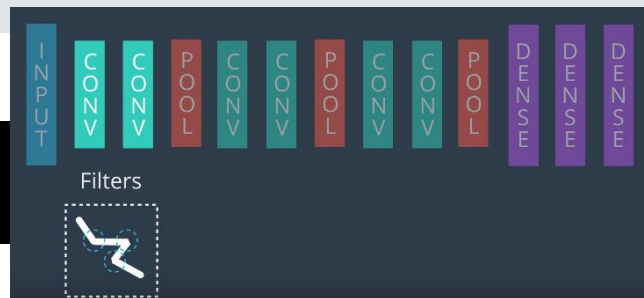
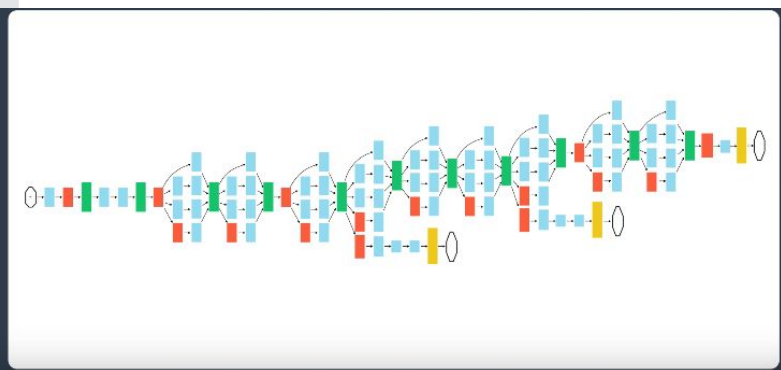
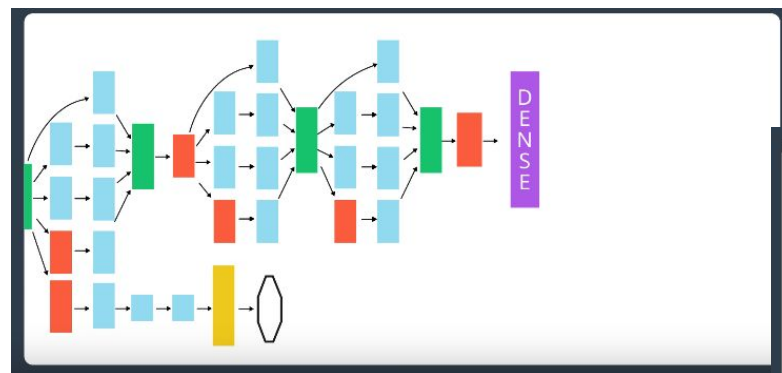
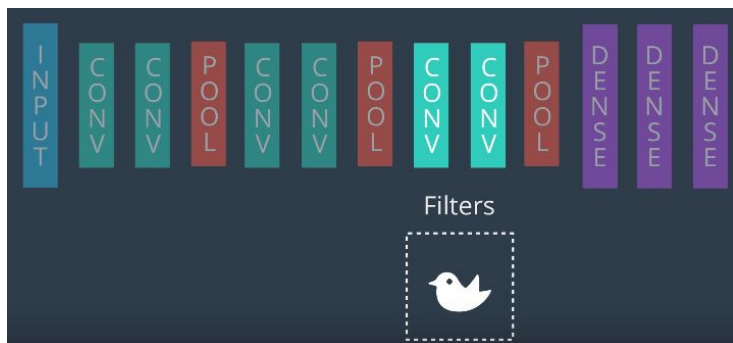
1 Contributor
0 Issues
4 Stars
2 Forks



<https://arxiv.org/abs/1508.06576>



Transfer learning



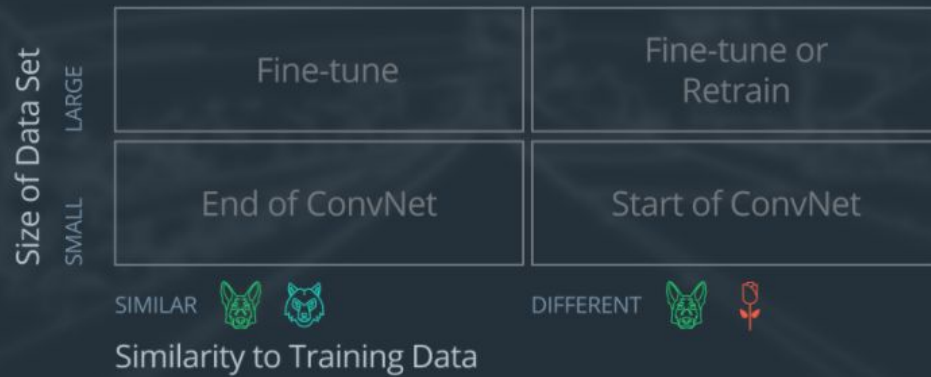
- Randomly initialize the weights in the new fully connected layer
- Initialize the rest of the weights using the pre-trained weights
- Re-train the entire neural network

Transfer learning



GNOMON[®]
DIGITAL

Guide for How to Use Transfer Learning

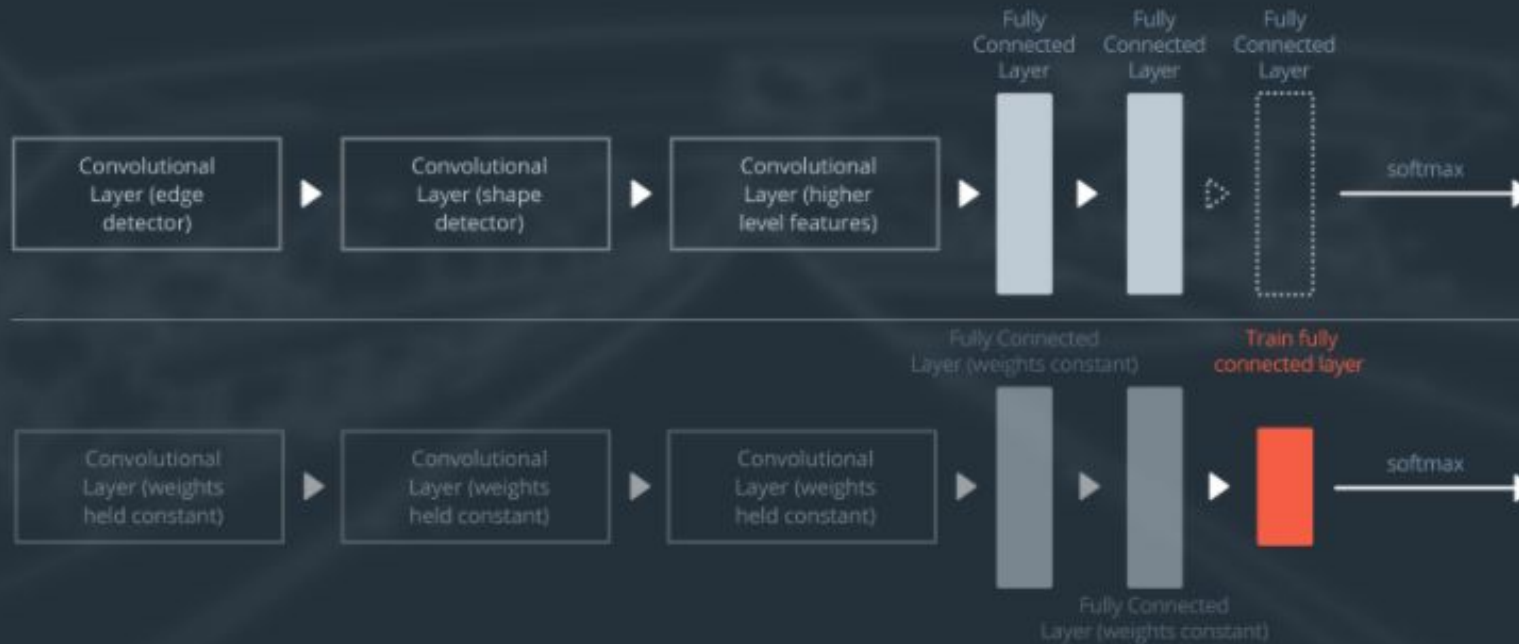


Pre-trained Convolutional Neural Network



Transfer learning

Case: Small Data Set, Similar Data

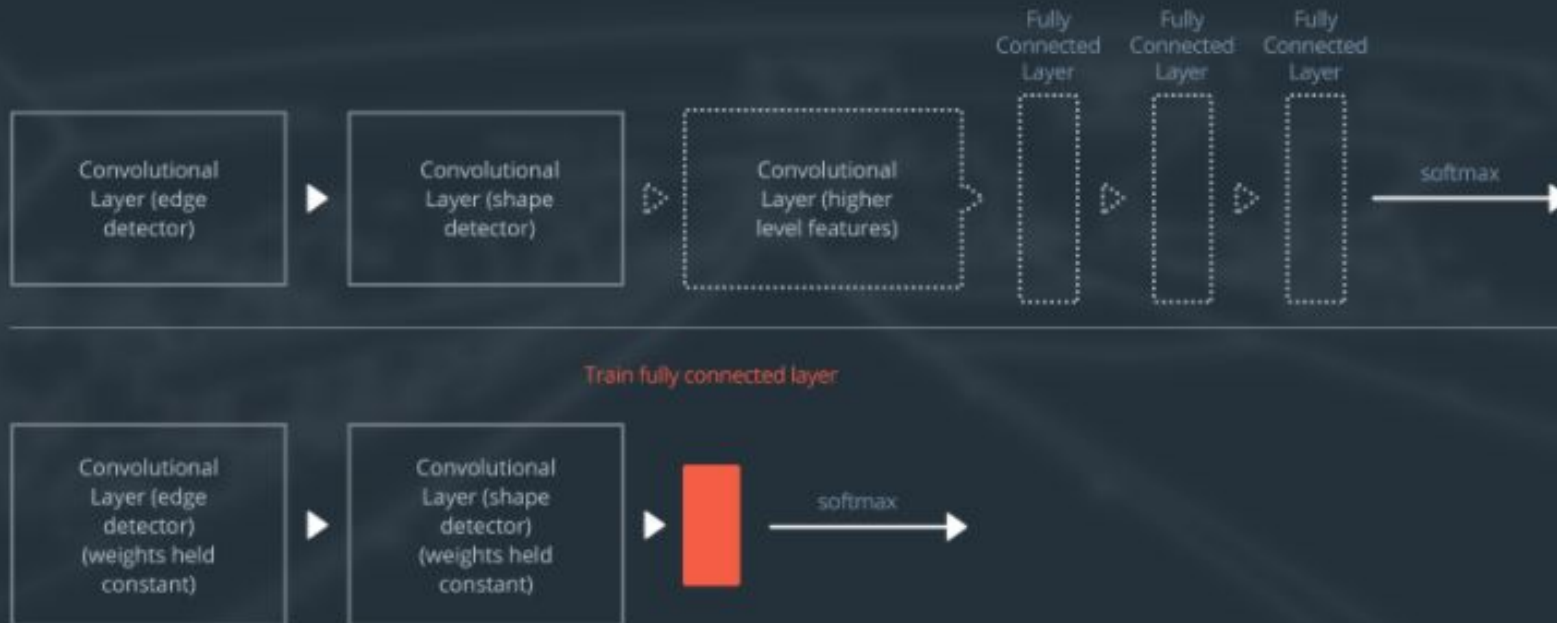


Case: Small Data Set, Similar Data

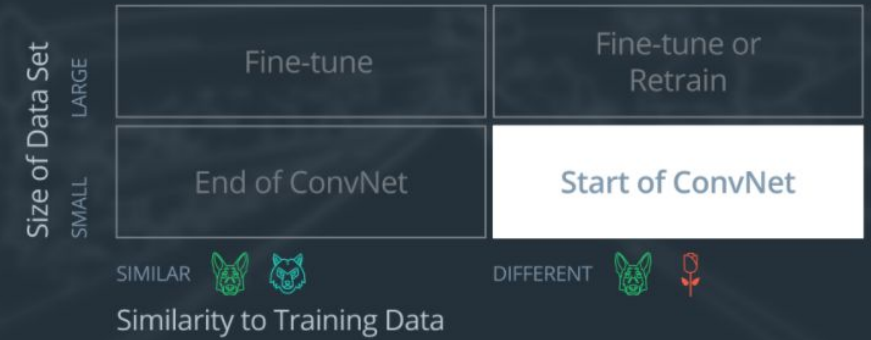
Size of Data Set	LARGE	Fine-tune	Fine-tune or Retrain
	SMALL	End of ConvNet	Start of ConvNet
SIMILAR		 	DIFFERENT  
Similarity to Training Data			

Transfer learning

Case: Small Data Set, Different Data



Case: Small Data Set, Different Data

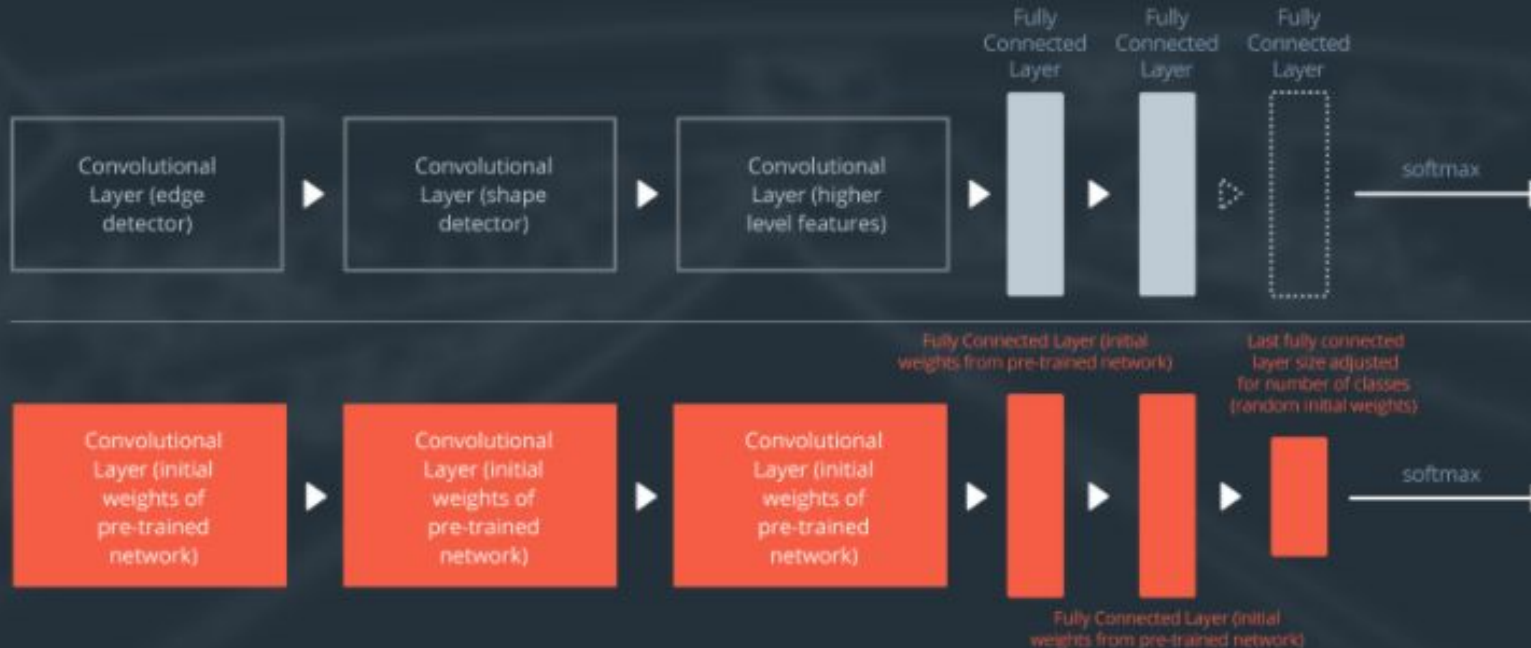


Transfer learning

Case: Large Data Set, Similar Data

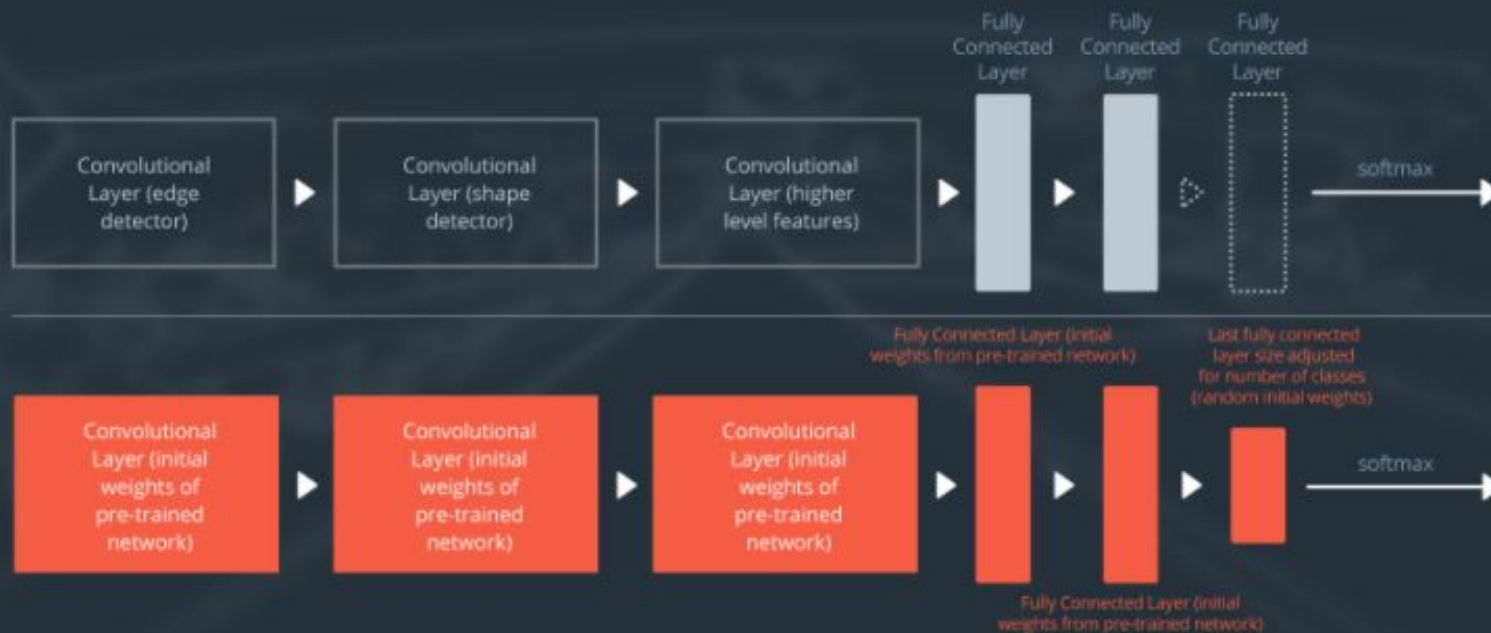
Size of Data Set	LARGE	Fine-tune	Fine-tune or Retrain
	SMALL	End of ConvNet	Start of ConvNet
		SIMILAR  	DIFFERENT  
Similarity to Training Data			

Case: Large Data Set, Similar Data



Transfer learning

Case: Large Data Set, Different Data



Case: Large Data Set, Different Data

Size of Data Set	LARGE	Fine-tune	Fine-tune or Retrain
	SMALL	End of ConvNet	Start of ConvNet
Similarity to Training Data		SIMILAR  	DIFFERENT  

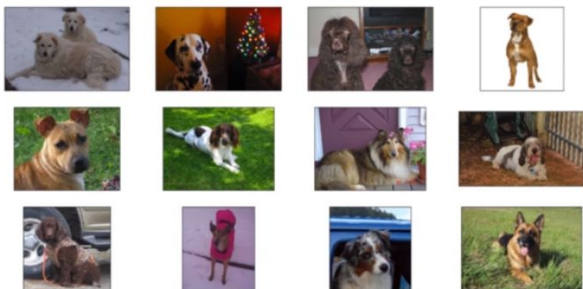
Transfer learning

2. Visualize the First 12 Training Images.

```
In [2]: import cv2
import matplotlib.pyplot as plt
matplotlib.interactive(True)

def visualize_img(img_path, ax):
    img = cv2.imread(img_path)
    ax.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))

fig = plt.figure(figsize=(20, 10))
for i in range(12):
    ax = fig.add_subplot(3, 4, i + 1, ticks=[])
    visualize_img(train_files[i], ax)
```



Dataset

8351 Images

133 Breeds

3. Obtain the VGG-16 Bottleneck Features

```
In [3]: bottleneck_features = np.load('bottleneck_features/DogVGG16Data.npz')
train_vgg16 = bottleneck_features['train']
valid_vgg16 = bottleneck_features['valid']
test_vgg16 = bottleneck_features['test']
```

VGG-16



VGG-16

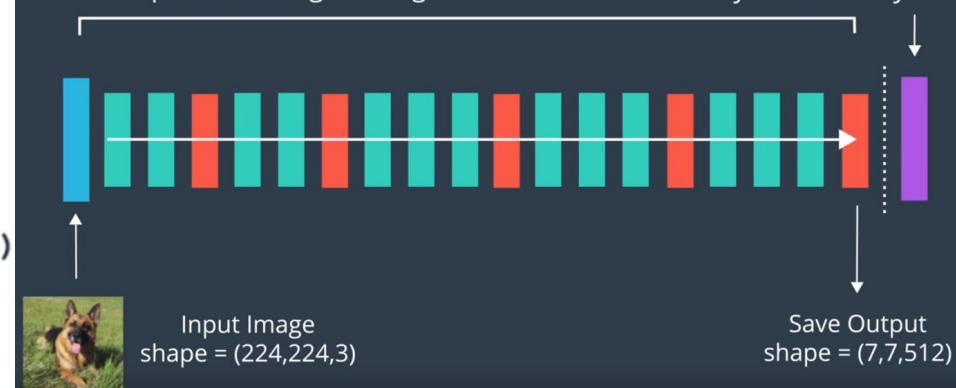
the only trainable layer



VGG-16

keep fixed during training

the only trainable layer



Transfer learning



```
from keras.layers import Dense, Flatten
from keras.models import Sequential

model = Sequential()
model.add(Flatten(input_shape=(7,7,512)))
model.add(Dense(133, activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='rmsprop',
              metrics=['accuracy'])
model.summary()
```

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 25088)	0
dense_1 (Dense)	(None, 133)	3336837

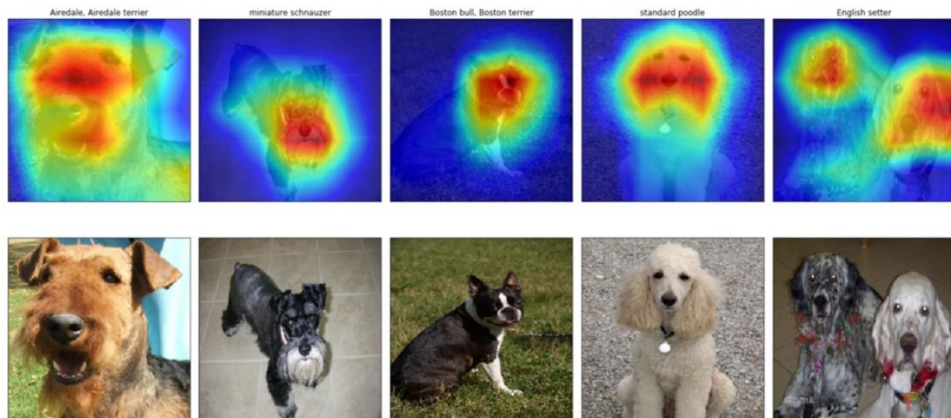
Total params: 3,336,837.0
Trainable params: 3,336,837.0
Non-trainable params: 0.0

```
from keras.layers import GlobalAveragePooling2D
```

```
model = Sequential()
model.add(GlobalAveragePooling2D(input_shape=(7,7,512)))
model.add(Dense(133, activation='softmax'))
model.summary()
```

Layer (type)	Output Shape	Param #
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 512)	0
dense_2 (Dense)	(None, 133)	68229

Total params: 68,229.0
Trainable params: 68,229.0
Non-trainable params: 0.0



Python For Data Science Cheat Sheet

Keras

Learn Python for data science [interactively](https://www.datacamp.com) at [www.DataCamp.com](https://www.datacamp.com)



Keras

Keras is a powerful and easy-to-use deep learning library for Theano and TensorFlow that provides a high-level neural networks API to develop and evaluate deep learning models.

A Basic Example

```
>>> import numpy as np
>>> from keras.models import Sequential
>>> from keras.layers import Dense
>>> data = np.random.random((1000,100))
>>> labels = np.random.randint(2,size=(1000,1))
>>> model = Sequential()
>>> model.add(Dense(32,
                    activation='relu',
                    input_dim=100))
>>> model.add(Dense(1, activation='sigmoid'))
>>> model.compile(optimizer='rmsprop',
                 loss='binary_crossentropy',
                 metrics=['accuracy'])
>>> model.fit(data, labels, epochs=10, batch_size=32)
>>> predictions = model.predict(data)
```

Data

Also see NumPy, Pandas & Scikit-Learn

Your data needs to be stored as NumPy arrays or as a list of NumPy arrays. Ideally, you split the data in training and test sets, for which you can also resort to the `train_test_split` module of `sklearn.cross_validation`.

Keras Data Sets

```
>>> from keras.datasets import boston_housing,
mnist,
cifar10,
imdb

>>> (x_train,y_train),(x_test,y_test) = mnist.load_data()
>>> (x_train2,y_train2),(x_test2,y_test2) = boston_housing.load_data()
>>> (x_train3,y_train3),(x_test3,y_test3) = cifar10.load_data()
>>> (x_train4,y_train4),(x_test4,y_test4) = imdb.load_data(num_words=20000)
>>> num_classes = 10
```

Other

```
>>> from urllib.request import urlopen
>>> data = np.loadtxt(urlopen("http://archive.ics.uci.edu/
ml/machine-learning-databases/pima-indians-diabetes/
pima-indians-diabetes.data"),delimiter=",")
>>> X = data[:,0:8]
>>> y = data[:,8]
```

Preprocessing

Sequence Padding

```
>>> from keras.preprocessing import sequence
>>> x_train4 = sequence.pad_sequences(x_train4,maxlen=80)
>>> x_test4 = sequence.pad_sequences(x_test4,maxlen=80)
```

One-Hot Encoding

```
>>> from keras.utils import to_categorical
>>> Y_train = to_categorical(y_train, num_classes)
>>> Y_test = to_categorical(y_test, num_classes)
>>> Y_train3 = to_categorical(y_train3, num_classes)
>>> Y_test3 = to_categorical(y_test3, num_classes)
```

Model Architecture

Sequential Model

```
>>> from keras.models import Sequential
>>> model = Sequential()
>>> model2 = Sequential()
>>> model3 = Sequential()
```

Multilayer Perceptron (MLP)

Binary Classification

```
>>> from keras.layers import Dense
>>> model.add(Dense(12,
                    input_dim=8,
                    kernel_initializer='uniform',
                    activation='relu'))
>>> model.add(Dense(8, kernel_initializer='uniform', activation='relu'))
>>> model.add(Dense(1, kernel_initializer='uniform', activation='sigmoid'))
```

Multi-Class Classification

```
>>> from keras.layers import Dropout
>>> model.add(Dense(512, activation='relu', input_shape=(784,)))
>>> model.add(Dropout(0.2))
>>> model.add(Dense(512, activation='relu'))
>>> model.add(Dropout(0.2))
>>> model.add(Dense(10, activation='softmax'))
```

Regression

```
>>> model.add(Dense(64, activation='relu', input_dim=train_data.shape[1]))
>>> model.add(Dense(1))
```

Convolutional Neural Network (CNN)

```
>>> from keras.layers import Activation,Conv2D,MaxPooling2D,Flatten
>>> model2.add(Conv2D(32,(3,3),padding='same',input_shape=x_train.shape[1:]))
>>> model2.add(Activation('relu'))
>>> model2.add(Conv2D(32,(3,3)))
>>> model2.add(Activation('relu'))
>>> model2.add(MaxPooling2D(pool_size=(2,2)))
>>> model2.add(Dropout(0.25))
>>> model2.add(Conv2D(64,(3,3),padding='same'))
>>> model2.add(Activation('relu'))
>>> model2.add(Conv2D(64,(3,3)))
>>> model2.add(Activation('relu'))
>>> model2.add(MaxPooling2D(pool_size=(2,2)))
>>> model2.add(Dropout(0.25))
>>> model2.add(Flatten())
>>> model2.add(Dense(512))
>>> model2.add(Activation('relu'))
>>> model2.add(Dropout(0.5))
>>> model2.add(Dense(num_classes))
>>> model2.add(Activation('softmax'))
```

Recurrent Neural Network (RNN)

```
>>> from keras.layers import Embedding,LSTM
>>> model3.add(Embedding(20000,128))
>>> model3.add(LSTM(128,dropout=0.2, recurrent_dropout=0.2))
>>> model3.add(Dense(1, activation='sigmoid'))
```

Also see NumPy & Scikit-Learn

Train and Test Sets

```
>>> from sklearn.model_selection import train_test_split
>>> X_train5,X_test5,y_train5,y_test5 = train_test_split(X,
                                                         y,
                                                         test_size=0.33,
                                                         random_state=42)
```

Standardization/Normalization

```
>>> from sklearn.preprocessing import StandardScaler
>>> scaler = StandardScaler().fit(x_train2)
>>> standardized_X = scaler.transform(x_train2)
>>> standardized_X_test = scaler.transform(x_test2)
```

Inspect Model

```
>>> model.output_shape
>>> model.summary()
>>> model.get_config()
>>> model.get_weights()
```

Model output shape
Model summary representation
Model configuration
List all weight tensors in the model

Compile Model

MLP: Binary Classification

```
>>> model.compile(optimizer='adam',
                 loss='binary_crossentropy',
                 metrics=['accuracy'])
```

MLP: Multi-Class Classification

```
>>> model.compile(optimizer='rmsprop',
                 loss='categorical_crossentropy',
                 metrics=['accuracy'])
```

MLP: Regression

```
>>> model.compile(optimizer='rmsprop',
                 loss='mse',
                 metrics=['mae'])
```

Recurrent Neural Network

```
>>> model3.compile(loss='binary_crossentropy',
                  optimizer='adam',
                  metrics=['accuracy'])
```

Model Training

```
>>> model3.fit(x_train4,
              y_train4,
              batch_size=32,
              epochs=15,
              verbose=1,
              validation_data=(x_test4,y_test4))
```

Evaluate Your Model's Performance

```
>>> score = model3.evaluate(x_test,
                           y_test,
                           batch_size=32)
```

Prediction

```
>>> model3.predict(x_test4, batch_size=32)
>>> model3.predict_classes(x_test4,batch_size=32)
```

Save/ Reload Models

```
>>> from keras.models import load_model
>>> model3.save('model_file.h5')
>>> my_model = load_model('my_model.h5')
```

Model Fine-tuning

Optimization Parameters

```
>>> from keras.optimizers import RMSprop
>>> opt = RMSprop(lr=0.0001, decay=1e-6)
>>> model2.compile(loss='categorical_crossentropy',
                  optimizer=opt,
                  metrics=['accuracy'])
```

Early Stopping

```
>>> from keras.callbacks import EarlyStopping
>>> early_stopping_monitor = EarlyStopping(patience=2)
>>> model3.fit(x_train4,
              y_train4,
              batch_size=32,
              epochs=15,
              validation_data=(x_test4,y_test4),
              callbacks=[early_stopping_monitor])
```

GNOMON[®]
DIGITAL



Python For Data Science Cheat Sheet

Keras

Learn Python for data science [interactively](#) at [www.DataCamp.com](#)



Keras

Keras is a powerful and easy-to-use deep learning library for Theano and TensorFlow that provides a high-level neural networks API to develop and evaluate deep learning models.

A Basic Example

```
>>> import numpy as np
>>> from keras.models import Sequential
>>> from keras.layers import Dense
>>> data = np.random.random((1000,100))
>>> labels = np.random.randint(2,size=(1000,1))
>>> model = Sequential()
>>> model.add(Dense(32,
                    activation='relu',
                    input_dim=100))
>>> model.add(Dense(1, activation='sigmoid'))
>>> model.compile(optimizer='rmsprop',
                 loss='binary_crossentropy',
                 metrics=['accuracy'])
>>> model.fit(data, labels, epochs=10, batch_size=32)
>>> predictions = model.predict(data)
```

Data

Also see NumPy, Pandas & Scikit-Learn

Your data needs to be stored as NumPy arrays or as a list of NumPy arrays. Ideally, you split the data in training and test sets, for which you can also resort to the `train_test_split` module of `sklearn.cross_validation`.

Keras Data Sets

```
>>> from keras.datasets import boston_housing,
mnist,
cifar10,
imdb

>>> (x_train,y_train),(x_test,y_test) = mnist.load_data()
>>> (x_train2,y_train2),(x_test2,y_test2) = boston_housing.load_data()
>>> (x_train3,y_train3),(x_test3,y_test3) = cifar10.load_data()
>>> (x_train4,y_train4),(x_test4,y_test4) = imdb.load_data(num_words=20000)
>>> num_classes = 10
```

Other

```
>>> from urllib.request import urlopen
>>> data = np.loadtxt(urlopen("http://archive.ics.uci.edu/
ml/machine-learning-databases/pima-indians-diabetes/
pima-indians-diabetes.data"),delimiter=",")
>>> X = data[:,0:8]
>>> y = data[:,8]
```

Preprocessing

Sequence Padding

```
>>> from keras.preprocessing import sequence
>>> x_train4 = sequence.pad_sequences(x_train4,maxlen=80)
>>> x_test4 = sequence.pad_sequences(x_test4,maxlen=80)
```

One-Hot Encoding

```
>>> from keras.utils import to_categorical
>>> Y_train = to_categorical(y_train, num_classes)
>>> Y_test = to_categorical(y_test, num_classes)
>>> Y_train3 = to_categorical(y_train3, num_classes)
>>> Y_test3 = to_categorical(y_test3, num_classes)
```

Model Architecture

Sequential Model

```
>>> from keras.models import Sequential
>>> model = Sequential()
>>> model2 = Sequential()
>>> model3 = Sequential()
```

Multilayer Perceptron (MLP)

Binary Classification

```
>>> from keras.layers import Dense
>>> model.add(Dense(12,
                    input_dim=8,
                    kernel_initializer='uniform',
                    activation='relu'))
>>> model.add(Dense(8, kernel_initializer='uniform', activation='relu'))
>>> model.add(Dense(1, kernel_initializer='uniform', activation='sigmoid'))
```

Multi-Class Classification

```
>>> from keras.layers import Dropout
>>> model.add(Dense(512, activation='relu', input_shape=(784,)))
>>> model.add(Dropout(0.2))
>>> model.add(Dense(512, activation='relu'))
>>> model.add(Dropout(0.2))
>>> model.add(Dense(10, activation='softmax'))
```

Regression

```
>>> model.add(Dense(64, activation='relu', input_dim=train_data.shape[1]))
>>> model.add(Dense(1))
```

Convolutional Neural Network (CNN)

```
>>> from keras.layers import Activation,Conv2D,MaxPooling2D,Flatten
>>> model2.add(Conv2D(32,(3,3),padding='same',input_shape=x_train.shape[1:]))
>>> model2.add(Activation('relu'))
>>> model2.add(Conv2D(32,(3,3)))
>>> model2.add(Activation('relu'))
>>> model2.add(MaxPooling2D(pool_size=(2,2)))
>>> model2.add(Dropout(0.25))
>>> model2.add(Conv2D(64,(3,3),padding='same'))
>>> model2.add(Activation('relu'))
>>> model2.add(Conv2D(64,(3,3)))
>>> model2.add(Activation('relu'))
>>> model2.add(MaxPooling2D(pool_size=(2,2)))
>>> model2.add(Dropout(0.25))
>>> model2.add(Flatten())
>>> model2.add(Dense(512))
>>> model2.add(Activation('relu'))
>>> model2.add(Dropout(0.5))
>>> model2.add(Dense(num_classes))
>>> model2.add(Activation('softmax'))
```

Recurrent Neural Network (RNN)

```
>>> from keras.layers import Embedding,LSTM
>>> model3.add(Embedding(20000,128))
>>> model3.add(LSTM(128,dropout=0.2, recurrent_dropout=0.2))
>>> model3.add(Dense(1, activation='sigmoid'))
```

Also see NumPy & Scikit-Learn

Train and Test Sets

```
>>> from sklearn.model_selection import train_test_split
>>> X_train5,X_test5,y_train5,y_test5 = train_test_split(X,
                                                         y,
                                                         test_size=0.33,
                                                         random_state=42)
```

Standardization/Normalization

```
>>> from sklearn.preprocessing import StandardScaler
>>> scaler = StandardScaler().fit(x_train2)
>>> standardized_X = scaler.transform(x_train2)
>>> standardized_X_test = scaler.transform(x_test2)
```

Inspect Model

```
>>> model.output_shape
>>> model.summary()
>>> model.get_config()
>>> model.get_weights()
```

Model output shape
Model summary representation
Model configuration
List all weight tensors in the model

Compile Model

MLP: Binary Classification

```
>>> model.compile(optimizer='adam',
                 loss='binary_crossentropy',
                 metrics=['accuracy'])
```

MLP: Multi-Class Classification

```
>>> model.compile(optimizer='rmsprop',
                 loss='categorical_crossentropy',
                 metrics=['accuracy'])
```

MLP: Regression

```
>>> model.compile(optimizer='rmsprop',
                 loss='mse',
                 metrics=['mae'])
```

Recurrent Neural Network

```
>>> model3.compile(loss='binary_crossentropy',
                  optimizer='adam',
                  metrics=['accuracy'])
```

Model Training

```
>>> model3.fit(x_train4,
              y_train4,
              batch_size=32,
              epochs=15,
              verbose=1,
              validation_data=(x_test4,y_test4))
```

Evaluate Your Model's Performance

```
>>> score = model3.evaluate(x_test,
                           y_test,
                           batch_size=32)
```

Prediction

```
>>> model3.predict(x_test4, batch_size=32)
>>> model3.predict_classes(x_test4,batch_size=32)
```

Save/ Reload Models

```
>>> from keras.models import load_model
>>> model3.save('model_file.h5')
>>> my_model = load_model('my_model.h5')
```

Model Fine-tuning

Optimization Parameters

```
>>> from keras.optimizers import RMSprop
>>> opt = RMSprop(lr=0.0001, decay=1e-6)
>>> model2.compile(loss='categorical_crossentropy',
                  optimizer=opt,
                  metrics=['accuracy'])
```

Early Stopping

```
>>> from keras.callbacks import EarlyStopping
>>> early_stopping_monitor = EarlyStopping(patience=2)
>>> model3.fit(x_train4,
              y_train4,
              batch_size=32,
              epochs=15,
              validation_data=(x_test4,y_test4),
              callbacks=[early_stopping_monitor])
```

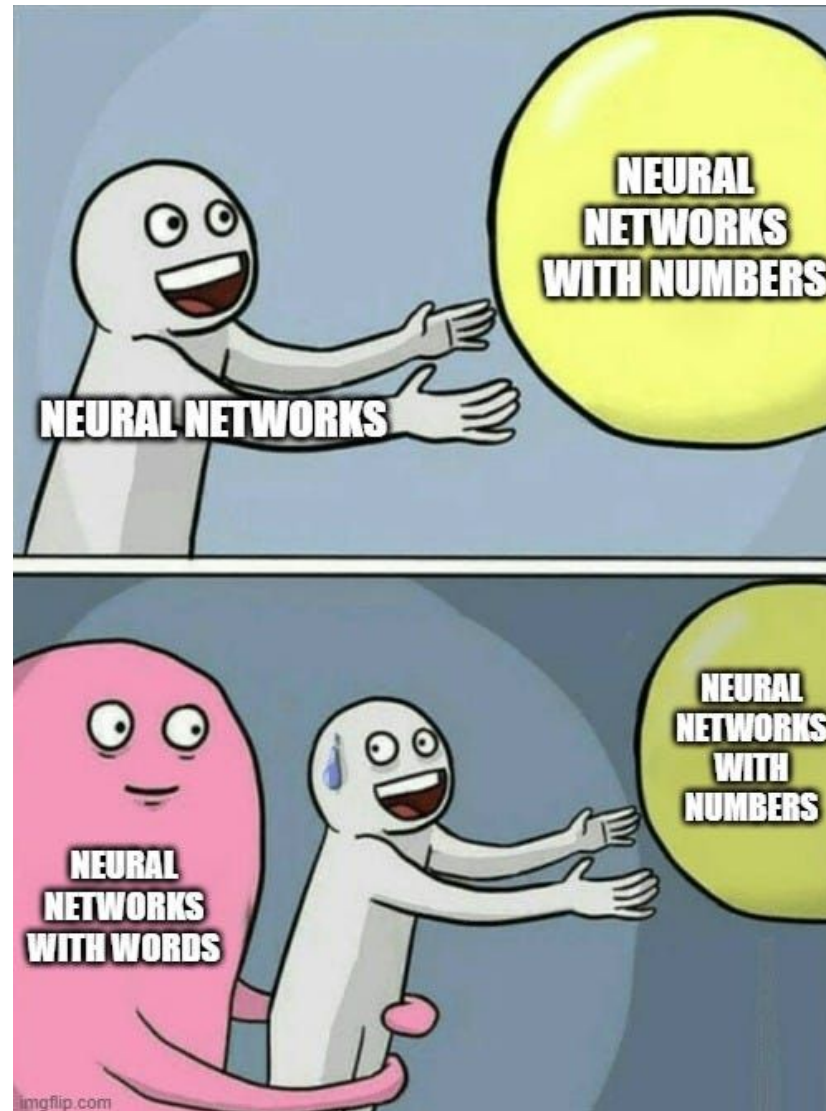
GNOMON[®]
DIGITAL



NLP

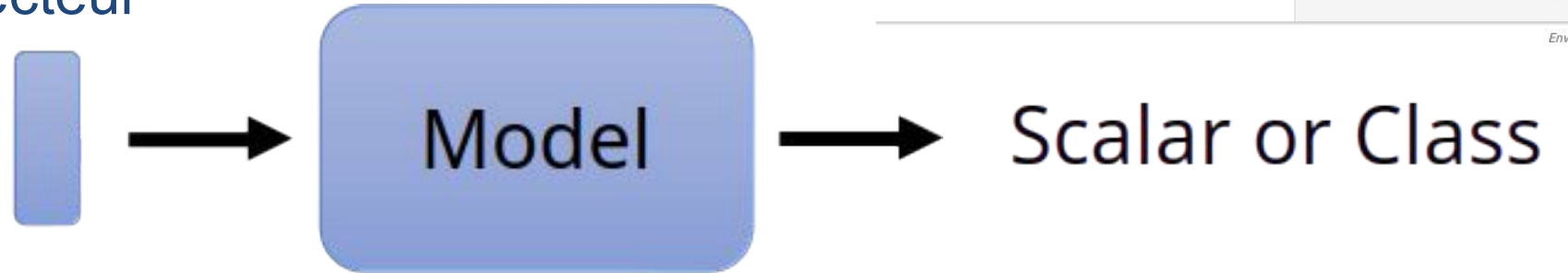


GNOMON[®]
DIGITAL



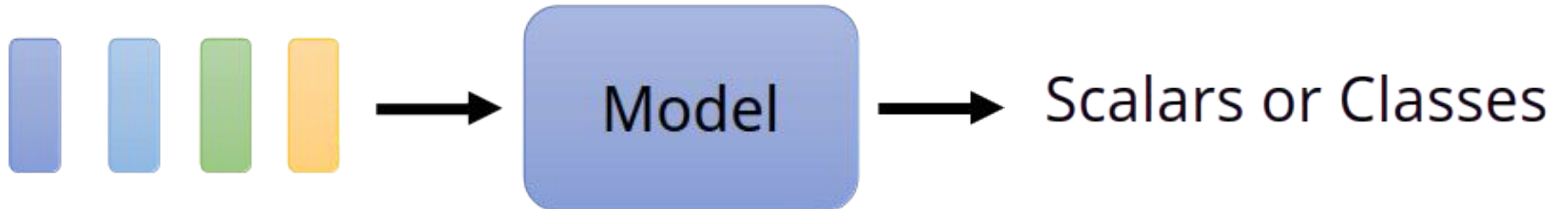
NLP

- L'entrée est un vecteur



Longueur fixé ou uniformisé

- L'entrée est une listes de vecteurs



Longueur peut varier

Comment fonctionne
Google Traduction ?

How does Google
Translate work?



38 / 5000

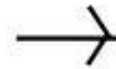


Envoyer des commentaires



Label Encoding

Food Name	Categorical #	Calories
Apple	1	95
Chicken	2	231
Broccoli	3	50



One Hot Encoding

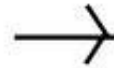
Apple	Chicken	Broccoli	Calories
1	0	0	95
0	1	0	231
0	0	1	50

this is a cat



Label Encoding

Food Name	Categorical #	Calories
Apple	1	95
Chicken	2	231
Broccoli	3	50



One Hot Encoding

Apple	Chicken	Broccoli	Calories
1	0	0	95
0	1	0	231
0	0	1	50

PB: I saw a saw.

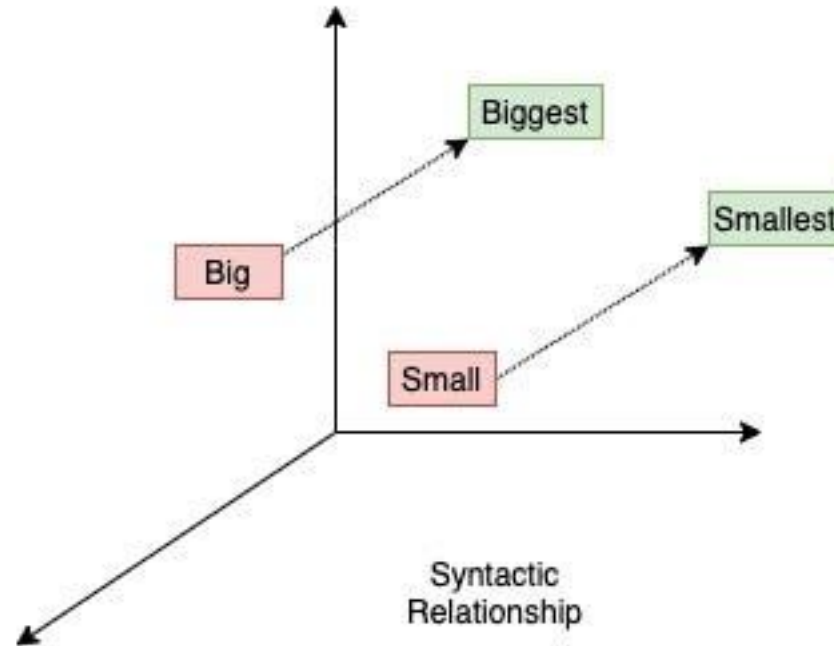
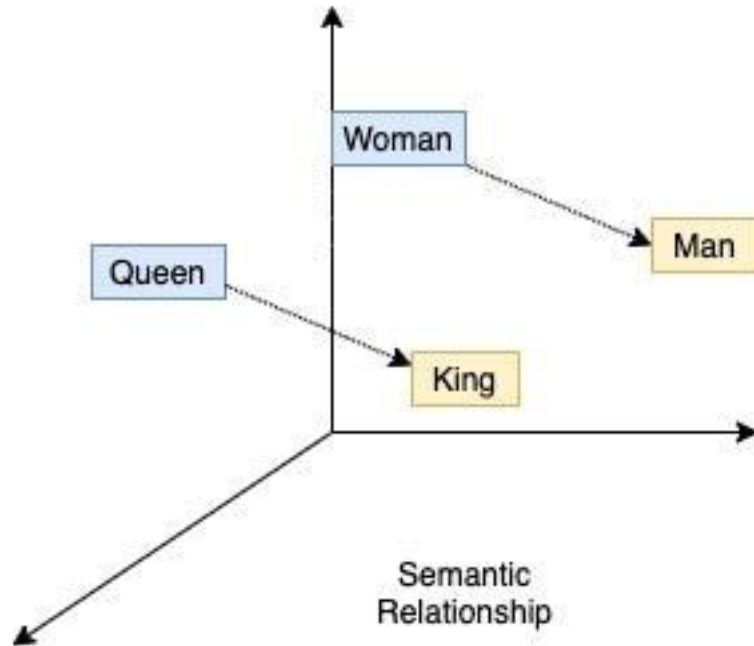
this is a cat



NLP



GNOMON[®]
DIGITAL

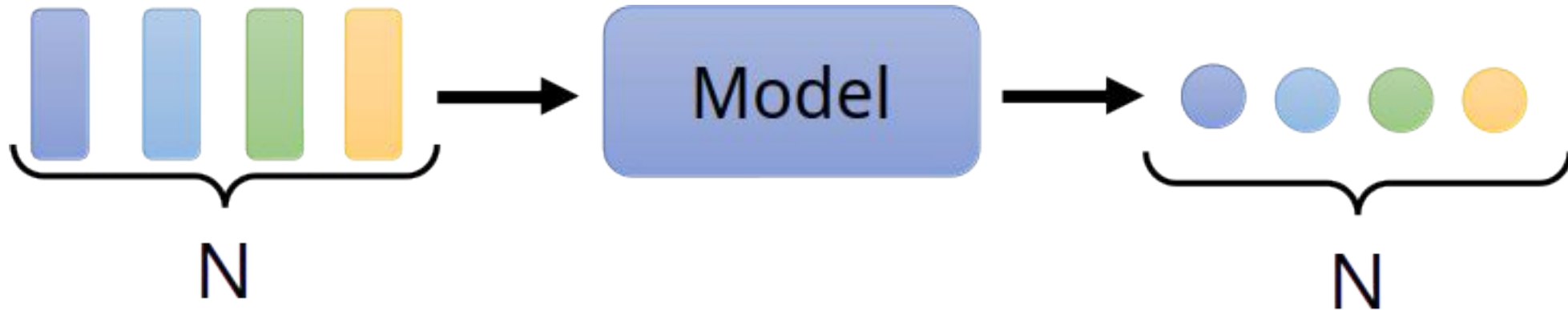


word2vec

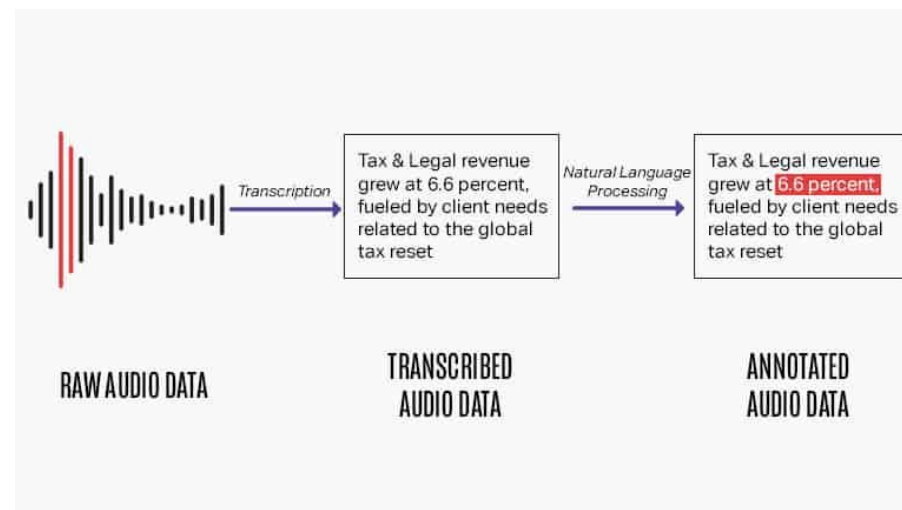
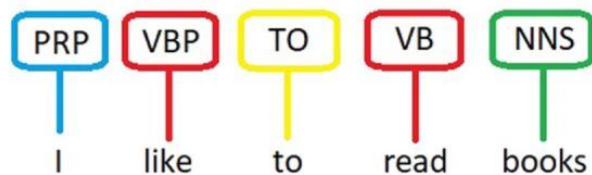
NLP



Chaque vecteur a un label

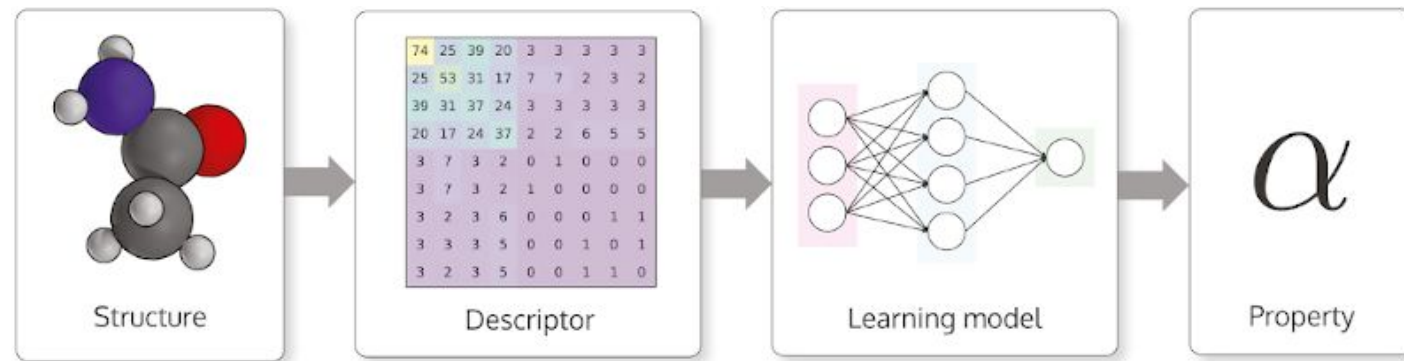
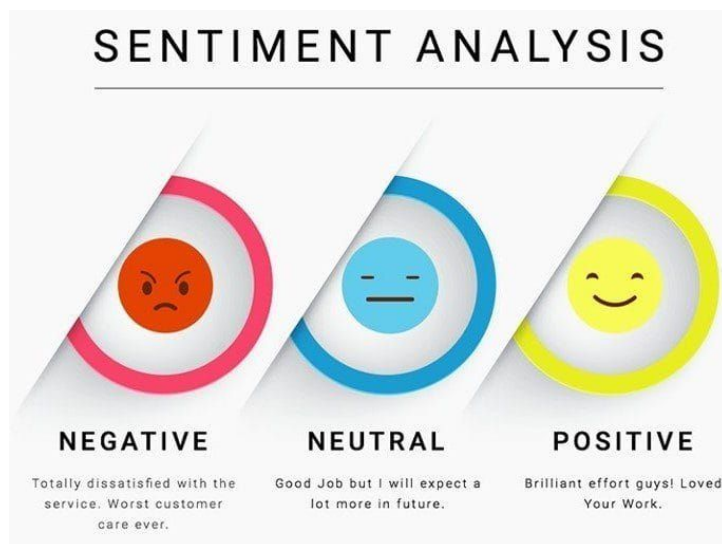


POS Tagging



NLP

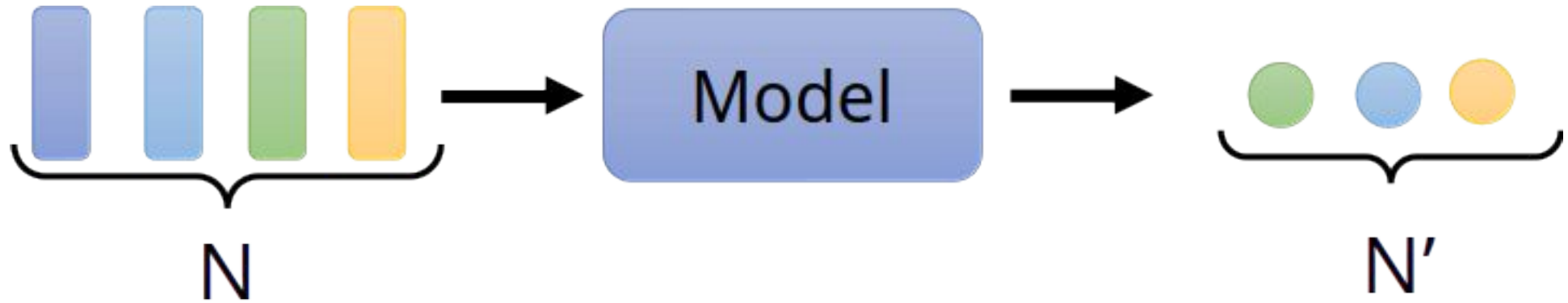
La séquence entière correspond un label.



NLP



Le modèle va déterminer lui même la sortie (seq2seq)



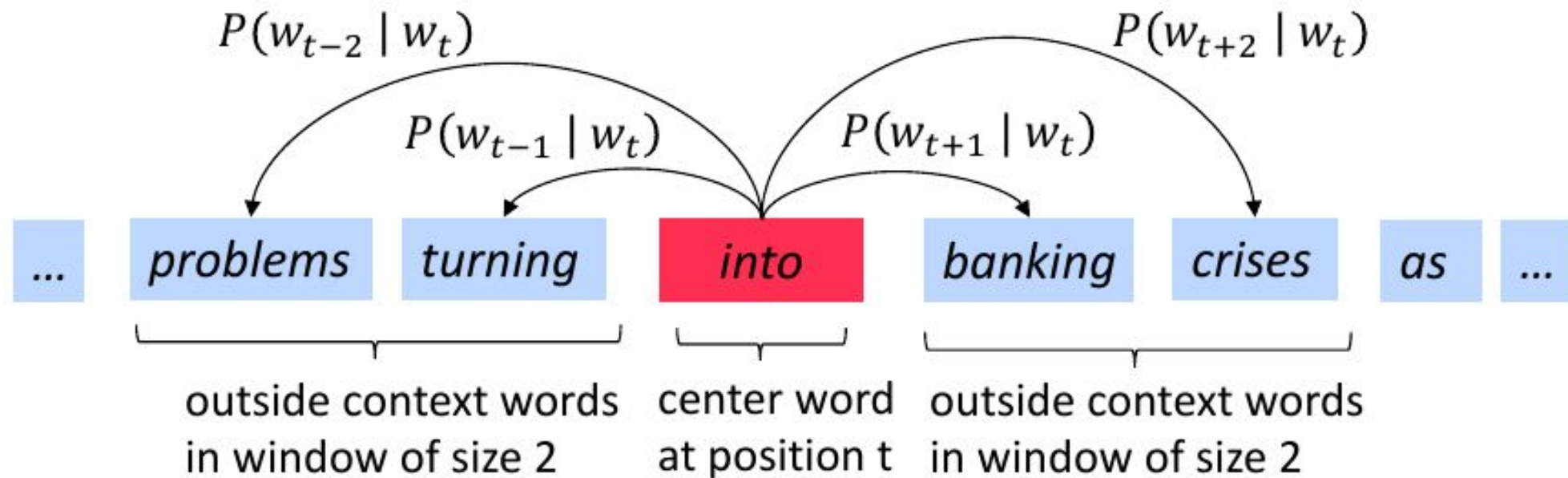
ChatGPT

NLP



GNOMON[®]
DIGITAL

I saw a saw
↓ ↓ ↓ ↓
N V DET N

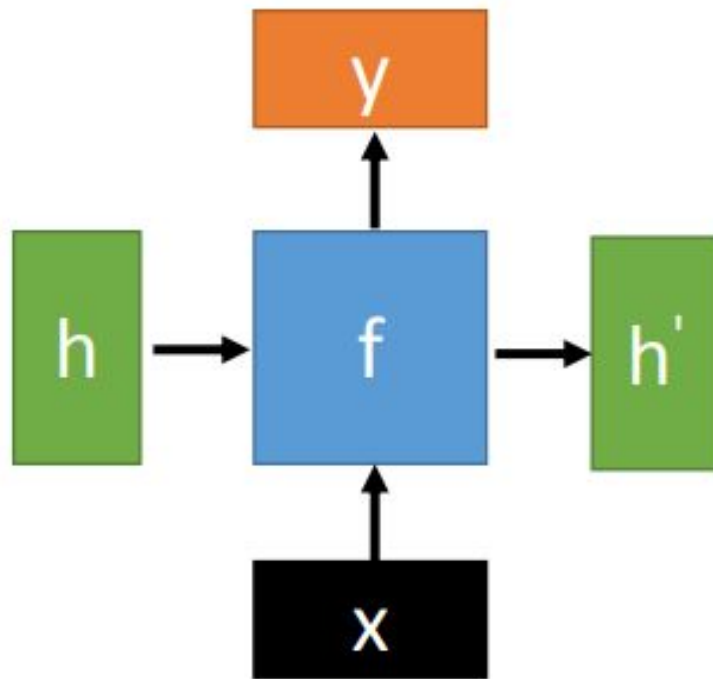


RNN: Recurrent Neural Networks



GNOMON[®]
DIGITAL

- Given function $f: h', y = f(h, x)$



$$h' = \sigma(W^h h + W^i x)$$

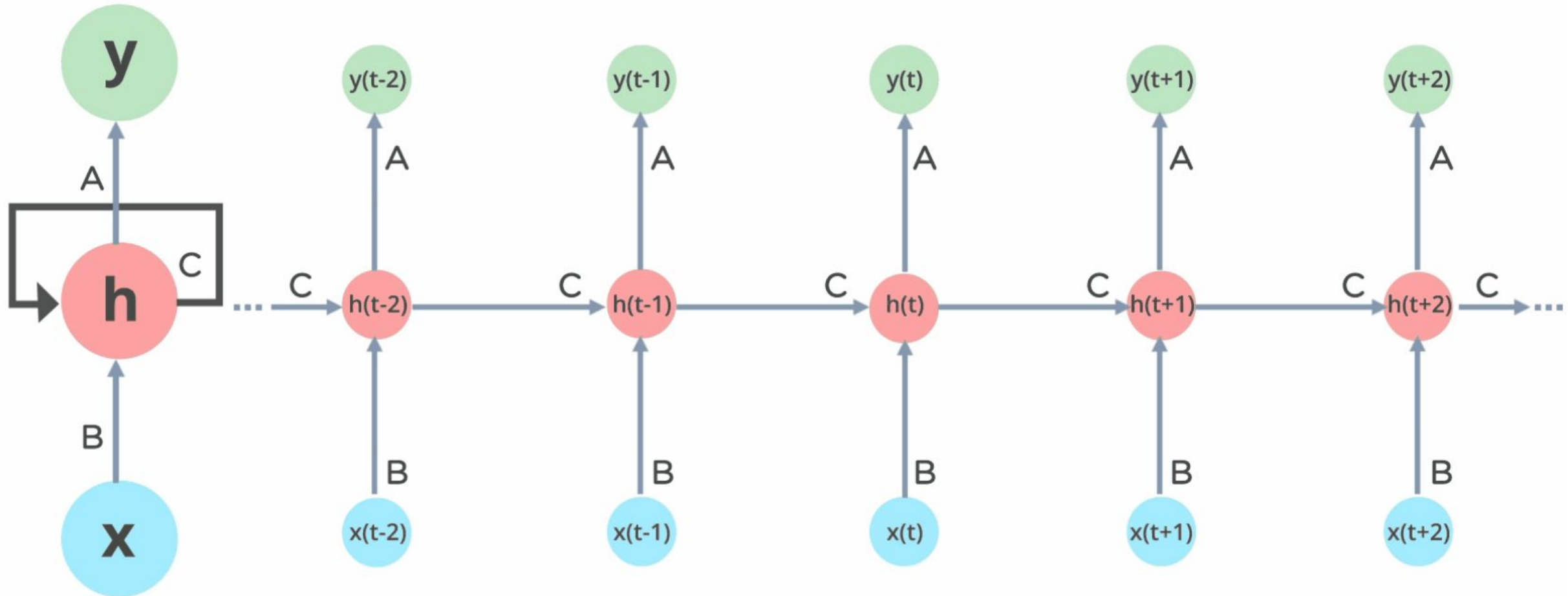
$$y = \sigma(W^o h')$$

softmax

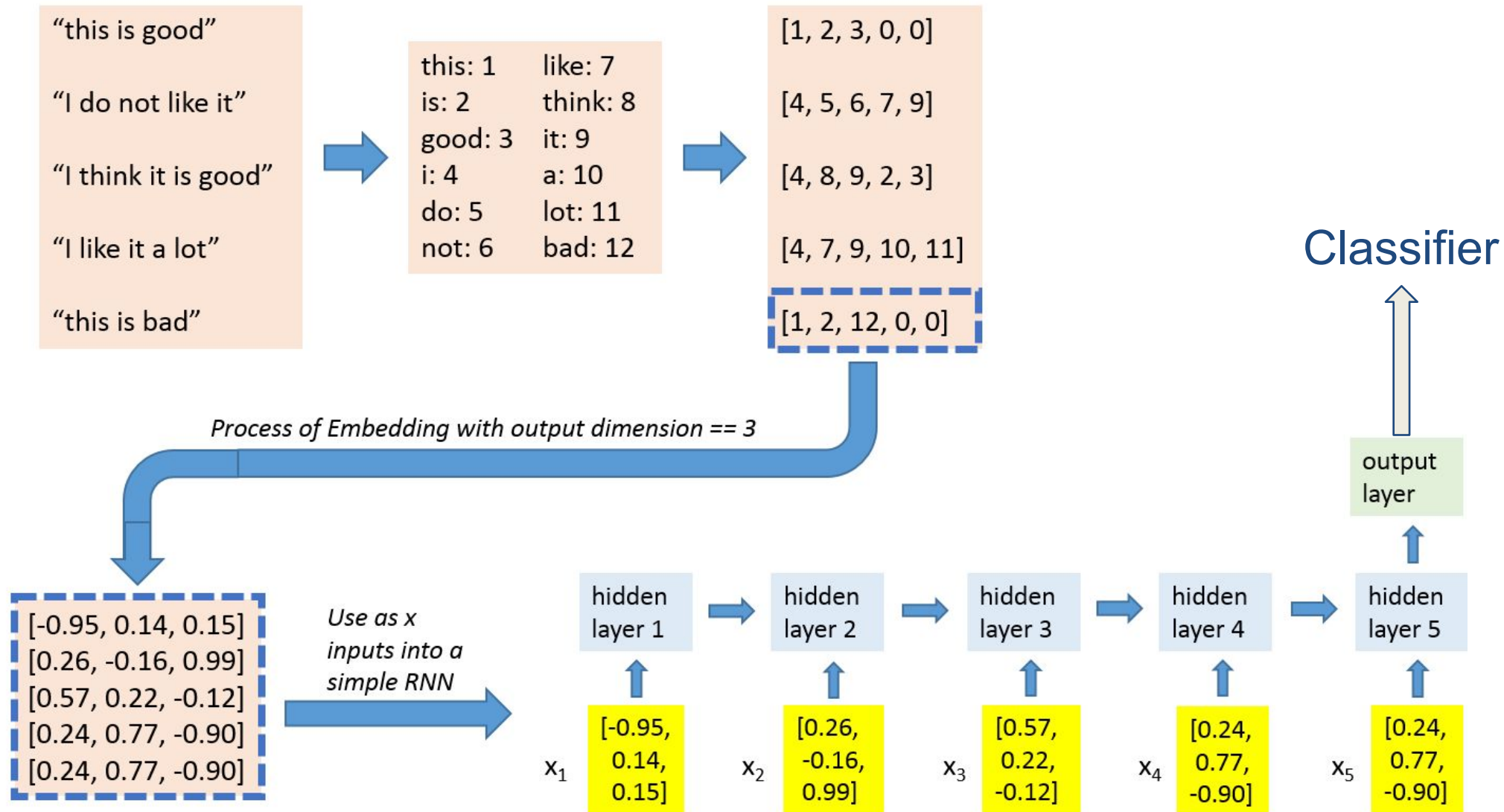
RNN: Recurrent Neural Networks



GNOMON[®]
DIGITAL



RNN: Recurrent Neural Networks

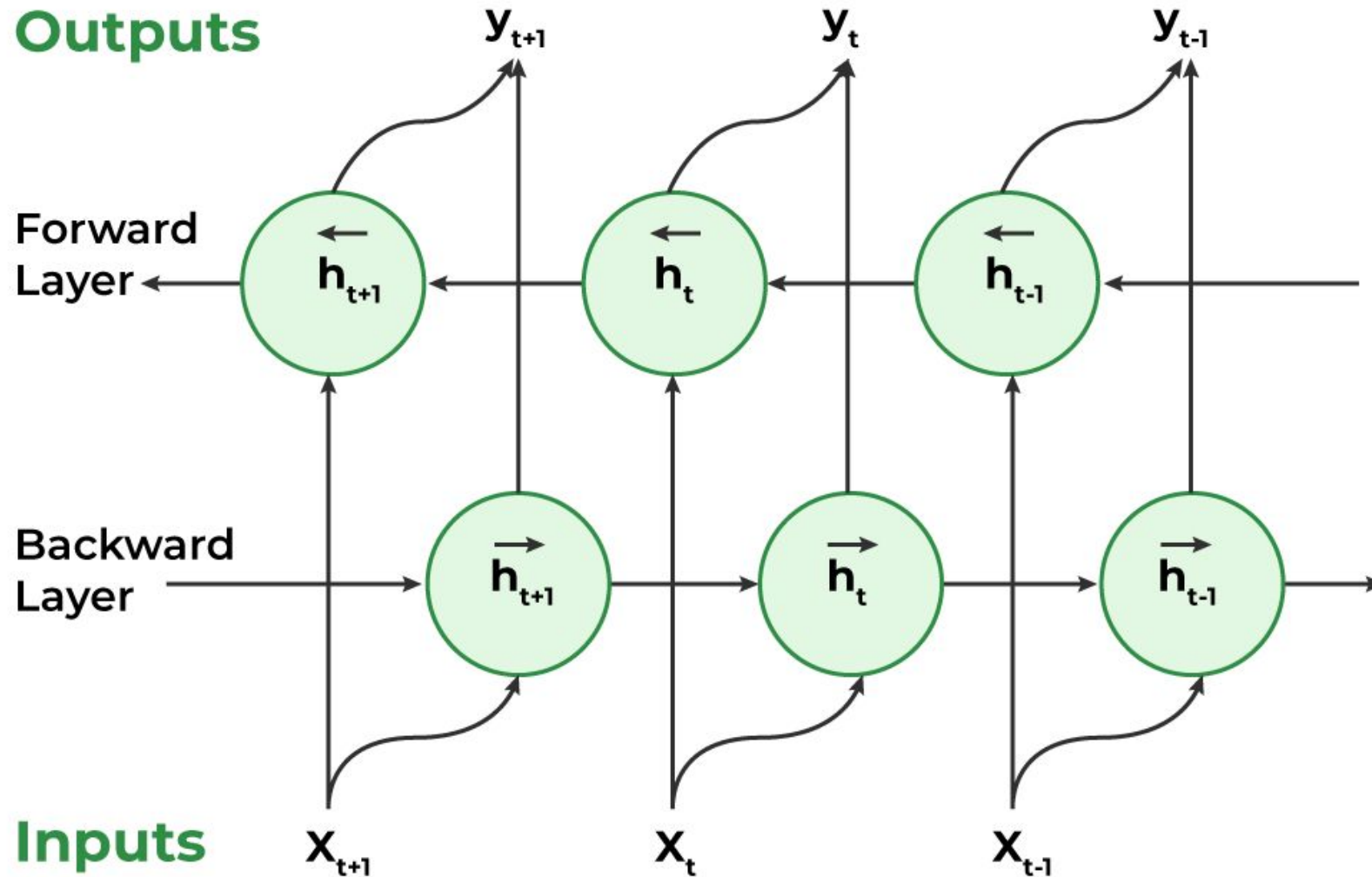


RNN: Recurrent Neural Networks



GNOMON[®]
DIGITAL

Outputs



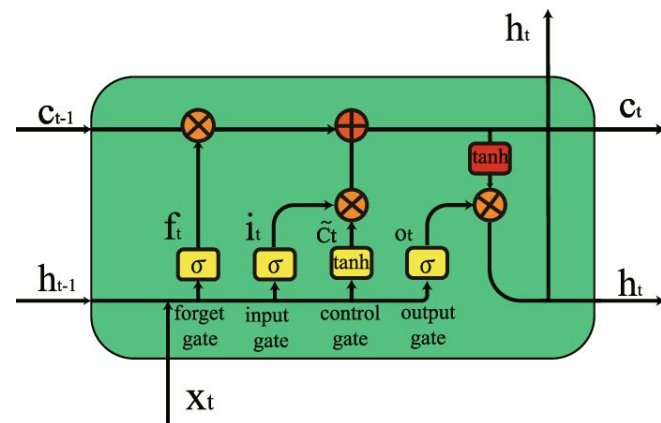
Bidirectional RNN

RNN: Recurrent Neural Networks

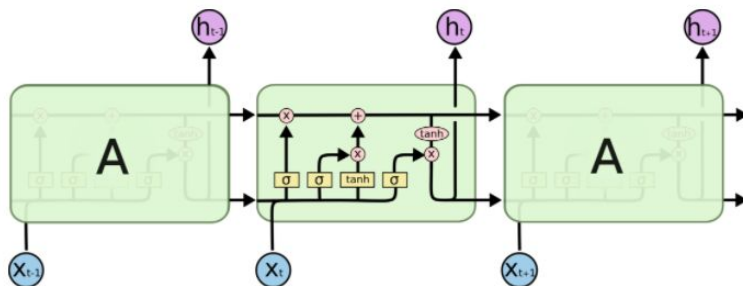


GNOMON[®]
DIGITAL

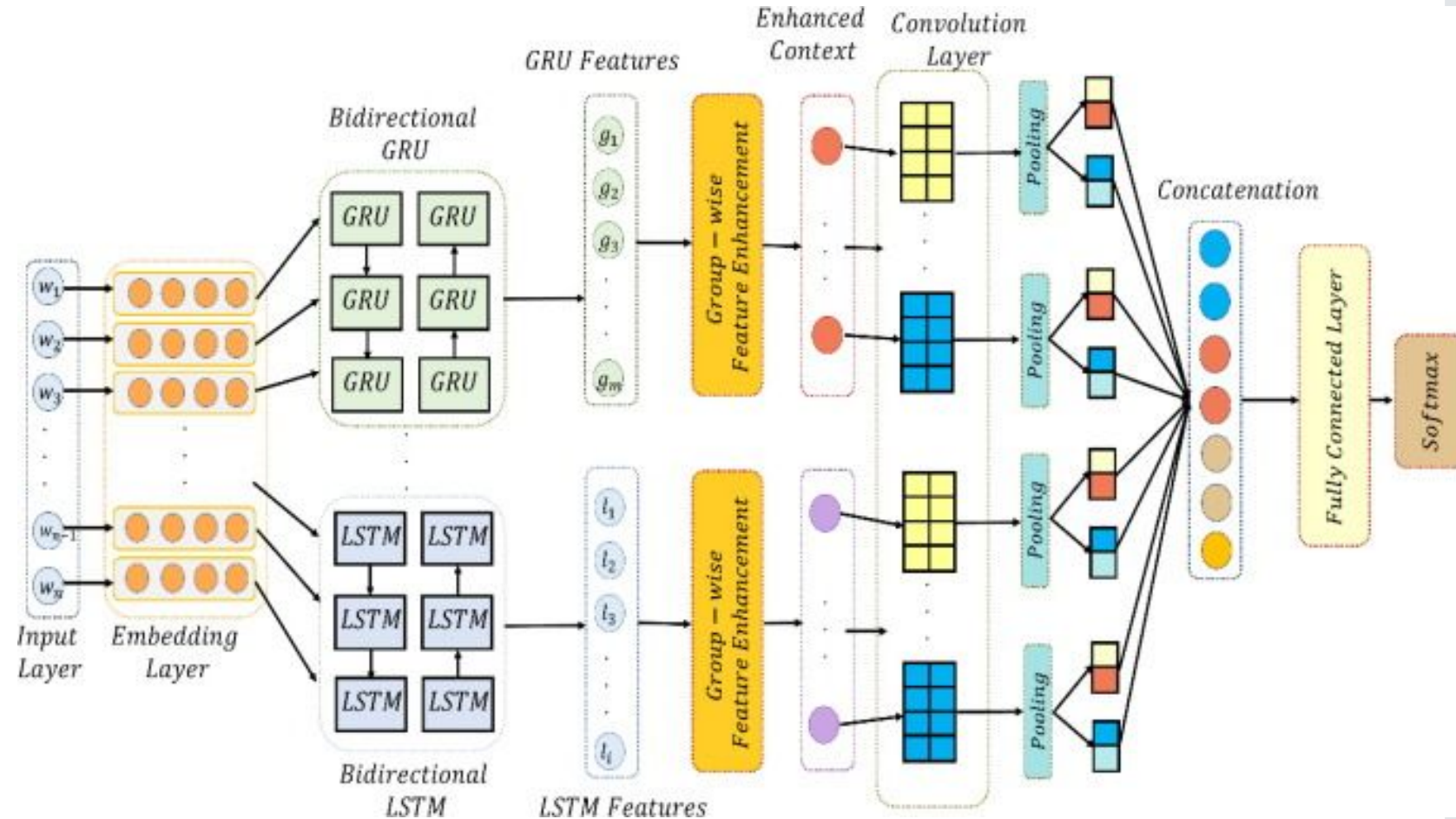
Keras peut gérer pour toi



- vector multiplication
- tanh neural networks
- tangent function
- vector addition
- sigmoid neural networks



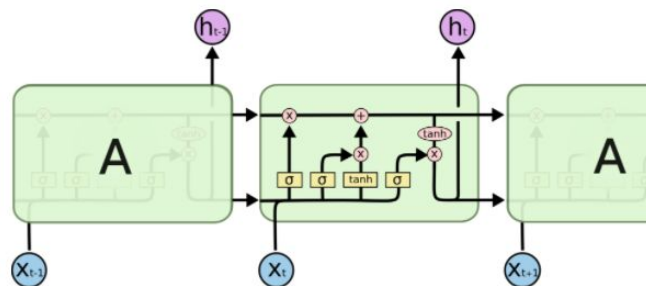
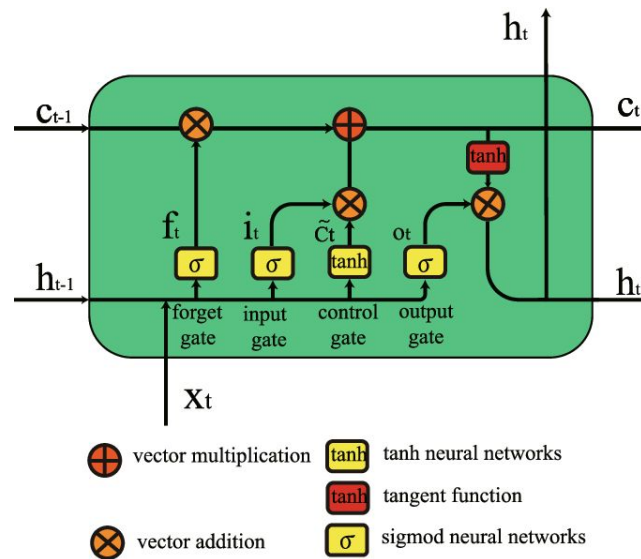
The repeating module in an LSTM contains four interacting layers.



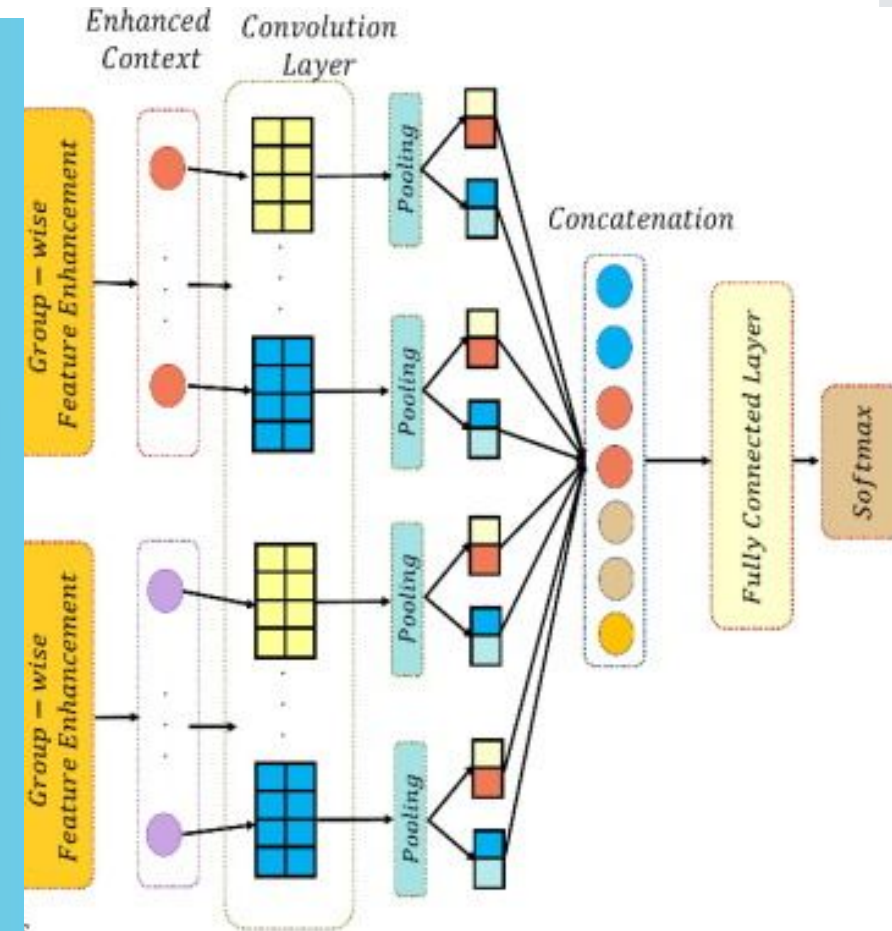
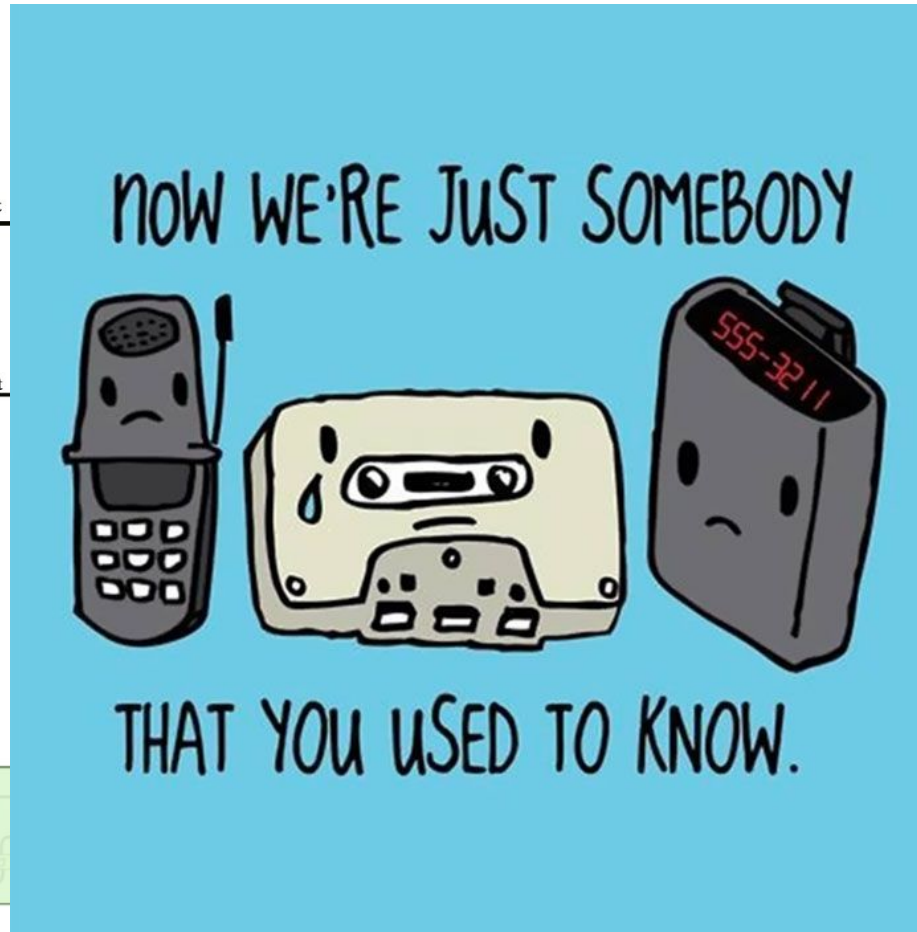
RNN: Recurrent Neural Networks



GNOMON[®]
DIGITAL

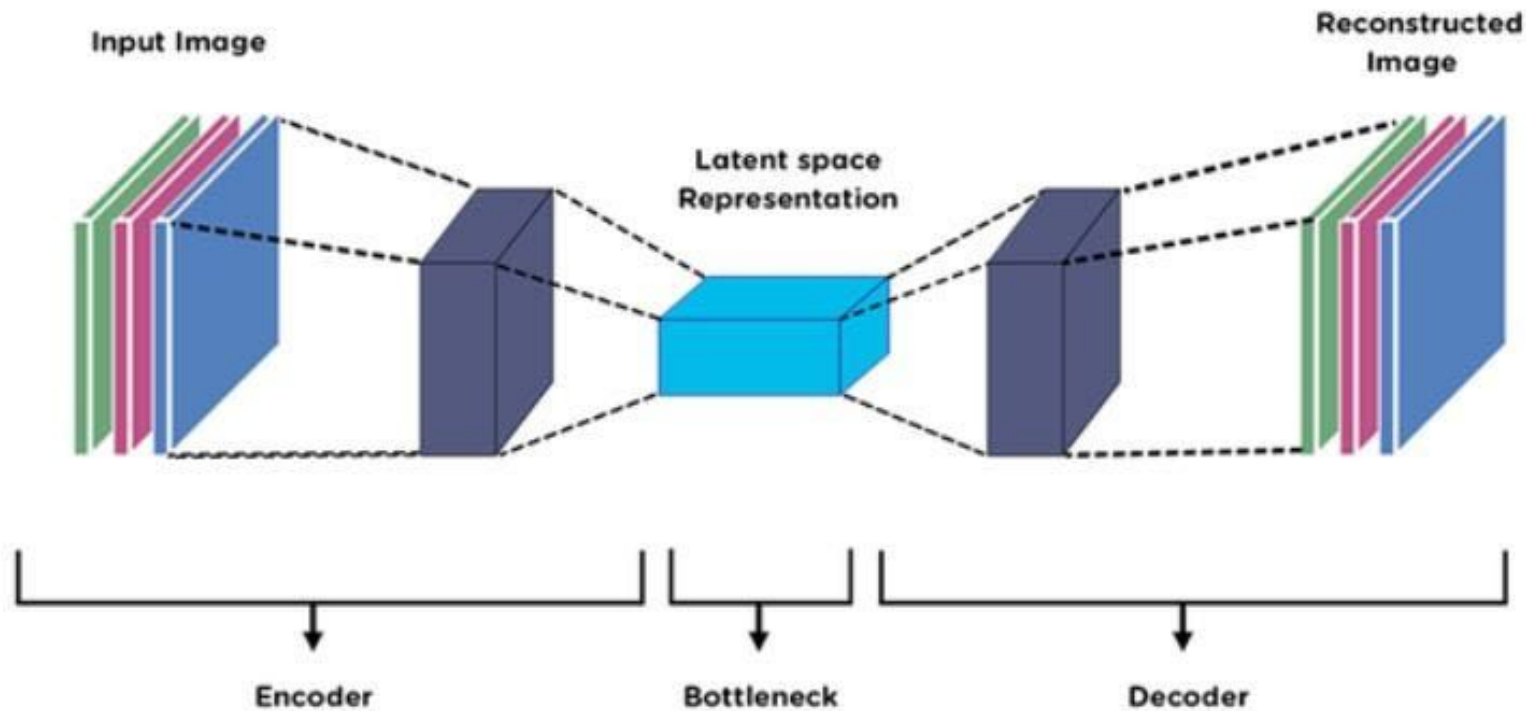


The repeating module in an LSTM contains four interacting layers.



Sequence-to-sequence (Encoder-Decoder Architecture)

Les entrées et les sorties sont des séquences avec des différentes longueurs (token).



Embedding

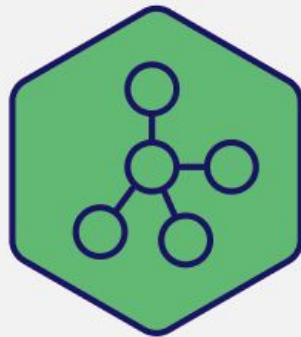


GNOMON[®]
DIGITAL

data object



embedding model



data as vector embedding

0.056	0.009	-0.07	...	0.023
-------	-------	-------	-----	-------

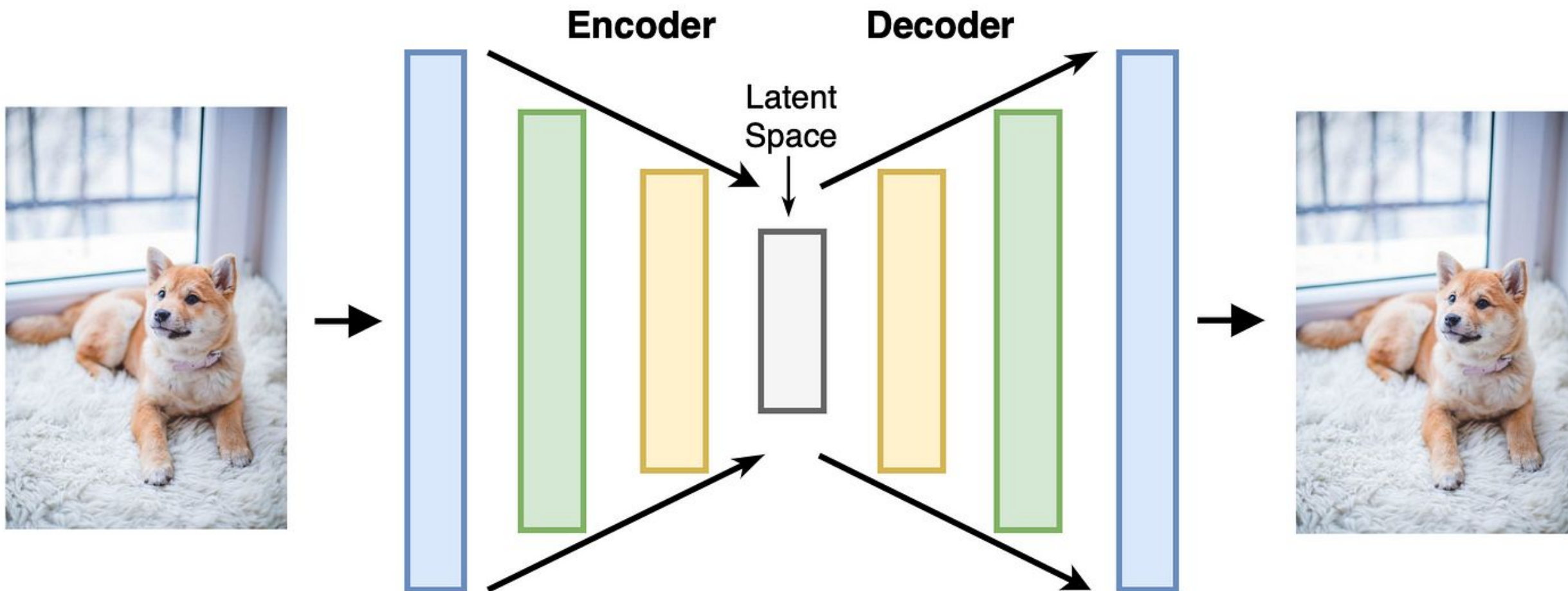
-0.06	0.039	0.137	...	0.045
-------	-------	-------	-----	-------

0.012	-0.01	0.079	...	-0.02
-------	-------	-------	-----	-------

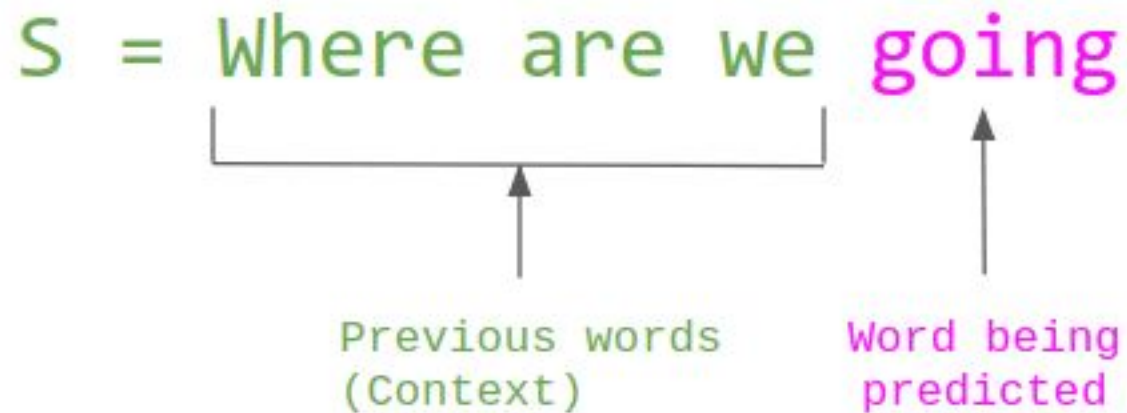
Embedding



GNOMON[®]
DIGITAL

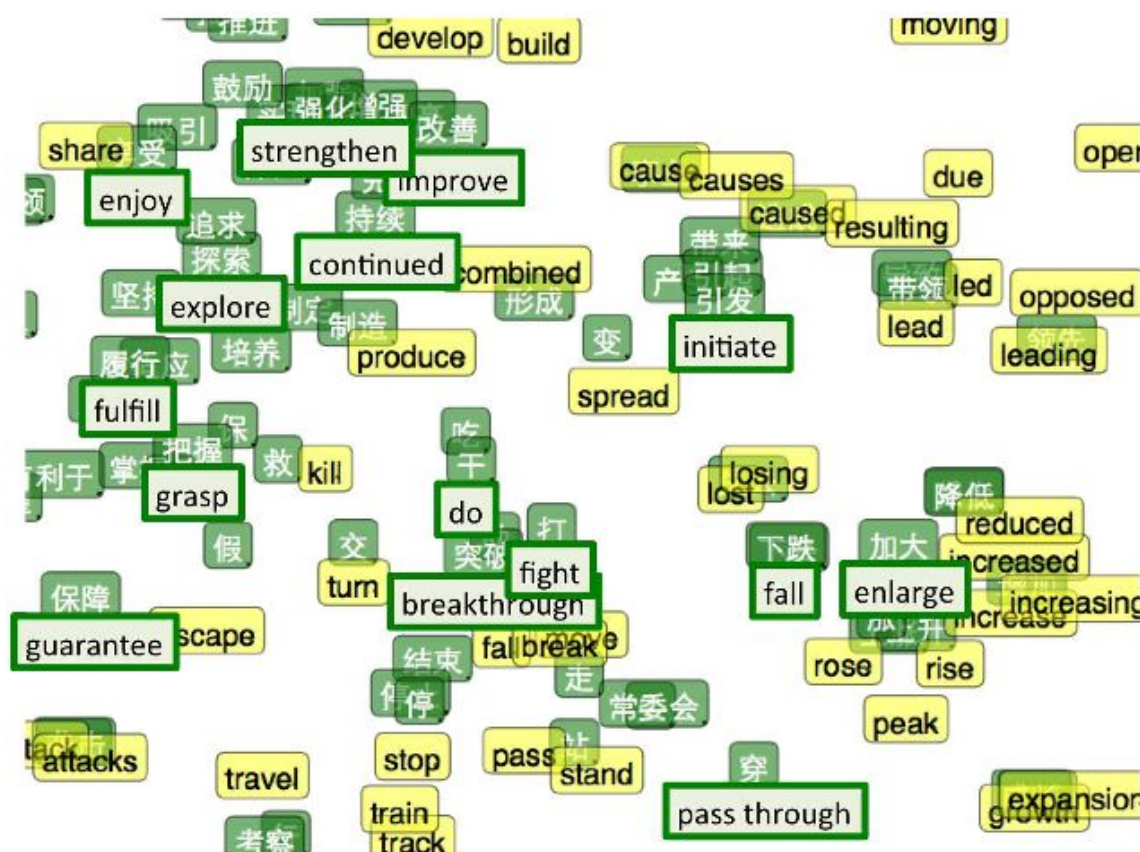


Language model



$$P(S) = P(\text{Where}) \times P(\text{are} \mid \text{Where}) \times P(\text{we} \mid \text{Where are}) \times P(\text{going} \mid \text{Where are we})$$

Multi-lingual Embedding

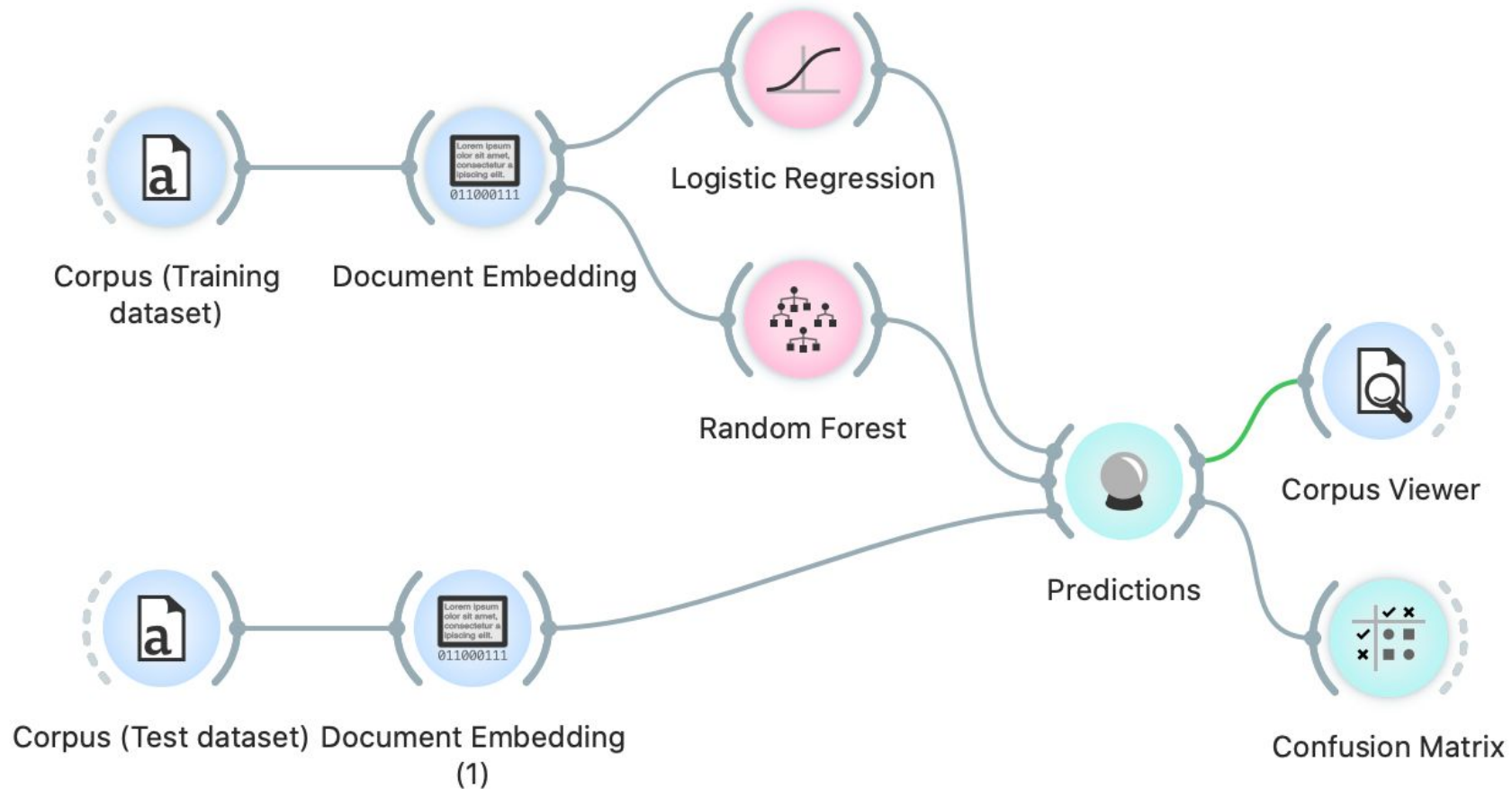


Embedding



GNOMON[®]
DIGITAL

Document Embedding



Embedding



GNOMON[®]
DIGITAL

Clustering



Bitext Mining



Retrieval



MTEB

Massive Text
Embedding Benchmark

8 Tasks

58 Datasets

STS



Summarization

SummEval

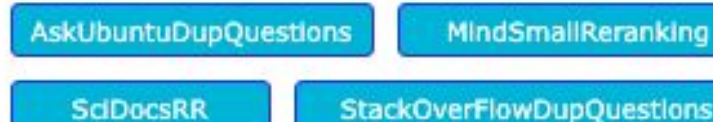
Classification



Pair Classification



Reranking

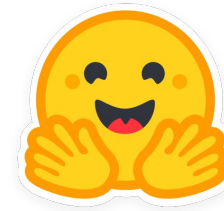
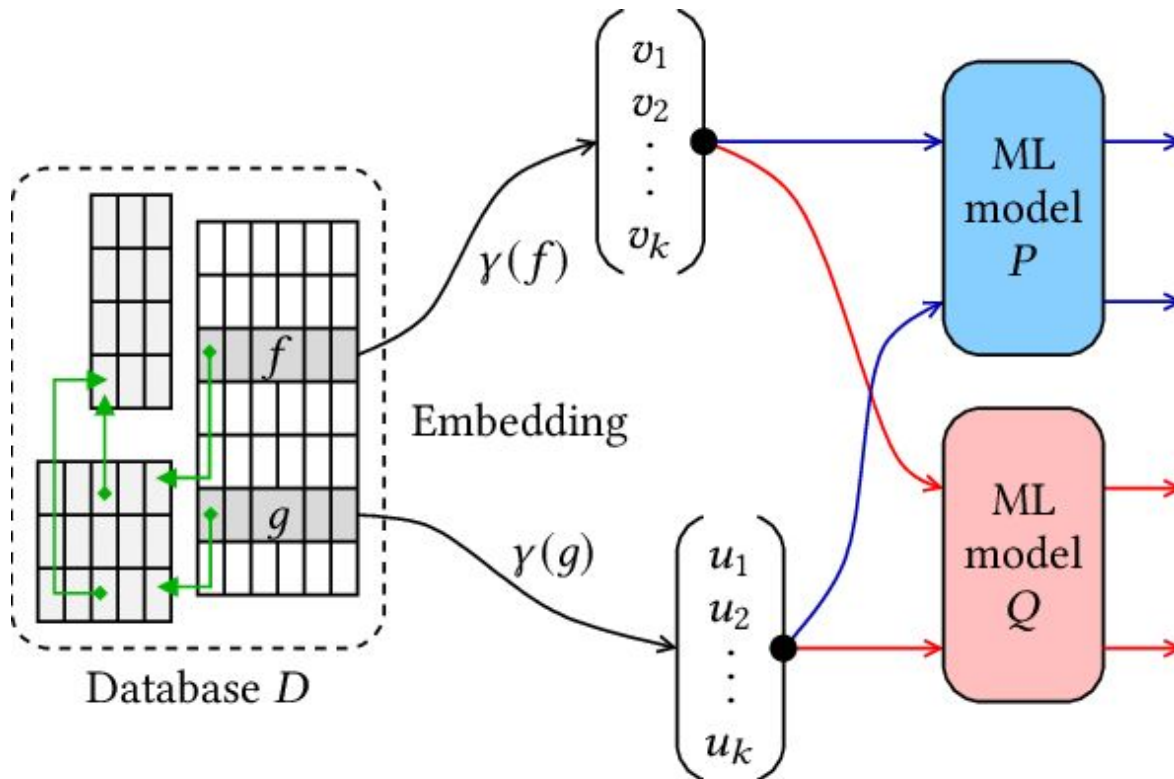




Hugging Face

Merci à Hugging face

Embedding



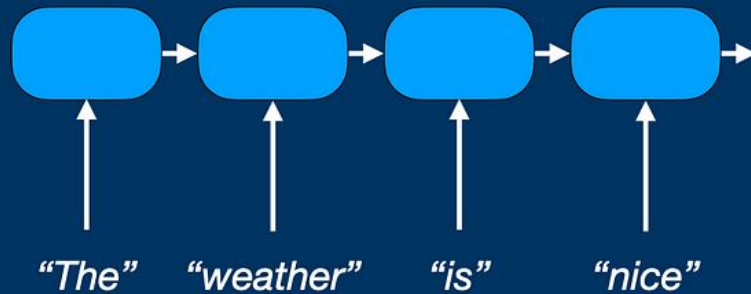
Hugging Face

Merci à Hugging face

<https://huggingface.co/blog/getting-started-with-embeddings>

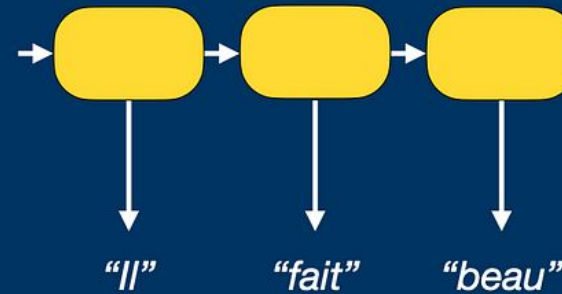
Seq2seq Architecture

LSTM Encoders



[context vector]

LSTM Decoders



English input sequence

French output sequence



Tokens

Characters

113

381

```
{"role": "assistant", "content": null, "function_call": {"name": "currentWeather", "arguments": "{ \"latitude\": \"37.773972\", \"longitude\": \"-122.431297\" }"}, {"role": "function", "name": "currentWeather", "content": "{ \"temperature\": \"73\", \"unit\": \"fahrenheit\", \"description\": \"Mostly sunny. High near 73, with temperatures falling to around 68 in the afternoon.\" }" }
```

Self Attention



Chat Generative Pre-trained Transformer -> Transformer

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

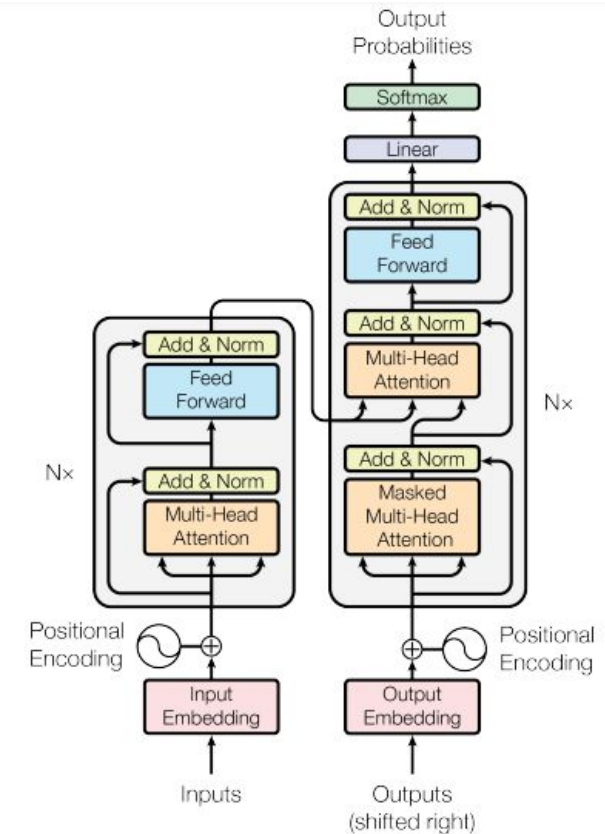
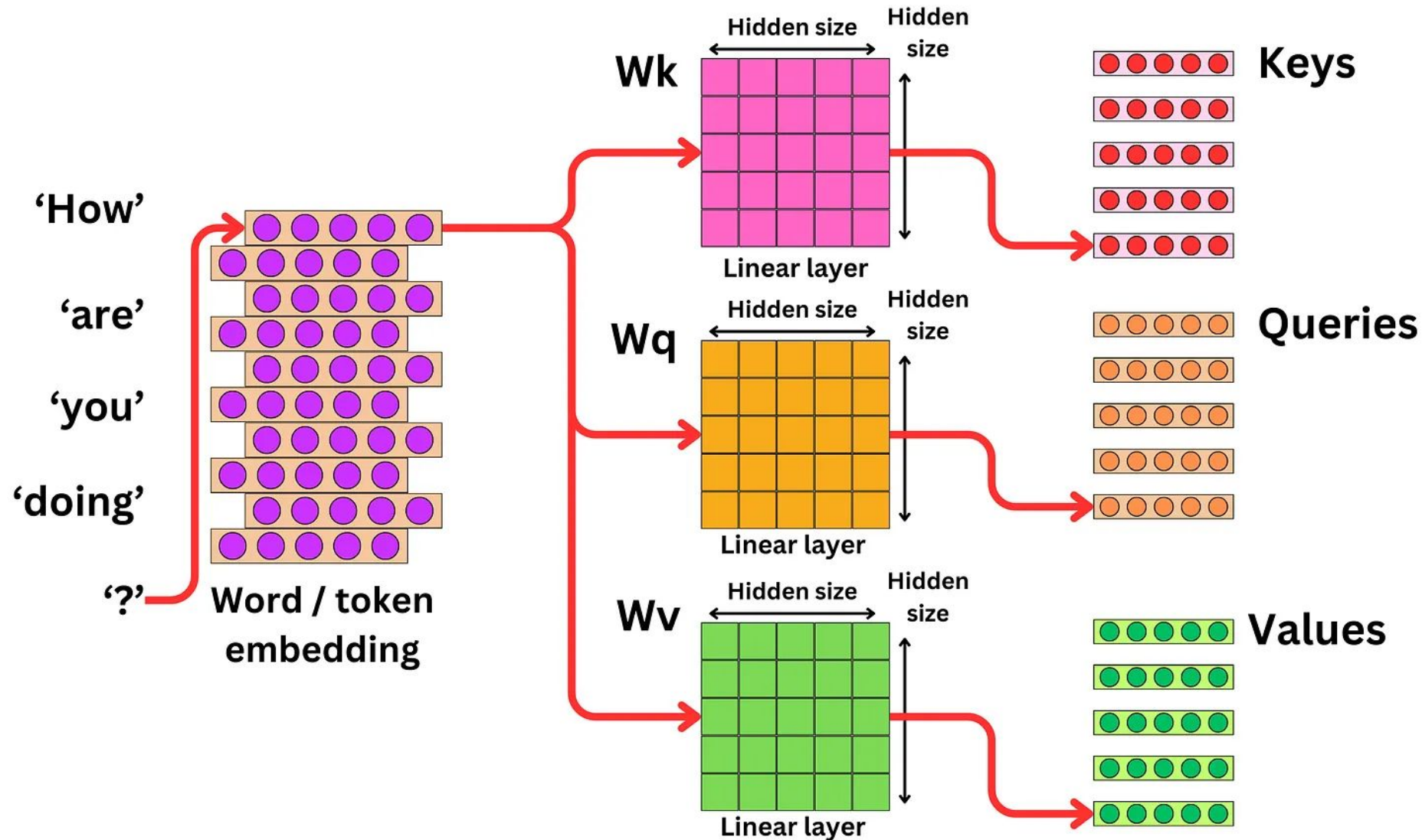


Figure 1: The Transformer - model architecture.

Self Attention



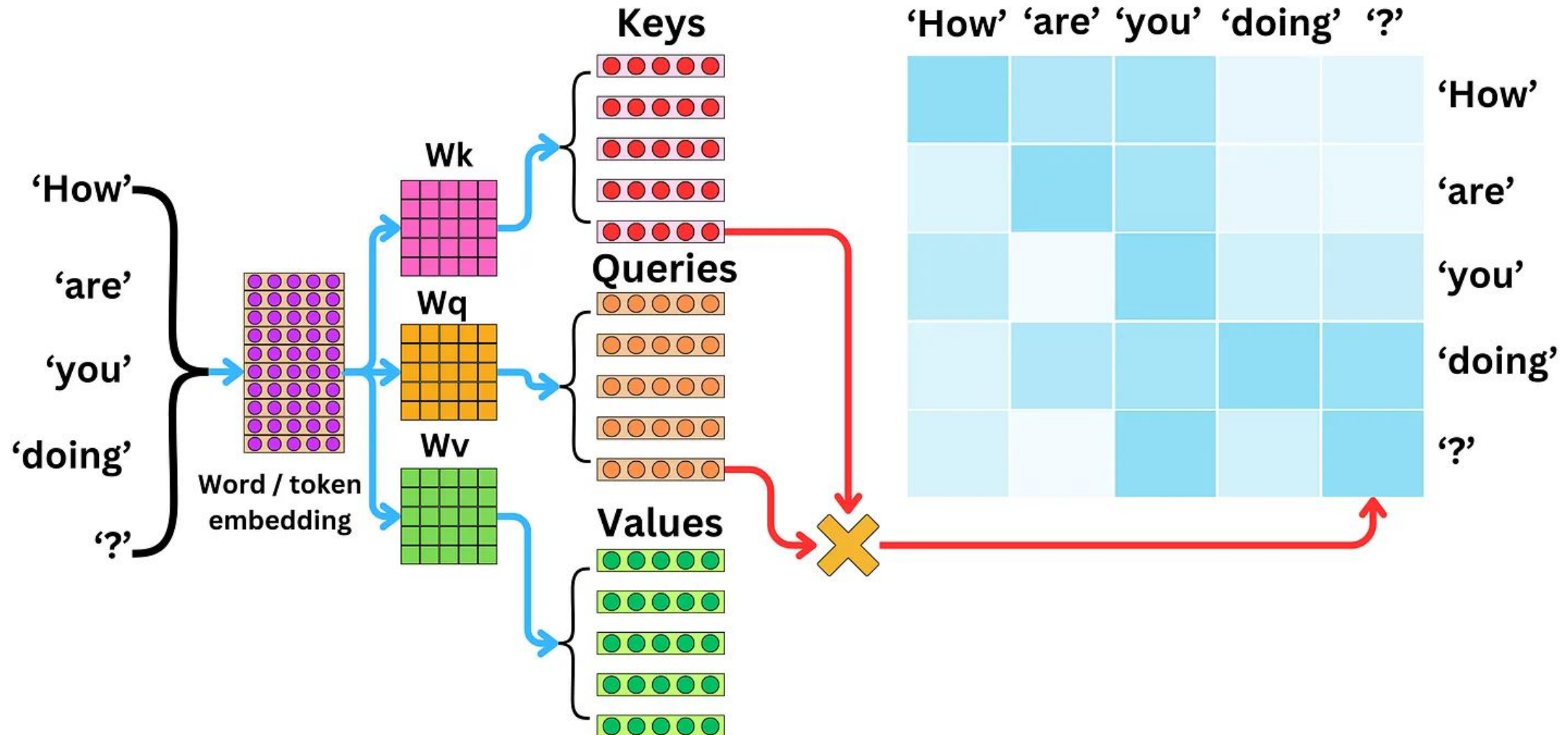
GNOMON[®]
DIGITAL



Self Attention



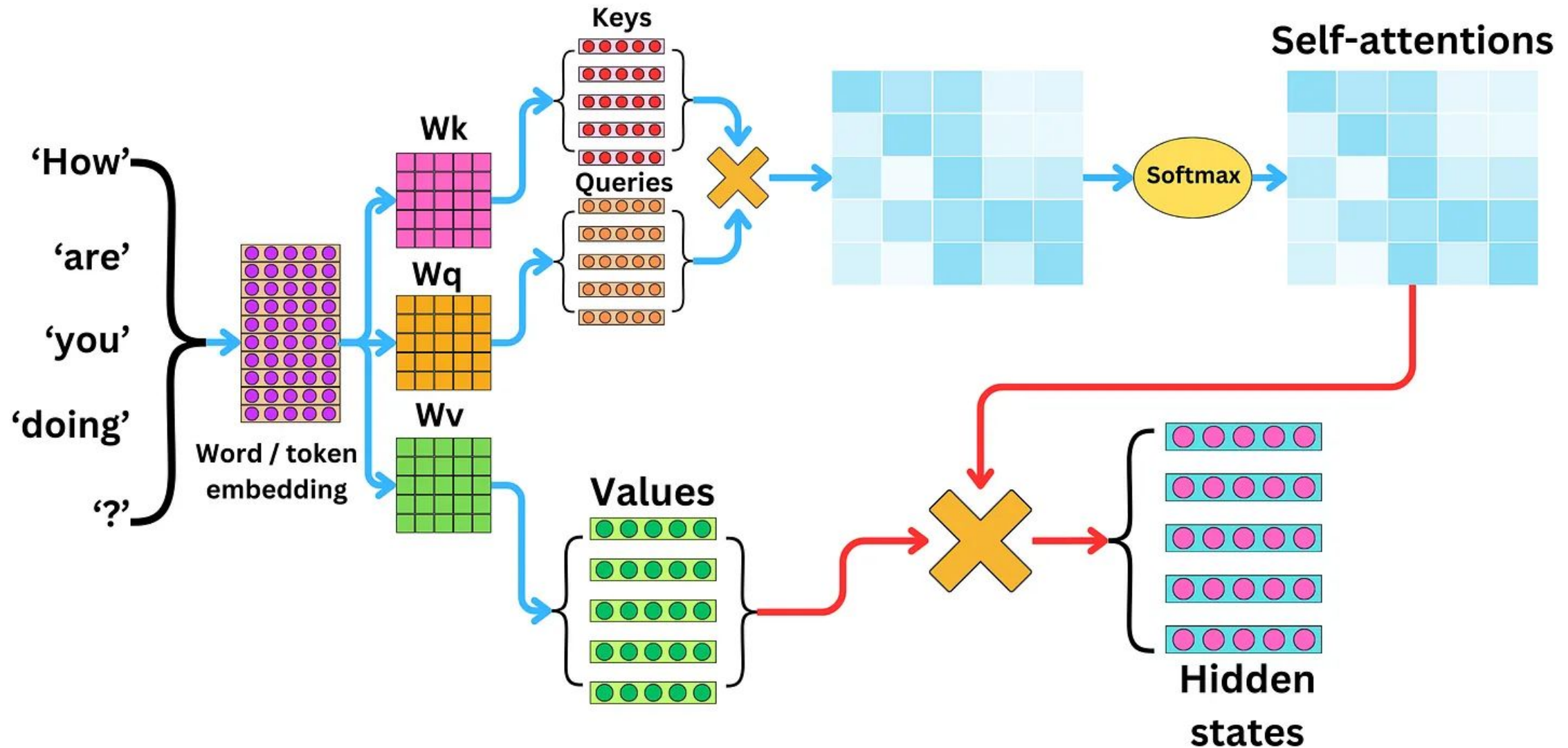
GNOMON[®]
DIGITAL



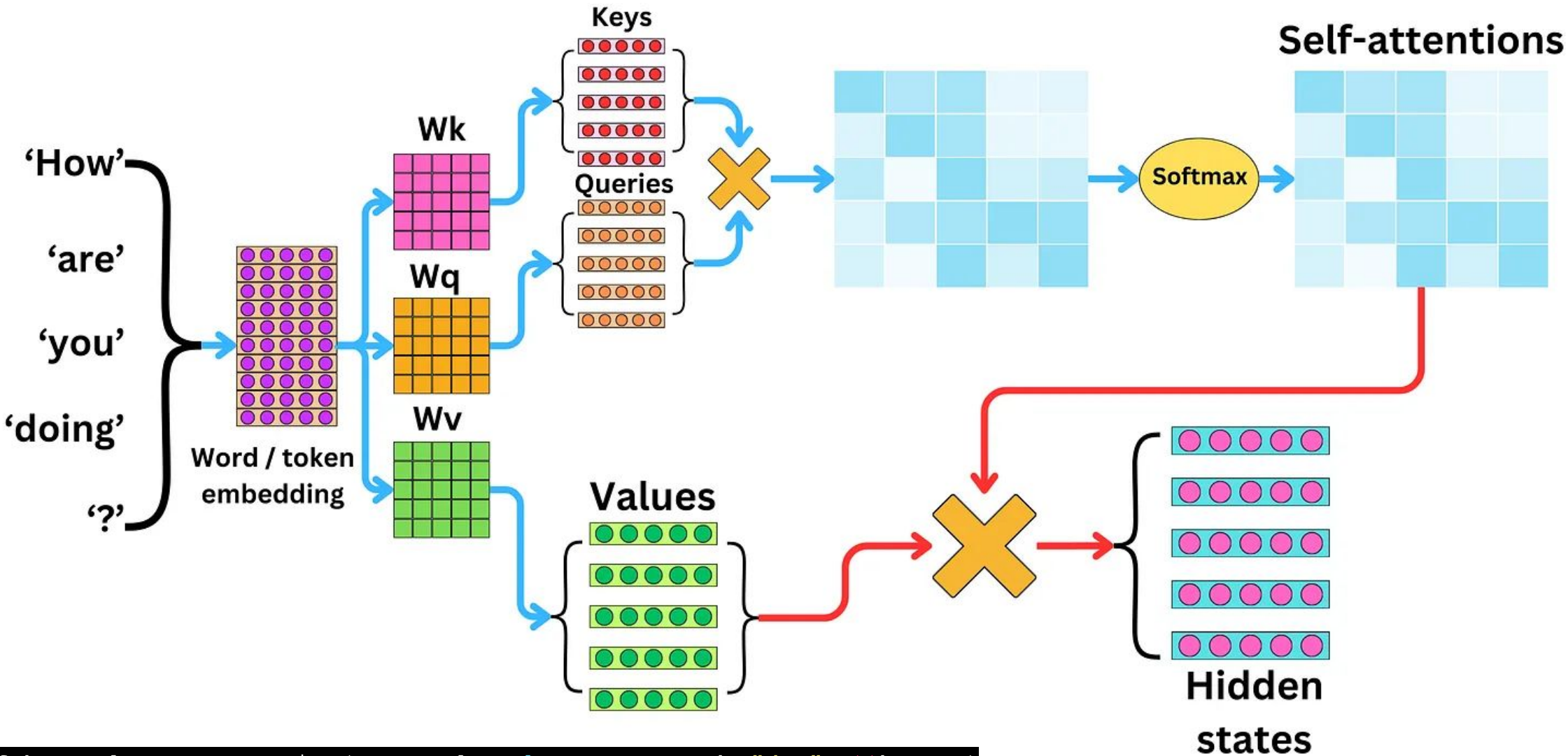
Self Attention



GNOMON[®]
DIGITAL



Self Attention



```
tf.keras.layers.Attention(use scale=False, score mode="dot", **kwargs)
```

Self Attention



GNOMON[®]
DIGITAL



Transformer

<https://arxiv.org/abs/1706.03762>



BERT

<https://arxiv.org/abs/1810.04805>

Self Attention



Chat Generative Pre-trained Transformer -> Transformer

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

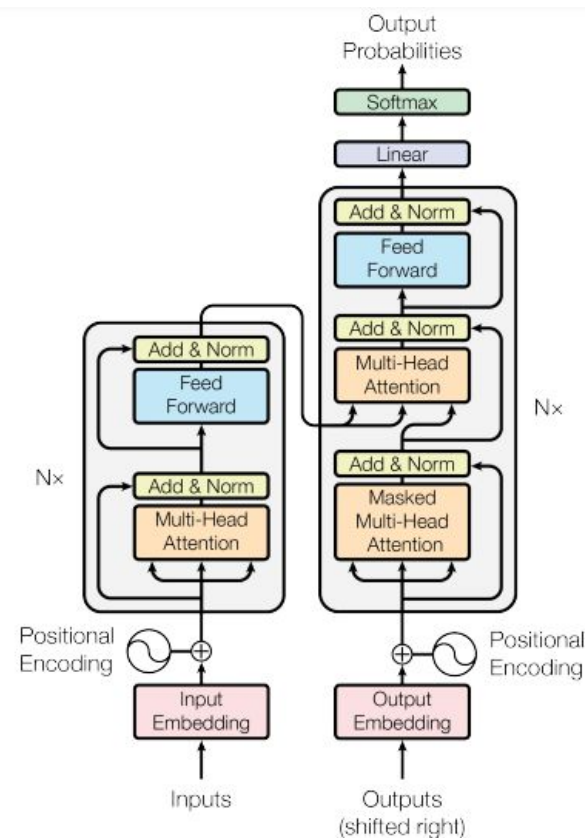


Figure 1: The Transformer - model architecture.